Elite 39-Day Interview Blueprint

Complete Parallel Prep System

DSA • Node.js • Java/Spring Boot • SQL • System Design

July 1 – August 8, 2026

MERN Stack Engineer → Full-Stack Backend Interview Ready
39 Days • 14 Daily Study Hours • 15 System Designs • 1,076 LeetCode Assignments
5 Mock Interview Days • Behavioral STAR • Resume & Job Search Playbook

Discipline beats talent when talent doesn't work hard.

Table of Contents

Section 1: Readiness Assessment

Baseline self-assessment across 22 skill areas (1–10 scale). Re-score weekly on Days 7, 14, 21, 28, 35, and 39 to track improvement.

Skill Area	Score	Visual
JavaScript/TypeScript	8/10	■■■■■■■■■■
Node.js/Express	8/10	■■■■■■■■■■
React/Frontend	8/10	■■■■■■■■■■
MongoDB	7/10	■■■■■■■■■■
REST API Design	8/10	■■■■■■■■■■
System Design	4/10	■■■■■■■■■■
DSA/Coding (LeetCode)	9/10	■■■■■■■■■■
SQL/Relational DBs	5/10	■■■■■■■■■■
Java Core	4/10	■■■■■■■■■■
Spring Boot	3/10	■■■■■■■■■■
OOP/Design Patterns	6/10	■■■■■■■■■■
Operating Systems	4/10	■■■■■■■■■■
Computer Networking	4/10	■■■■■■■■■■
Docker/Containers	5/10	■■■■■■■■■■
AWS/Cloud	5/10	■■■■■■■■■■
Redis/Caching	5/10	■■■■■■■■■■
Message Queues	4/10	■■■■■■■■■■
Microservices	5/10	■■■■■■■■■■
Testing (Unit/Integration)	6/10	■■■■■■■■■■
Behavioral/STAR Stories	6/10	■■■■■■■■■■
CI/CD	5/10	■■■■■■■■■■
Security (OWASP/API)	5/10	■■■■■■■■■■

Strengths

- 5 years production MERN stack experience with end-to-end feature ownership
- Strong Node.js/Express REST API development and MongoDB schema design
- Solid React frontend skills enabling full-stack debugging and system thinking
- DSA at 4.5/5 — strong pattern recognition; ready for high-volume Medium/Hard drills
- Practical deployment experience with real users and production incident handling
- Good communication skills from cross-functional MERN project delivery
- Fast learner — actively building Spring Boot projects to expand backend breadth

Weaknesses to Address

- Hard-problem speed under 25-minute interview timers still needs sharpening
- System design depth limited — few large-scale distributed architecture experiences
- Java/Spring Boot still early (3/10) — must frame as growth story with portfolio work
- SQL advanced patterns (windows, complex joins) rusty compared to NoSQL comfort
- OS and networking fundamentals need structured review for backend interviews

- Limited experience with Kafka, Kubernetes, and large-scale queue-based systems

Risk Factors

- Over-indexing on MERN strengths while Java rounds expose Spring Boot gaps
- 39-day timeline is aggressive for simultaneous DSA, SD, and Java ramp-up
- Burnout from 35+ daily LeetCode problems without scheduled rest on Sundays
- Applying too early before Spring Boot project is demo-ready for Java-heavy companies
- System design answers may sound theoretical without quantified tradeoff analysis
- Behavioral stories may lack metrics if not rehearsed with specific numbers

Top Priorities (39-Day Focus)

- High-volume LeetCode: 15 problems Day 1 → 35+ by July 31 (~87% Medium/Hard)
- Build one polished Spring Boot REST project with JPA, Security, tests, and README
- Complete 15 system designs with back-of-envelope math and explicit tradeoffs
- Node.js deep-dive on event loop, streams, auth, caching — make answers automatic
- SQL daily drills: window functions, joins, subqueries until speed matches NoSQL comfort
- Weekly full mocks with written post-mortem gap analysis and next-week adjustment
- Tailor resume bullets per stack (Node vs Java) and apply strategically by company type

Section 2: Daily Schedule — 14 Study Hours

Fixed daily rhythm from **05:30 wake** to **22:30 sleep**. Study blocks total **14 hours** including 5.5h LeetCode across 3 blocks. Volume ramps from 15 to 35+ problems/day by July 31. Mandatory breaks prevent burnout. Adjust weekend blocks slightly for mock interview days (see Section 14).

Time	Activity	Study Hours
05:30 – 06:00	Wake, hydrate, stretch, review today's LeetCode targets	Prep
06:00 – 08:30	LeetCode Block 1 — new M/H problems, strict 20-min timers	2.5h
08:30 – 09:00	Breakfast and mental reset	Break
09:00 – 11:00	Backend deep-dive — Node.js or Java/Spring (alternating focus)	2.0h
11:00 – 11:15	Short break — walk, no screens	Break
11:15 – 12:45	LeetCode Block 2 — continue new problems + pattern notes	1.5h
12:45 – 13:30	Lunch and light walk	Break
13:30 – 15:00	SQL drills + hands-on exercises	1.5h
15:00 – 15:15	Short break	Break
15:15 – 16:45	System Design / OS / Networking study block	1.5h
16:45 – 17:00	Short break	Break
17:00 – 18:30	LeetCode Block 3 — revision + Hard problems from weak patterns	1.5h
18:30 – 19:15	Dinner and decompress	Break
19:15 – 20:15	Behavioral STAR practice + resume tailoring	1.0h
20:15 – 21:30	Job applications, mock interviews, or project work	1.25h
21:30 – 22:00	Spaced repetition review + update progress tracker	0.5h
22:00 – 22:30	Wind down — no study screens	Break
22:30	Sleep — target 7 hours for cognitive recovery	Sleep

Weekly Rhythm

Day Type	Adjustment
Mon–Fri	Full 14h schedule; 15→35+ LeetCode; 2–3 job applications daily
Saturday	Mock interview or deep system design day; lighter LeetCode
Sunday	Review week gaps; prep next week; 1 hour rest from new material

Phase Overview

Days	Phase	Focus
Days 1–10	Foundation	Core patterns, Node/Java basics, first system designs
Days 11–20	Build	Advanced DSA, Spring depth, SQL speed drills
Days 21–30	Intensify	Hard problems, full system designs, weekly mocks
Days 31–39	Final Sprint	Revision, mocks, applications, interview readiness

Section 3: LeetCode Roadmap — 39 Days

597-problem database calibrated for DSA 4.5/5. Volume ramps **15** → **35+ problems/day** by July 31 (~13% Easy, ~50% Medium, ~37% Hard). Pattern progression: Arrays, Strings, HashMap, Sorting, Two Pointers, Sliding Window, Binary Search, Stack… and more. Use spaced repetition dates in the Revision column.

Day 1 — July 1, 2026 | Phase: Foundation | 15 problems | Focus: Map the Node event loop cold — this is your home turf, make it interview-crisp.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
448	Find Disappeared Numbers	E	Array	Index marking	20	442	D1, D3, D7, D14, D21, D30	M
13	Roman to Integer	E	String	Pair parsing	15	12	D1, D3, D7, D14, D21, D30	M
36	Valid Sudoku	M	HashSet	Row/col/box sets	25	37	D1, D3, D7, D14, D21, D30	H
56	Merge Intervals	M	Sorting	Interval merge	25	57	D1, D3, D7, D14, D21, D30	VH
11	Container With Most Water	M	Two Pointers	Greedy shrink	20	42	D1, D3, D7, D14, D21, D30	VH
3	Longest Substring Without Repeating Characters	M	Sliding Window	Unique char window	25	159	D1, D3, D7, D14, D21, D30	VH
33	Search in Rotated Sorted Array	M	Binary Search	Pivot detection	25	81	D1, D3, D7, D14, D21, D30	VH
71	Simplify Path	M	Stack	Path normalization	20	227	D1, D3, D7, D14, D21, D30	M
622	Design Circular Queue	M	Queue	Ring buffer	25	641	D1, D3, D7, D14, D21, D30	M
25	Reverse Nodes in k-Group	H	Linked List	k-group reverse	35	206	D1, D3, D7, D14, D21, D30	VH
124	Binary Tree Maximum Path Sum	H	Tree	Global max path	35	543	D1, D3, D7, D14, D21, D30	VH
23	Merge k Sorted Lists	H	Heap / divide	k-way merge	35	21	D1, D3, D7, D14, D21, D30	VH
317	Shortest Distance from All Buildings	H	Graph BFS	Multi-source sum	40	286	D1, D3, D7, D14, D21, D30	M
269	Alien Dictionary	H	Topological Sort	Custom order graph	40	210	D1, D3, D7, D14, D21, D30	VH
685	Redundant Connection II	H	Union Find	Directed cycle	35	684	D1, D3, D7, D14, D21, D30	M

Day 2 — July 2, 2026 | Phase: Foundation | 16 problems | Focus: Solidify module system knowledge and sketch URL Shortener end-to-end.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
860	Lemonade Change	E	Greedy	Change making	15	406	D2, D4, D8, D15, D22, D31	M
70	Climbing Stairs	E	DP	Fibonacci pattern	12	746	D2, D4, D8, D15, D22, D31	VH
743	Network Delay Time	M	Dijkstra	Shortest path	30	787	D2, D4, D8, D15, D22, D31	VH

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
146	LRU Cache	M	HashMap + DLL	Ordered eviction	35	460	D2, D4, D8, D15, D22, D31	VH
53	Maximum Subarray	M	Kadane	DP / greedy subarray	20	152	D2, D4, D8, D15, D22, D31	VH
5	Longest Palindromic Substring	M	Expand center	Palindrome expansion	30	647	D2, D4, D8, D15, D22, D31	VH
49	Group Anagrams	M	HashMap	Sorted key grouping	20	242	D2, D4, D8, D15, D22, D31	VH
75	Sort Colors	M	Dutch Flag	Three-way partition	20	215	D2, D4, D8, D15, D22, D31	VH
15	3Sum	M	Two Pointers	Sorted triplets	30	18	D2, D4, D8, D15, D22, D31	VH
159	Longest Substring with At Most Two Distinct Characters	M	Sliding Window	Distinct limit	25	340	D2, D4, D8, D15, D22, D31	H
4	Median of Two Sorted Arrays	H	Binary Search	Partition merge	45	295	D2, D4, D8, D15, D22, D31	VH
32	Longest Valid Parentheses	H	Stack	Valid paren length	35	20	D2, D4, D8, D15, D22, D31	VH
127	Word Ladder	H	BFS	Shortest transform	35	126	D2, D4, D8, D15, D22, D31	VH
295	Find Median from Data Stream	H	Heap	Two heap balance	35	480	D2, D4, D8, D15, D22, D31	VH
773	Sliding Puzzle	H	Graph BFS	Board BFS	40	127	D2, D4, D8, D15, D22, D31	M
1203	Sort Items Groups	H	Topological Sort	Grouped topo	40	269	D2, D4, D8, D15, D22, D31	M

Day 3 — July 3, 2026 | Phase: Foundation | 16 problems | Focus: Master streams — frequent in Node interviews — and Mongo document fundamentals.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
665	Non-decreasing Array	E	Greedy	One fix allowed	15	581	D3, D5, D9, D16, D23, D32	M
746	Min Cost Climbing Stairs	E	DP	Stair cost	15	70	D3, D5, D9, D16, D23, D32	H
787	Cheapest Flights K Stops	M	Bellman-Ford	K-stop path	35	743	D3, D5, D9, D16, D23, D32	VH
281	Zigzag Iterator	M	Design	Interleave lists	25	1424	D3, D5, D9, D16, D23, D32	M
238	Product of Array Except Self	M	Prefix/Suffix	No-division product	25	42	D3, D5, D9, D16, D23, D32	VH
8	String to Integer (atoi)	M	String	Overflow handling	25	65	D3, D5, D9, D16, D23, D32	H
128	Longest Consecutive Sequence	M	HashSet	Sequence start detection	25	298	D3, D5, D9, D16, D23, D32	VH
179	Largest Number	M	Custom sort	String concat order	25	921	D3, D5, D9, D16, D23, D32	M
16	3Sum Closest	M	Two Pointers	Closest sum target	25	15	D3, D5, D9, D16, D23, D32	H
209	Minimum Size Subarray Sum	M	Sliding Window	Shrinkable window	25	3	D3, D5, D9, D16, D23, D32	VH
154	Find Minimum in Rotated II	H	Binary Search	Dupes in rotated	30	153	D3, D5, D9, D16, D23, D32	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
84	Largest Rectangle in Histogram	H	Monotonic Stack	Bar expansion	35	85	D3, D5, D9, D16, D23, D32	VH
297	Serialize and Deserialize Binary Tree	H	Tree	Codec design	35	449	D3, D5, D9, D16, D23, D32	VH
502	IPO	H	Heap	Capital max	35	621	D3, D5, D9, D16, D23, D32	M
815	Bus Routes	H	Graph BFS	Route as node	40	127	D3, D5, D9, D16, D23, D32	M
305	Number of Islands II	H	Union Find	Dynamic connectivity	35	200	D3, D5, D9, D16, D23, D32	M

Day 4 — July 4, 2026 | Phase: Foundation | 17 problems | Focus: Express middleware depth — you'll live here in backend rounds.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
485	Max Consecutive Ones	E	Linear scan	Run length	10	1004	D4, D6, D10, D17, D24, D33	M
28	Find the Index of the First Occurrence	E	String	KMP / built-in	15	214	D4, D6, D10, D17, D24, D33	H
454	4Sum II	M	HashMap	Pair sum count	25	18	D4, D6, D10, D17, D24, D33	M
324	Wiggle Sort II	M	Sorting	Median partition	30	280	D4, D6, D10, D17, D24, D33	M
18	4Sum	M	Two Pointers	k-sum extension	35	454	D4, D6, D10, D17, D24, D33	H
340	Longest Substring with At Most K Distinct Characters	M	Sliding Window	K distinct limit	25	159	D4, D6, D10, D17, D24, D33	H
34	Find First and Last Position of Element in Sorted Array	M	Binary Search	Boundary search	25	35	D4, D6, D10, D17, D24, D33	VH
155	Min Stack	M	Stack	O(1) min	20	716	D4, D6, D10, D17, D24, D33	VH
362	Design Hit Counter	M	Queue	Sliding window count	20	359	D4, D6, D10, D17, D24, D33	M
19	Remove Nth Node From End of List	M	Linked List	Two-pointer gap	20	206	D4, D6, D10, D17, D24, D33	VH
102	Binary Tree Level Order Traversal	M	BFS	Level-by-level	20	107	D4, D6, D10, D17, D24, D33	VH
218	The Skyline Problem	H	Heap	Sweep line	45	699	D4, D6, D10, D17, D24, D33	H
839	Similar String Groups	H	Union Find	Swap similarity	35	721	D4, D6, D10, D17, D24, D33	M
37	Sudoku Solver	H	Backtracking	Constraint propagation	45	36	D4, D6, D10, D17, D24, D33	H
135	Candy	H	Greedy	Two-pass ratings	30	134	D4, D6, D10, D17, D24, D33	H
312	Burst Balloons	H	DP	Interval DP	40	1039	D4, D6, D10, D17, D24, D33	H
126	Word Ladder II	H	BFS + backtrack	All shortest paths	45	127	D4, D6, D10, D17, D24, D33	H

Day 5 — July 5, 2026 | Phase: Foundation | 18 problems | Focus: Async mastery plus Rate Limiter design — both high-frequency interview topics.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
724	Find Pivot Index	E	Prefix Sum	Balance index	15	1991	D5, D7, D11, D18, D25, D34	M
415	Add Strings	E	String	Digit add	15	2	D5, D7, D11, D18, D25, D34	M
560	Subarray Sum Equals K	M	HashMap	Prefix count	25	974	D5, D7, D11, D18, D25, D34	VH
912	Sort an Array	M	Merge/Quick	Classic sort	30	215	D5, D7, D11, D18, D25, D34	M
167	Two Sum II - Input Array Is Sorted	M	Two Pointers	Sorted pair search	15	1	D5, D7, D11, D18, D25, D34	VH

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
424	Longest Repeating Character Replacement	M	Sliding Window	Max freq	30	1004	D5, D7, D11, D18, D25, D34	VH
74	Search a 2D Matrix	M	Binary Search	Flattened search	20	240	D5, D7, D11, D18, D25, D34	H
227	Basic Calculator II	M	Stack	Expression parsing	30	224	D5, D7, D11, D18, D25, D34	H
92	Reverse Linked List II	M	Linked List	Partial reverse	25	206	D5, D7, D11, D18, D25, D34	H
236	Lowest Common Ancestor of a Binary Tree	M	Tree	LCA postorder	25	235	D5, D7, D11, D18, D25, D34	VH
98	Validate Binary Search Tree	M	BST	Range validation	25	230	D5, D7, D11, D18, D25, D34	VH
480	Sliding Window Median	H	Heap/BST	Window median	40	295	D5, D7, D11, D18, D25, D34	H
2157	Groups Strings Connected	H	Union Find	One char diff	35	839	D5, D7, D11, D18, D25, D34	M
51	N-Queens	H	Backtracking	Constraint placement	40	52	D5, D7, D11, D18, D25, D34	VH
330	Patching Array	H	Greedy	Missing coverage	35	45	D5, D7, D11, D18, D25, D34	M
44	Wildcard Matching	H	DP	Pattern matching	40	10	D5, D7, D11, D18, D25, D34	H
332	Reconstruct Itinerary	H	Eulerian path	Lexicographic route	35	207	D5, D7, D11, D18, D25, D34	VH
308	Range Sum Query 2D Mutable	H	Segment Tree	2D fenwick	40	307	D5, D7, D11, D18, D25, D34	M

Day 6 — July 6, 2026 | Phase: Foundation | 18 problems | Focus: Production-grade error handling — differentiate senior from mid-level answers.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
136	Single Number	E	Bit	XOR cancel	10	137	D6, D8, D12, D19, D26, D35	H
541	Reverse String II	E	String	Chunk reverse	12	344	D6, D8, D12, D19, D26, D35	M
974	Subarray Sums Divisible by K	M	HashMap	Mod prefix	25	560	D6, D8, D12, D19, D26, D35	H
539	Minimum Time Difference	M	Sorting	Circular time	25	253	D6, D8, D12, D19, D26, D35	M
611	Valid Triangle Number	M	Two Pointers	Sorted triple	25	15	D6, D8, D12, D19, D26, D35	M
438	Find All Anagrams	M	Sliding Window	Fixed window	25	567	D6, D8, D12, D19, D26, D35	H
153	Find Minimum in Rotated Sorted Array	M	Binary Search	Min in rotated	20	154	D6, D8, D12, D19, D26, D35	VH
394	Decode String	M	Stack	Nested decode	25	20	D6, D8, D12, D19, D26, D35	H
138	Copy List with Random Pointer	M	Linked List	HashMap clone	30	133	D6, D8, D12, D19, D26, D35	VH
95	Unique Binary Search Trees II	M	Tree	Generate all BSTs	30	96	D6, D8, D12, D19, D26, D35	M
230	Kth Smallest in BST	M	BST	Inorder kth	20	98	D6, D8, D12, D19, D26, D35	VH

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
499	Max Value of Equation	H	Heap	Line equation	35	149	D6, D8, D12, D19, D26, D35	M
465	Optimal Account Balancing	H	Backtracking	Min transfers	40	1300	D6, D8, D12, D19, D26, D35	M
358	Rearrange String k Distance Apart	H	Heap/Greedy	K distance apart	35	767	D6, D8, D12, D19, D26, D35	M
87	Scramble String	H	DP	Recursion + memo	40	44	D6, D8, D12, D19, D26, D35	M
753	Cracking the Safe	H	Graph/Euler	De Bruijn seq	40	332	D6, D8, D12, D19, D26, D35	M
363	Max Sum of Rectangle No Larger Than K	H	TreeSet/DP	Submatrix sum	40	53	D6, D8, D12, D19, D26, D35	M
41	First Missing Positive	H	Arrays	Index placement	30	448	D6, D8, D12, D19, D26, D35	VH

Day 7 — July 7, 2026 | Phase: Foundation | 19 problems | Focus: Week 1 mock day — test Node/SQL/behavioral under pressure, then review gaps.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
709	To Lower Case	E	String	ASCII convert	8	344	D7, D9, D13, D20, D27, D36	L
1	Two Sum	E	HashMap	Complement lookup	15	167	D7, D9, D13, D20, D27, D36	VH
881	Boats to Save People	M	Two Pointers	Greedy pair	20	167	D7, D9, D13, D20, D27, D36	H
567	Permutation in String	M	Sliding Window	Anagram window	25	438	D7, D9, D13, D20, D27, D36	H
162	Find Peak Element	M	Binary Search	Peak finding	20	852	D7, D9, D13, D20, D27, D36	H
503	Next Greater Element II	M	Monotonic Stack	Circular NGE	20	739	D7, D9, D13, D20, D27, D36	M
142	Linked List Cycle II	M	Fast/slow	Cycle entry point	25	141	D7, D9, D13, D20, D27, D36	VH
103	Binary Tree Zigzag Level Order	M	Tree BFS	Alternate direction	25	102	D7, D9, D13, D20, D27, D36	H
235	Lowest Common Ancestor of a BST	M	BST	BST LCA walk	15	236	D7, D9, D13, D20, D27, D36	VH
215	Kth Largest Element in an Array	M	Quickselect / heap	Partition selection	25	347	D7, D9, D13, D20, D27, D36	VH
208	Implement Trie (Prefix Tree)	M	Trie	Prefix tree ops	25	211	D7, D9, D13, D20, D27, D36	VH
130	Surrounded Regions	M	Graph DFS	Boundary capture	25	417	D7, D9, D13, D20, D27, D36	H
679	24 Game	H	Backtracking	Expression build	35	241	D7, D9, D13, D20, D27, D36	M
420	Strong Password Checker	H	Greedy	Edit distance rules	35	72	D7, D9, D13, D20, D27, D36	M
132	Palindrome Partitioning II	H	DP	Min cuts	35	131	D7, D9, D13, D20, D27, D36	M
847	Shortest Path All Keys	H	BFS+Bitmask	State compression	45	127	D7, D9, D13, D20, D27, D36	M
587	Erect the Fence	H	Geometry	Convex hull	40	149	D7, D9, D13, D20, D27, D36	M
212	Word Search II	H	Trie + DFS	Board word search	40	79	D7, D9, D13, D20, D27, D36	VH
65	Valid Number	H	String	State machine parse	35	8	D7, D9, D13, D20, D27, D36	M

Day 8 — July 8, 2026 | Phase: Foundation | 20 problems | Focus: Auth is non-negotiable for backend roles — nail JWT and OAuth flows.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
242	Valid Anagram	E	HashMap	Frequency count	10	438	D8, D10, D14, D21, D28, D37	VH
252	Meeting Rooms	E	Sorting	Interval overlap check	15	253	D8, D10, D14, D21, D28, D37	VH
26	Remove Duplicates from Sorted Array	E	Two Pointers	In-place overwrite	15	80	D8, D10, D14, D21, D28, D37	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
713	Subarray Product Less Than K	M	Sliding Window	Product window	25	209	D8, D10, D14, D21, D28, D37	H
875	Koko Eating Bananas	M	Binary Search	Answer space	25	1011	D8, D10, D14, D21, D28, D37	VH
739	Daily Temperatures	M	Monotonic Stack	NGE template	25	496	D8, D10, D14, D21, D28, D37	VH
328	Odd Even Linked List	M	Linked List	Partition nodes	20	86	D8, D10, D14, D21, D28, D37	M
105	Construct BT from Preorder Inorder	M	Tree	Divide by root	30	106	D8, D10, D14, D21, D28, D37	VH
450	Delete Node in a BST	M	BST	BST deletion	25	98	D8, D10, D14, D21, D28, D37	H
767	Reorganize String	M	Heap	No adjacent dup	25	621	D8, D10, D14, D21, D28, D37	H
211	Design Add and Search Words Data Structure	M	Trie	Wildcard search	30	212	D8, D10, D14, D21, D28, D37	H
133	Clone Graph	M	Graph BFS/DFS	Graph deep copy	25	138	D8, D10, D14, D21, D28, D37	VH
210	Course Schedule II	M	Topological Sort	Order construction	30	207	D8, D10, D14, D21, D28, D37	VH
980	Unique Paths III	H	Backtracking	Visit all cells	35	62	D8, D10, D14, D21, D28, D37	H
757	Set Intersection Size At Least Two	H	Greedy	Interval cover	35	435	D8, D10, D14, D21, D28, D37	M
174	Dungeon Game	H	DP	Reverse min health	35	64	D8, D10, D14, D21, D28, D37	M
864	Shortest Path All Keys	H	BFS+Bitmask	Keys bitmask	45	847	D8, D10, D14, D21, D28, D37	M
699	Falling Squares	H	Segment Tree	Height timeline	40	218	D8, D10, D14, D21, D28, D37	M
329	Longest Increasing Path in a Matrix	H	Graph DFS + memo	Grid longest path	35	300	D8, D10, D14, D21, D28, D37	VH
214	Shortest Palindrome	H	String	KMP prefix	35	5	D8, D10, D14, D21, D28, D37	M

Day 9 — July 9, 2026 | Phase: Foundation | 20 problems | Focus: API design polish — interviewers probe REST depth even for Node roles.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
205	Isomorphic Strings	E	HashMap	Bijection mapping	15	290	D9, D11, D15, D22, D29, D38	M
27	Remove Element	E	Two Pointers	Partition array	12	283	D9, D11, D15, D22, D29, D38	M
35	Search Insert Position	E	Binary Search	Lower bound	12	34	D9, D11, D15, D22, D29, D38	H
150	Evaluate Reverse Polish Notation	M	Stack	RPN eval	20	224	D9, D11, D15, D22, D29, D38	H
2	Add Two Numbers	M	Linked List	Digit carry chain	25	445	D9, D11, D15, D22, D29, D38	VH
106	Construct BT from Inorder Postorder	M	Tree	Postorder root	30	105	D9, D11, D15, D22, D29, D38	H
669	Trim a Binary Search Tree	M	BST	Range trim	20	450	D9, D11, D15, D22, D29, D38	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
973	K Closest Points to Origin	M	Heap	k nearest	20	215	D9, D11, D15, D22, D29, D38	VH
648	Replace Words	M	Trie	Prefix replace	20	208	D9, D11, D15, D22, D29, D38	M
200	Number of Islands	M	Graph DFS/BFS	Connected components	25	695	D9, D11, D15, D22, D29, D38	VH
310	Minimum Height Trees	M	Topological Sort	Leaf pruning	30	210	D9, D11, D15, D22, D29, D38	H
547	Number of Provinces	M	Union Find	Connected comps	25	200	D9, D11, D15, D22, D29, D38	VH
17	Letter Combinations of a Phone Number	M	Backtracking	Choice tree DFS	25	22	D9, D11, D15, D22, D29, D38	VH
188	Best Time to Buy and Sell Stock IV	H	DP	k transactions	40	123	D9, D11, D15, D22, D29, D38	H
715	Range Module	H	TreeMap	Interval module	40	352	D9, D11, D15, D22, D29, D38	M
759	Employee Free Time	H	Heap/Intervals	Common free	35	253	D9, D11, D15, D22, D29, D38	M
149	Max Points on a Line	H	HashMap	Slope counting	35	228	D9, D11, D15, D22, D29, D38	M
42	Trapping Rain Water	H	Two Pointers	Left/right max	35	407	D9, D11, D15, D22, D29, D38	VH
30	Substring with Concatenation of All Words	H	Sliding Window	Multi-word window	40	438	D9, D11, D15, D22, D29, D38	H
352	Data Stream as Disjoint Intervals	H	Binary Search	Merge intervals	35	56	D9, D11, D15, D22, D29, D38	M

Day 10 — July 10, 2026 | Phase: Foundation | 21 problems | Focus: Close Foundation phase with testing discipline on both stacks.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
20	Valid Parentheses	E	Stack	Bracket matching	15	32	D10, D12, D16, D23, D30, D39	VH
225	Implement Stack using Queues	E	Queue	Stack from queues	15	232	D10, D12, D16, D23, D30, D39	M
21	Merge Two Sorted Lists	E	Linked List	Merge sorted lists	15	23	D10, D12, D16, D23, D30, D39	VH
109	Convert Sorted List to BST	M	Tree	Inorder simulation	30	108	D10, D12, D16, D23, D30, D39	M
701	Insert into BST	M	BST	BST insert	15	450	D10, D12, D16, D23, D30, D39	M
1424	Diagonal Traverse II	M	Heap	Diagonal order	25	498	D10, D12, D16, D23, D30, D39	M
677	Map Sum Pairs	M	Trie	Prefix sum	25	208	D10, D12, D16, D23, D30, D39	M
207	Course Schedule	M	Graph DFS	Cycle detection	25	210	D10, D12, D16, D23, D30, D39	VH
802	Find Eventual Safe States	M	Topological Sort	Safe nodes	30	207	D10, D12, D16, D23, D30, D39	H
684	Redundant Connection	M	Union Find	Cycle edge	25	685	D10, D12, D16, D23, D30, D39	H
22	Generate Parentheses	M	Backtracking	Valid paren generation	25	301	D10, D12, D16, D23, D30, D39	VH
45	Jump Game II	M	Greedy	Min jumps	25	55	D10, D12, D16, D23, D30, D39	VH
57	Insert Interval	M	Intervals	Merge on insert	25	56	D10, D12, D16, D23, D30, D39	VH
446	Arithmetic Slices II Subsequence	H	DP	Subseq AP count	40	413	D10, D12, D16, D23, D30, D39	M
732	My Calendar III	H	TreeMap	Triple overlap	35	731	D10, D12, D16, D23, D30, D39	M
778	Swim in Rising Water	H	Dijkstra/UF	Min max path	35	743	D10, D12, D16, D23, D30, D39	M
220	Contains Duplicate III	H	Bucket/SortedSet	Window abs diff	35	219	D10, D12, D16, D23, D30, D39	H
76	Minimum Window Substring	H	Sliding Window	Cover all chars	35	3	D10, D12, D16, D23, D30, D39	VH
410	Split Array Largest Sum	H	Binary Search	Minimize max sum	35	875	D10, D12, D16, D23, D30, D39	H
85	Maximal Rectangle	H	Monotonic Stack	2D histogram reduce	40	84	D10, D12, D16, D23, D30, D39	H
987	Vertical Order Traversal	H	Tree BFS	Column sort	35	314	D10, D12, D16, D23, D30, D39	H

Day 11 — July 11, 2026 | Phase: Build | 22 problems | Focus: Bridge MERN Mongo skills to Mongoose + start Spring Boot journey.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
108	Convert Sorted Array to Binary Search Tree	E	BST	Balanced build	15	109	D11, D13, D17, D24, D31	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
1046	Last Stone Weight	E	Heap	Max-heap simulation	15	295	D11, D13, D17, D24, D31	M
14	Longest Common Prefix	E	Trie / scan	Vertical scan	15	208	D11, D13, D17, D24, D31	H
417	Pacific Atlantic Water Flow	M	Graph DFS	Multi-source	30	130	D11, D13, D17, D24, D31	H
1136	Parallel Courses	M	Topological Sort	Semester simulation	25	207	D11, D13, D17, D24, D31	M
721	Accounts Merge	M	Union Find	Email merge	30	547	D11, D13, D17, D24, D31	VH
39	Combination Sum	M	Backtracking	Unbounded combinations	25	40	D11, D13, D17, D24, D31	VH
55	Jump Game	M	Greedy	Reachability	20	45	D11, D13, D17, D24, D31	VH
72	Edit Distance	M	DP	Levenshtein distance	30	1143	D11, D13, D17, D24, D31	VH
1514	Path Max Probability	M	Dijkstra	Max prob path	30	743	D11, D13, D17, D24, D31	M
307	Range Sum Query Mutable	M	Segment Tree	Point update	30	315	D11, D13, D17, D24, D31	M
274	H-Index	M	Counting sort	Citation threshold	20	275	D11, D13, D17, D24, D31	M
43	Multiply Strings	M	String	Grade-school multiply	30	415	D11, D13, D17, D24, D31	M
166	Fraction to Recurring Decimal	M	HashMap	Cycle detection	25	141	D11, D13, D17, D24, D31	M
992	Subarrays K Different Integers	H	Sliding Window	At-most K trick	35	904	D11, D13, D17, D24, D31	H
719	Find K-th Smallest Pair Distance	H	Binary Search	Pair distance	40	658	D11, D13, D17, D24, D31	H
239	Sliding Window Maximum	H	Monotonic Deque	Window max	30	862	D11, D13, D17, D24, D31	VH
675	Cut Off Trees for Golf Event	H	BFS	Multi-target BFS	40	994	D11, D13, D17, D24, D31	M
632	Smallest Range Covering K Lists	H	Heap/Two Ptr	Range cover	35	373	D11, D13, D17, D24, D31	H
996	Number of Squareful Arrays	H	Backtracking	Perfect square adj	40	46	D11, D13, D17, D24, D31	M
466	Count The Repetitions	H	DP	String cycle	35	418	D11, D13, D17, D24, D31	M
140	Word Break II	H	Backtracking +DP	All segmentations	40	139	D11, D13, D17, D24, D31	H

Day 12 — July 12, 2026 | Phase: Build | 22 problems | Focus: Caching patterns — essential for system design and Node production apps.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
219	Contains Duplicate II	E	HashMap	Index distance	12	220	D12, D14, D18, D25, D32	H
125	Valid Palindrome	E	Two Pointers	Alphanumeric filter	12	680	D12, D14, D18, D25, D32	H
69	Sqrt(x)	E	Binary Search	Integer sqrt	12	367	D12, D14, D18, D25, D32	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
173	Binary Search Tree Iterator	M	Stack	Controlled inorder	25	230	D12, D14, D18, D25, D32	H
61	Rotate List	M	Linked List	Circular rotate	25	19	D12, D14, D18, D25, D32	M
113	Path Sum II	M	Tree DFS	All root-leaf paths	25	437	D12, D14, D18, D25, D32	H
285	Inorder Successor in BST	M	BST	Successor walk	20	510	D12, D14, D18, D25, D32	M
373	Find K Pairs with Smallest Sums	M	Heap	K-way merge	30	23	D12, D14, D18, D25, D32	H
1268	Search Suggestions System	M	Trie	Autocomplete	25	208	D12, D14, D18, D25, D32	H
695	Max Area of Island	M	Graph DFS	Component area	20	200	D12, D14, D18, D25, D32	H
990	Satisfiability Equality Equations	M	Union Find	Var union	25	684	D12, D14, D18, D25, D32	H
46	Permutations	M	Backtracking	Permutation generation	25	47	D12, D14, D18, D25, D32	VH
134	Gas Station	M	Greedy	Circuit feasibility	25	135	D12, D14, D18, D25, D32	H
139	Word Break	M	DP	Segmentation DP	25	140	D12, D14, D18, D25, D32	VH
158	Read N Characters Given Read4 II	H	Array	Multiple calls	30	157	D12, D14, D18, D25, D32	M
391	Perfect Rectangle	H	HashMap	Area + corners	35	85	D12, D14, D18, D25, D32	M
1851	Min Interval Include Query	H	Binary Search	Interval query	40	56	D12, D14, D18, D25, D32	H
862	Shortest Subarray Sum K	H	Monotonic Deque	Prefix deque	35	239	D12, D14, D18, D25, D32	H
854	K Similar Strings	H	BFS	Swap distance	35	72	D12, D14, D18, D25, D32	M
514	Freedom Trail	H	DP	Ring typing	40	72	D12, D14, D18, D25, D32	M
296	Best Meeting Point	H	Math	Manhattan median	30	286	D12, D14, D18, D25, D32	M
710	Random Pick with Blacklist	H	HashMap	Remap indices	35	398	D12, D14, D18, D25, D32	M

Day 13 — July 13, 2026 | Phase: Build | 23 problems | Focus: Rate limiting implementation ties Node skills to tomorrow's system design review.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
283	Move Zeroes	E	Two Pointers	In-place compaction	12	27	D13, D15, D19, D26, D33	H
278	First Bad Version	E	Binary Search	First true predicate	15	35	D13, D15, D19, D26, D33	H
496	Next Greater Element I	E	Monotonic Stack	NGE template	15	739	D13, D15, D19, D26, D33	H
82	Remove Duplicates from Sorted List II	M	Linked List	Skip all dupes	20	83	D13, D15, D19, D26, D33	M
114	Flatten Binary Tree to Linked List	M	Tree	Morris / reverse	30	430	D13, D15, D19, D26, D33	H
510	Inorder Successor BST II	M	BST	Parent pointer	20	285	D13, D15, D19, D26, D33	M
1834	Single Thread CPU	M	Heap	Task scheduling	30	621	D13, D15, D19, D26, D33	H
676	Implement Magic Dictionary	M	Trie	One char diff	25	211	D13, D15, D19, D26, D33	M
797	All Paths Source Target	M	Graph DFS	Path enum	20	113	D13, D15, D19, D26, D33	M
1319	Network Connected	M	Union Find	Extra cables	25	547	D13, D15, D19, D26, D33	M
78	Subsets	M	Backtracking	Subset enumeration	20	90	D13, D15, D19, D26, D33	VH
253	Meeting Rooms II	M	Greedy / heap	Min rooms needed	25	252	D13, D15, D19, D26, D33	VH
152	Maximum Product Subarray	M	DP	Min/max product track	25	53	D13, D15, D19, D26, D33	VH
1584	Min Cost Connect Points	M	MST	Manhattan MST	30	743	D13, D15, D19, D26, D33	H
301	Remove Invalid Parentheses	H	BFS/Backtrack	Min removals	40	22	D13, D15, D19, D26, D33	H
224	Basic Calculator	H	Stack	Expression + parens	35	227	D13, D15, D19, D26, D33	H
629	K Inverse Pairs Array	H	DP	Inversion count	40	315	D13, D15, D19, D26, D33	M
315	Count of Smaller Numbers After Self	H	Merge Sort/BIT	Inversion count	40	493	D13, D15, D19, D26, D33	H
770	Basic Calculator IV	H	Stack	Polynomial eval	45	224	D13, D15, D19, D26, D33	M
639	Decode Ways II	H	DP	Wildcard decode	35	91	D13, D15, D19, D26, D33	M
327	Count of Range Sum	H	Merge Sort	Range count	45	315	D13, D15, D19, D26, D33	M
772	Basic Calculator III	H	Stack	Nested parens	40	224	D13, D15, D19, D26, D33	M
730	Count Different Palindromes	H	DP	Distinct palins	45	647	D13, D15, D19, D26, D33	M

Day 14 — July 14, 2026 | Phase: Build | 24 problems | Focus: Week 2 mock — Spring + distributed cache design; prioritize gap review.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
157	Read N Characters Given Read4	E	Array	Buffer read	15	158	D14, D16, D20, D27, D34	M
290	Word Pattern	E	HashMap	Bijection check	15	205	D14, D16, D20, D27, D34	M
344	Reverse String	E	Two Pointers	In-place swap	8	541	D14, D16, D20, D27, D34	M
904	Fruit Into Baskets	M	Sliding Window	At most 2 distinct	25	992	D14, D16, D20, D27, D34	H
1011	Capacity Ship Packages	M	Binary Search	Feasibility	30	875	D14, D16, D20, D27, D34	H
385	Mini Parser	M	Stack	Nested integer	25	341	D14, D16, D20, D27, D34	M
86	Partition List	M	Linked List	Before/after pivot	25	328	D14, D16, D20, D27, D34	M
116	Populating Next Right Pointers	M	Tree BFS	Level connect	25	117	D14, D16, D20, D27, D34	M
536	Construct BST from Preorder	M	BST	Upper bound DFS	25	105	D14, D16, D20, D27, D34	M
1166	Design File System	M	Trie	Path trie	30	588	D14, D16, D20, D27, D34	M
994	Rotting Oranges	M	Graph BFS	Multi-source	25	286	D14, D16, D20, D27, D34	VH
737	Sentence Similarity II	M	Union Find	Word groups	25	721	D14, D16, D20, D27, D34	M
79	Word Search	M	Backtracking	Grid path search	25	212	D14, D16, D20, D27, D34	VH
406	Queue Reconstruction by Height	M	Greedy	Sort insert	25	763	D14, D16, D20, D27, D34	H
198	House Robber	M	DP	Linear choice DP	20	213	D14, D16, D20, D27, D34	VH
354	Russian Doll Envelopes	H	DP/Binary Search	LIS variant	35	300	D14, D16, D20, D27, D34	H
741	Cherry Pickup	H	DP	Grid two-pass	40	174	D14, D16, D20, D27, D34	M
381	Insert Delete GetRandom O(1) Duplicates	H	HashMap+Set	Dup allowed	35	380	D14, D16, D20, D27, D34	M
940	Distinct Subsequences II	H	DP	Mod count subseq	35	115	D14, D16, D20, D27, D34	M
403	Frog Jump	H	DP/HashSet	Stone jump	35	55	D14, D16, D20, D27, D34	M
407	Trapping Rain Water II	H	Heap/BFS	3D rain water	40	42	D14, D16, D20, D27, D34	M
432	All O`one Data Structure	H	HashMap+DLL	Freq buckets	40	460	D14, D16, D20, D27, D34	M
440	K-th Smallest in Lex Order	H	Math/DFS	Lex order nums	35	386	D14, D16, D20, D27, D34	M
458	Poor Pigs	H	Math	Information theory	30	319	D14, D16, D20, D27, D34	M

Day 15 — July 15, 2026 | Phase: Build | 24 problems | Focus: File upload flows appear in many backend interviews — know S3 patterns cold.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
349	Intersection of Two Arrays	E	HashSet	Set intersection	12	350	D15, D17, D21, D28, D35	M
680	Valid Palindrome II	E	Two Pointers	One deletion allowed	15	125	D15, D17, D21, D28, D35	H
704	Binary Search	E	Binary Search	Classic template	12	35	D15, D17, D21, D28, D35	M
388	Longest Absolute File Path	M	Stack	Path depth	25	71	D15, D17, D21, D28, D35	M
143	Reorder List	M	Linked List	Mid reverse merge	30	234	D15, D17, D21, D28, D35	H
117	Populating Next Right Pointers II	M	Tree BFS	Imperfect tree	25	116	D15, D17, D21, D28, D35	M
286	Walls and Gates	M	Graph BFS	Multi-source BFS	25	994	D15, D17, D21, D28, D35	H
947	Most Stones Removed	M	Union Find	Row/col union	25	547	D15, D17, D21, D28, D35	H
131	Palindrome Partitioning	M	Backtracking	Partition + palin check	30	5	D15, D17, D21, D28, D35	H
435	Non-overlapping Intervals	M	Greedy	Interval scheduling	25	452	D15, D17, D21, D28, D35	VH
213	House Robber II	M	DP	Circular robber	25	198	D15, D17, D21, D28, D35	VH
1631	Path Minimum Effort	M	Dijkstra	Min max edge	30	778	D15, D17, D21, D28, D35	H
348	Design Tic-Tac-Toe	M	Design	Row/col/diag track	25	794	D15, D17, D21, D28, D35	M
347	Top K Frequent Elements	M	Heap/Bucket	Frequency top-k	25	692	D15, D17, D21, D28, D35	VH
151	Reverse Words in a String	M	String	Two-pass reverse	20	186	D15, D17, D21, D28, D35	H
460	LFU Cache	H	HashMap+DLL	Freq eviction	45	146	D15, D17, D21, D28, D35	H
483	Smallest Good Base	H	Math	Base conversion	40	168	D15, D17, D21, D28, D35	M
493	Reverse Pairs	H	Merge Sort	Inversion variant	40	315	D15, D17, D21, D28, D35	H
548	Split Array with Equal Sum	H	Prefix Sum	4-partition	35	416	D15, D17, D21, D28, D35	M
588	Design In-Memory File System	H	Trie/HashMap	Path trie	45	1166	D15, D17, D21, D28, D35	M
668	Kth Smallest in Lex Order	H	Math	Lex order	35	440	D15, D17, D21, D28, D35	M
689	Maximum Sum of 3 Non-Overlapping Subarrays	H	DP/Prefix	Three windows	35	53	D15, D17, D21, D28, D35	H
691	Stickers to Spell Word	H	Backtracking/DP	Sticker cover	40	638	D15, D17, D21, D28, D35	M
736	Parse Lisp Expression	H	Stack/Recursion	Mini interpreter	45	224	D15, D17, D21, D28, D35	M

Day 16 — July 16, 2026 | Phase: Build | 25 problems | Focus: Real-time Node (Socket.io) plus News Feed design — high-yield combo.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
350	Intersection of Two Arrays II	E	HashMap	Freq intersection	15	349	D16, D18, D22, D29, D36	M
922	Sort Array By Parity	E	Two Pointers	Even first	10	905	D16, D18, D22, D29, D36	M
374	Guess Number Higher or Lower	E	Binary Search	Classic guess	10	278	D16, D18, D22, D29, D36	M
456	132 Pattern	M	Stack	Monotonic stack	25	84	D16, D18, D22, D29, D36	H
148	Sort List	M	Linked List	Merge sort LL	35	21	D16, D18, D22, D29, D36	H
129	Sum Root to Leaf Numbers	M	Tree DFS	Path digit sum	20	437	D16, D18, D22, D29, D36	M
399	Evaluate Division	M	Graph/Union Find	Weighted graph	30	947	D16, D18, D22, D29, D36	H
216	Combination Sum III	M	Backtracking	Fixed-size combinations	25	39	D16, D18, D22, D29, D36	M
452	Min Arrows Burst Balloons	M	Greedy	End-point sort	25	435	D16, D18, D22, D29, D36	H
300	Longest Increasing Subsequence	M	DP	LIS O(n log n)	30	674	D16, D18, D22, D29, D36	VH
505	The Maze II	M	Dijkstra	Shortest stop	30	490	D16, D18, D22, D29, D36	M
353	Design Snake Game	M	Design	Queue + set	30	499	D16, D18, D22, D29, D36	M
380	Insert Delete GetRandom O(1)	M	HashMap+Array	Swap delete	30	381	D16, D18, D22, D29, D36	H
161	One Edit Distance	M	String	Edit check	20	72	D16, D18, D22, D29, D36	M
229	Majority Element II	M	HashMap	Boyer-Moore II	25	169	D16, D18, D22, D29, D36	M
986	Interval List Intersections	M	Two Pointers	Merge intervals	25	56	D16, D18, D22, D29, D36	H
768	Max Chunks To Make Sorted II	H	Stack/Greedy	Monotonic chunks	35	763	D16, D18, D22, D29, D36	M
871	Minimum Number Refueling Stops	H	Heap/DP	Max heap greedy	35	134	D16, D18, D22, D29, D36	H
1383	Max Performance Team	H	Greedy/Sort	Speed * efficiency	35	502	D16, D18, D22, D29, D36	H
4	Median of Two Sorted Arrays	H	Binary Search	Partition merge	45	295	D16, D18, D22, D29, D36	VH
23	Merge k Sorted Lists	H	Heap / divide	k-way merge	35	21	D16, D18, D22, D29, D36	VH
25	Reverse Nodes in k-Group	H	Linked List	k-group reverse	35	206	D16, D18, D22, D29, D36	VH
32	Longest Valid Parentheses	H	Stack	Valid paren length	35	20	D16, D18, D22, D29, D36	VH
41	First Missing Positive	H	Arrays	Index placement	30	448	D16, D18, D22, D29, D36	VH
42	Trapping Rain Water	H	Two Pointers	Left/right max	35	407	D16, D18, D22, D29, D36	VH

Day 17 — July 17, 2026 | Phase: Build | 26 problems | Focus: Observability is a senior signal — implement it, don't just talk about it.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
359	Logger Rate Limiter	E	HashMap	Message throttle	12	362	D17, D19, D23, D30, D37	M
977	Squares of Sorted Array	E	Two Pointers	Merge ends	12	88	D17, D19, D23, D30, D37	M
232	Implement Queue using Stacks	E	Queue	Queue from stacks	15	225	D17, D19, D23, D30, D37	M
369	Plus One Linked List	M	Linked List	Reverse add	20	2	D17, D19, D23, D30, D37	M
156	Binary Tree Upside Down	M	Tree	Pointer rewrite	25	226	D17, D19, D23, D30, D37	M
433	Minimum Genetic Mutation	M	Graph BFS	Word ladder variant	25	127	D17, D19, D23, D30, D37	M
40	Combination Sum II	M	Backtracking	No reuse combos	25	39	D17, D19, D23, D30, D37	H
621	Task Scheduler	M	Greedy/Heap	Cooldown slots	25	767	D17, D19, D23, D30, D37	VH
322	Coin Change	M	DP	Min coins	25	518	D17, D19, D23, D30, D37	VH
1135	Connecting Cities Min Cost	M	MST	Kruskal/Prim	30	1584	D17, D19, D23, D30, D37	H
355	Design Twitter	M	Design	Feed + follow	35	Tweet	D17, D19, D23, D30, D37	H
641	Design Circular Deque	M	Deque	Double-ended ring	25	622	D17, D19, D23, D30, D37	M
165	Compare Version Numbers	M	String	Segment compare	20	43	D17, D19, D23, D30, D37	M
325	Maximum Size Subarray Sum Equals k	M	HashMap	Prefix sum	25	560	D17, D19, D23, D30, D37	H
443	String Compression	M	Two Pointers	In-place compress	20	38	D17, D19, D23, D30, D37	M
1004	Max Consecutive Ones III	M	Sliding Window	Flip K zeros	25	485	D17, D19, D23, D30, D37	VH
51	N-Queens	H	Backtracking	Constraint placement	40	52	D17, D19, D23, D30, D37	VH
76	Minimum Window Substring	H	Sliding Window	Cover all chars	35	3	D17, D19, D23, D30, D37	VH
84	Largest Rectangle in Histogram	H	Monotonic Stack	Bar expansion	35	85	D17, D19, D23, D30, D37	VH
124	Binary Tree Maximum Path Sum	H	Tree	Global max path	35	543	D17, D19, D23, D30, D37	VH
127	Word Ladder	H	BFS	Shortest transform	35	126	D17, D19, D23, D30, D37	VH
212	Word Search II	H	Trie + DFS	Board word search	40	79	D17, D19, D23, D30, D37	VH
239	Sliding Window Maximum	H	Monotonic Deque	Window max	30	862	D17, D19, D23, D30, D37	VH
269	Alien Dictionary	H	Topological Sort	Custom order graph	40	210	D17, D19, D23, D30, D37	VH
295	Find Median from Data Stream	H	Heap	Two heap balance	35	480	D17, D19, D23, D30, D37	VH
297	Serialize and Deserialize Binary Tree	H	Tree	Codec design	35	449	D17, D19, D23, D30, D37	VH

Day 18 — July 18, 2026 | Phase: Build | 26 problems | Focus: Security depth closes a common gap — OWASP answers must be automatic.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
346	Moving Average from Data Stream	E	Queue	Fixed-size average	15	933	D18, D20, D24, D31, D38	M
141	Linked List Cycle	E	Fast/slow	Cycle detection	15	142	D18, D20, D24, D31, D38	VH
94	Binary Tree Inorder Traversal	E	Tree	Inorder DFS/iter	15	144	D18, D20, D24, D31, D38	VH
490	The Maze	M	Graph DFS/BFS	Rolling ball	25	505	D18, D20, D24, D31, D38	M
47	Permutations II	M	Backtracking	Duplicate handling	25	46	D18, D20, D24, D31, D38	H
763	Partition Labels	M	Greedy	Last occurrence	20	856	D18, D20, D24, D31, D38	H
416	Partition Equal Subset Sum	M	DP	0/1 knapsack	30	494	D18, D20, D24, D31, D38	VH
1334	Find City With Threshold	M	Floyd-Warshall	All pairs	35	743	D18, D20, D24, D31, D38	M
379	Design Phone Directory	M	Design	Available number pool	25	362	D18, D20, D24, D31, D38	M
647	Palindromic Substrings	M	Expand	Count palindromes	25	5	D18, D20, D24, D31, D38	H
186	Reverse Words in a String II	M	String	In-place reverse	20	151	D18, D20, D24, D31, D38	M
447	Number of Boomerangs	M	HashMap	Slope counting	35	149	D18, D20, D24, D31, D38	M
581	Shortest Unsorted Continuous Subarray	M	Two Pointers	Sort boundary	25	280	D18, D20, D24, D31, D38	H
1456	Max Vowels Substring	M	Sliding Window	Fixed window	20	438	D18, D20, D24, D31, D38	M
50	Pow(x, n)	M	Binary Search	Fast exponentiation	20	372	D18, D20, D24, D31, D38	H
636	Exclusive Time of Functions	M	Stack	Call stack sim	30	227	D18, D20, D24, D31, D38	M
329	Longest Increasing Path in a Matrix	H	Graph DFS + memo	Grid longest path	35	300	D18, D20, D24, D31, D38	VH
332	Reconstruct Itinerary	H	Eulerian path	Lexicographic route	35	207	D18, D20, D24, D31, D38	VH
30	Substring with Concatenation of All Words	H	Sliding Window	Multi-word window	40	438	D18, D20, D24, D31, D38	H
37	Sudoku Solver	H	Backtracking	Constraint propagation	45	36	D18, D20, D24, D31, D38	H
85	Maximal Rectangle	H	Monotonic Stack	2D histogram reduce	40	84	D18, D20, D24, D31, D38	H
126	Word Ladder II	H	BFS + backtrack	All shortest paths	45	127	D18, D20, D24, D31, D38	H
312	Burst Balloons	H	DP	Interval DP	40	1039	D18, D20, D24, D31, D38	H
862	Shortest Subarray Sum K	H	Monotonic Deque	Prefix deque	35	239	D18, D20, D24, D31, D38	H
987	Vertical Order Traversal	H	Tree BFS	Column sort	35	314	D18, D20, D24, D31, D38	H
992	Subarrays K Different Integers	H	Sliding Window	At-most K trick	35	904	D18, D20, D24, D31, D38	H

Day 19 — July 19, 2026 | Phase: Build | 27 problems | Focus: Microservices narrative + Search design — connect your MERN experience to scale.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
933	Number of Recent Calls	E	Queue	Sliding window queue	12	346	D19, D21, D25, D32, D39	M
160	Intersection of Two Linked Lists	E	Linked List	Pointer switch trick	15	141	D19, D21, D25, D32, D39	H
104	Maximum Depth of Binary Tree	E	Tree	Recursive depth	10	111	D19, D21, D25, D32, D39	VH
530	Minimum Absolute Difference in BST	E	BST	Inorder adjacent	15	783	D19, D21, D25, D32, D39	M
529	Minesweeper	M	Graph DFS/BFS	Grid reveal	25	733	D19, D21, D25, D32, D39	M
90	Subsets II	M	Backtracking	Sorted dedupe	25	78	D19, D21, D25, D32, D39	H
849	Maximize Distance to Closest Person	M	Greedy	Seat gap analysis	20	855	D19, D21, D25, D32, D39	M
494	Target Sum	M	DP	+/- subset	30	416	D19, D21, D25, D32, D39	VH
382	Linked List Random Node	M	Reservoir	Reservoir sampling	20	398	D19, D21, D25, D32, D39	M
918	Max Sum Circular Subarray	M	Kadane	Circular wrap	25	53	D19, D21, D25, D32, D39	M
468	Validate IP Address	M	String	IPv4/IPv6 parse	25	93	D19, D21, D25, D32, D39	M
523	Continuous Subarray Sum	M	HashMap	Mod prefix	25	560	D19, D21, D25, D32, D39	H
948	Bag of Tokens	M	Two Pointers	Greedy tokens	25	881	D19, D21, D25, D32, D39	M
395	Longest Substring K Distinct	M	Sliding Window	At most K	25	340	D19, D21, D25, D32, D39	H
81	Search in Rotated Sorted Array II	M	Binary Search	Duplicates	25	33	D19, D21, D25, D32, D39	H
722	Remove Comments	M	Stack	Source parse	25	227	D19, D21, D25, D32, D39	M
430	Flatten Multilevel Doubly LL	M	Linked List	DFS flatten	30	114	D19, D21, D25, D32, D39	M
44	Wildcard Matching	H	DP	Pattern matching	40	10	D19, D21, D25, D32, D39	H
135	Candy	H	Greedy	Two-pass ratings	30	134	D19, D21, D25, D32, D39	H
140	Word Break II	H	Backtracking +DP	All segmentations	40	139	D19, D21, D25, D32, D39	H
154	Find Minimum in Rotated II	H	Binary Search	Dupes in rotated	30	153	D19, D21, D25, D32, D39	H
188	Best Time to Buy and Sell Stock IV	H	DP	k transactions	40	123	D19, D21, D25, D32, D39	H
218	The Skyline Problem	H	Heap	Sweep line	45	699	D19, D21, D25, D32, D39	H
220	Contains Duplicate III	H	Bucket/SortedSet	Window abs diff	35	219	D19, D21, D25, D32, D39	H
224	Basic Calculator	H	Stack	Expression + parens	35	227	D19, D21, D25, D32, D39	H
301	Remove Invalid Parentheses	H	BFS/Backtrack	Min removals	40	22	D19, D21, D25, D32, D39	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
315	Count of Smaller Numbers After Self	H	Merge Sort/BIT	Inversion count	40	493	D19, D21, D25, D32, D39	H

Day 20 — July 20, 2026 | Phase: Build | 28 problems | Focus: Close Build phase — GraphQL + Spring testing round out full-stack credibility.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
110	Balanced Binary Tree	E	Tree	Height balance check	15	543	D20, D22, D26, D33	H
703	Kth Largest in Stream	E	Heap	Min heap k	15	215	D20, D22, D26, D33	H
733	Flood Fill	E	Graph DFS/BFS	Paint fill	15	200	D20, D22, D26, D33	M
163	Missing Ranges	E	Array	Gap enumeration	15	228	D20, D22, D26, D33	M
686	Repeated String Match	M	String	Period check	25	28	D20, D22, D26, D33	M
525	Contiguous Array	M	HashMap	0/1 balance	30	560	D20, D22, D26, D33	H
2337	Move Pieces Obtain String	M	Two Pointers	L/R move sim	25	777	D20, D22, D26, D33	M
487	Max Consecutive Ones II	M	Sliding Window	Flip at most 1	20	1004	D20, D22, D26, D33	M
378	Kth Smallest in Sorted Matrix	M	Heap/Binary Search	Matrix kth	30	373	D20, D22, D26, D33	H
735	Asteroid Collision	M	Stack	Collision sim	25	739	D20, D22, D26, D33	H
707	Design Linked List	M	Linked List	Full LL API	30	146	D20, D22, D26, D33	M
199	Binary Tree Right Side View	M	Tree BFS	Last per level	20	545	D20, D22, D26, D33	H
542	01 Matrix	M	Graph BFS	Nearest 0	25	994	D20, D22, D26, D33	H
93	Restore IP Addresses	M	Backtracking	IP segment validation	30	17	D20, D22, D26, D33	M
12	Integer to Roman	M	Greedy	Symbol subtraction	20	13	D20, D22, D26, D33	M
518	Coin Change II	M	DP	Combination count	25	322	D20, D22, D26, D33	H
384	Shuffle an Array	M	Design	Fisher-Yates	20	382	D20, D22, D26, D33	M
31	Next Permutation	M	Array	In-place rearrange	25	60	D20, D22, D26, D33	H
354	Russian Doll Envelopes	H	DP/Binary Search	LIS variant	35	300	D20, D22, D26, D33	H
410	Split Array Largest Sum	H	Binary Search	Minimize max sum	35	875	D20, D22, D26, D33	H
460	LFU Cache	H	HashMap+DLL	Freq eviction	45	146	D20, D22, D26, D33	H
480	Sliding Window Median	H	Heap/BST	Window median	40	295	D20, D22, D26, D33	H
493	Reverse Pairs	H	Merge Sort	Inversion variant	40	315	D20, D22, D26, D33	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
632	Smallest Range Covering K Lists	H	Heap/Two Ptr	Range cover	35	373	D20, D22, D26, D33	H
689	Maximum Sum of 3 Non-Overlapping Subarrays	H	DP/Prefix	Three windows	35	53	D20, D22, D26, D33	H
719	Find K-th Smallest Pair Distance	H	Binary Search	Pair distance	40	658	D20, D22, D26, D33	H
871	Minimum Number Refueling Stops	H	Heap/DP	Max heap greedy	35	134	D20, D22, D26, D33	H
980	Unique Paths III	H	Backtracking	Visit all cells	35	62	D20, D22, D26, D33	H

Day 21 — July 21, 2026 | Phase: Intensify | 28 problems | Focus: Week 3 mock + Payment Gateway — intensity ramps; review DSA weak spots.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
387	First Unique Character	E	HashMap	Freq scan	10	442	D21, D23, D27, D34	M
88	Merge Sorted Array	E	Two Pointers	Fill from end	15	977	D21, D23, D27, D34	H
206	Reverse Linked List	E	Linked List	Iterative reverse	15	92	D21, D23, D27, D34	VH
144	Binary Tree Preorder Traversal	E	Tree	Preorder DFS/iter	15	94	D21, D23, D27, D34	H
565	Array Nesting	M	Graph/Union Find	Longest cycle	25	684	D21, D23, D27, D34	M
320	Generalized Abbreviation	M	Backtracking	Bit mask abbr	25	78	D21, D23, D27, D34	M
280	Wiggle Sort	M	Greedy	Peak valley	25	324	D21, D23, D27, D34	M
1143	Longest Common Subsequence	M	DP	2D string DP	25	72	D21, D23, D27, D34	VH
398	Random Pick Index	M	Reservoir	Equal probability	20	382	D21, D23, D27, D34	M
48	Rotate Image	M	Matrix	Transpose + reverse	20	54	D21, D23, D27, D34	H
532	K-diff Pairs	M	HashMap	Two sum variant	20	1	D21, D23, D27, D34	M
1052	Grumpy Bookstore Owner	M	Sliding Window	Fixed window tech	25	1004	D21, D23, D27, D34	H
436	Find Right Interval	M	Binary Search	Interval lookup	25	56	D21, D23, D27, D34	M
856	Score of Parentheses	M	Stack	Nested score	20	32	D21, D23, D27, D34	H
708	Insert Sorted Circular LL	M	Linked List	Circular insert	25	61	D21, D23, D27, D34	M
314	Binary Tree Vertical Order	M	Tree BFS	Column map	25	987	D21, D23, D27, D34	H
694	Number of Distinct Islands	M	Graph DFS	Shape hash	25	200	D21, D23, D27, D34	M
473	Matchsticks to Square	M	Backtracking	Partition 4	30	416	D21, D23, D27, D34	M
1383	Max Performance Team	H	Greedy/Sort	Speed * efficiency	35	502	D21, D23, D27, D34	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
1851	Min Interval Include Query	H	Binary Search	Interval query	40	56	D21, D23, D27, D34	H
502	IPO	H	Heap	Capital max	35	621	D21, D23, D27, D34	M
685	Redundant Connection II	H	Union Find	Directed cycle	35	684	D21, D23, D27, D34	M
759	Employee Free Time	H	Heap/Intervals	Common free	35	253	D21, D23, D27, D34	M
778	Swim in Rising Water	H	Dijkstra/UF	Min max path	35	743	D21, D23, D27, D34	M
1203	Sort Items Groups	H	Topological Sort	Grouped topo	40	269	D21, D23, D27, D34	M
65	Valid Number	H	String	State machine parse	35	8	D21, D23, D27, D34	M
87	Scramble String	H	DP	Recursion + memo	40	44	D21, D23, D27, D34	M
132	Palindrome Partitioning II	H	DP	Min cuts	35	131	D21, D23, D27, D34	M

Day 22 — July 22, 2026 | Phase: Intensify | 29 problems | Focus: Patterns day — map OOP patterns to Node idioms; Twitter design in afternoon.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
190	Reverse Bits	E	Bit	Bit manipulation	12	191	D22, D24, D28, D35	M
705	Design HashSet	E	HashMap	Basic hash set	15	706	D22, D24, D28, D35	M
392	Is Subsequence	E	Two Pointers	Greedy match	12	1143	D22, D24, D28, D35	H
234	Palindrome Linked List	E	Linked List	Find mid + reverse	20	206	D22, D24, D28, D35	VH
428	Serialize N-ary Tree	M	Tree	N-ary codec	30	297	D22, D24, D28, D35	M
752	Open the Lock	M	Graph BFS	Combo lock	30	127	D22, D24, D28, D35	H
526	Beautiful Arrangement	M	Backtracking	Divisibility perm	25	46	D22, D24, D28, D35	M
334	Increasing Triplet Subsequence	M	Greedy	Three pointers	20	300	D22, D24, D28, D35	H
62	Unique Paths	M	DP	Grid combinatorics	20	63	D22, D24, D28, D35	VH
497	Random Point in Non-overlapping Rectangles	M	Reservoir	Weighted pick	30	398	D22, D24, D28, D35	M
54	Spiral Matrix	M	Matrix	Boundary walk	25	59	D22, D24, D28, D35	H
535	Encode and Decode TinyURL	M	HashMap	Bijection encode	20	271	D22, D24, D28, D35	M
1343	Subarray Size K Avg	M	Sliding Window	Fixed window avg	20	209	D22, D24, D28, D35	M
475	Heaters	M	Binary Search	Min radius	25	875	D22, D24, D28, D35	M
907	Sum Subarray Mins	M	Monotonic Stack	Contribution	30	84	D22, D24, D28, D35	H
1524	Odd Even Linked List	M	Linked List	Reorder	20	328	D22, D24, D28, D35	M
429	N-ary Tree Level Order	M	Tree BFS	N-ary levels	20	102	D22, D24, D28, D35	M
886	Possible Bipartition	M	Graph BFS	2-coloring	25	207	D22, D24, D28, D35	H
149	Max Points on a Line	H	HashMap	Slope counting	35	228	D22, D24, D28, D35	M
158	Read N Characters Given Read4 II	H	Array	Multiple calls	30	157	D22, D24, D28, D35	M
174	Dungeon Game	H	DP	Reverse min health	35	64	D22, D24, D28, D35	M
214	Shortest Palindrome	H	String	KMP prefix	35	5	D22, D24, D28, D35	M
296	Best Meeting Point	H	Math	Manhattan median	30	286	D22, D24, D28, D35	M
305	Number of Islands II	H	Union Find	Dynamic connectivity	35	200	D22, D24, D28, D35	M
308	Range Sum Query 2D Mutable	H	Segment Tree	2D fenwick	40	307	D22, D24, D28, D35	M
317	Shortest Distance from All Buildings	H	Graph BFS	Multi-source sum	40	286	D22, D24, D28, D35	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
327	Count of Range Sum	H	Merge Sort	Range count	45	315	D22, D24, D28, D35	M
330	Patching Array	H	Greedy	Missing coverage	35	45	D22, D24, D28, D35	M
352	Data Stream as Disjoint Intervals	H	Binary Search	Merge intervals	35	56	D22, D24, D28, D35	M

Day 23 — July 23, 2026 | Phase: Intensify | 30 problems | Focus: TypeScript strengthens your Node story; CompletableFuture mirrors async/await mentally.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
191	Number of 1 Bits	E	Bit	Hamming weight	10	338	D23, D25, D29, D36	H
706	Design HashMap	E	HashMap	Basic hash map	15	705	D23, D25, D29, D36	M
876	Middle of the Linked List	E	Fast/slow	Midpoint detection	10	141	D23, D25, D29, D36	H
226	Invert Binary Tree	E	Tree	Mirror swap	12	101	D23, D25, D29, D36	VH
909	Snakes and Ladders	M	Graph BFS	Board shortest	30	127	D23, D25, D29, D36	M
698	Partition to K Equal Sum Subsets	M	Backtracking	K partition	35	416	D23, D25, D29, D36	H
481	Magical String	M	Greedy	Pattern generation	25	390	D23, D25, D29, D36	M
63	Unique Paths II	M	DP	Obstacle grid	20	64	D23, D25, D29, D36	H
519	Random Flip Matrix	M	Reservoir	Uniform flip	30	384	D23, D25, D29, D36	M
59	Spiral Matrix II	M	Matrix	Fill spiral	25	54	D23, D25, D29, D36	M
554	Brick Wall	M	HashMap	Min cross count	25	391	D23, D25, D29, D36	M
1438	Longest K Absolute Diff	M	Sliding Window	Sorted window	30	992	D23, D25, D29, D36	H
528	Random Pick with Weight	M	Binary Search	Weighted pick	25	398	D23, D25, D29, D36	H
1130	Min Valid Parenthesis Remove	M	Stack	Min removals	25	301	D23, D25, D29, D36	H
437	Path Sum III	M	Tree DFS	Prefix on path	30	112	D23, D25, D29, D36	H
934	Shortest Bridge	M	Graph DFS+BFS	Bridge connect	35	200	D23, D25, D29, D36	H
1980	Find Unique Binary String	M	Backtracking	Cantor diagonal	25	78	D23, D25, D29, D36	M
624	Maximum Distance in Arrays	M	Greedy	Min/max cross	20	149	D23, D25, D29, D36	M
64	Minimum Path Sum	M	DP	Grid min sum	20	120	D23, D25, D29, D36	H
358	Rearrange String k Distance Apart	H	Heap/Greedy	K distance apart	35	767	D23, D25, D29, D36	M
363	Max Sum of Rectangle No Larger Than K	H	TreeSet/DP	Submatrix sum	40	53	D23, D25, D29, D36	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
381	Insert Delete GetRandom O(1) Duplicates	H	HashMap+Set	Dup allowed	35	380	D23, D25, D29, D36	M
391	Perfect Rectangle	H	HashMap	Area + corners	35	85	D23, D25, D29, D36	M
403	Frog Jump	H	DP/HashSet	Stone jump	35	55	D23, D25, D29, D36	M
407	Trapping Rain Water II	H	Heap/BFS	3D rain water	40	42	D23, D25, D29, D36	M
420	Strong Password Checker	H	Greedy	Edit distance rules	35	72	D23, D25, D29, D36	M
432	All O`one Data Structure	H	HashMap+DLL	Freq buckets	40	460	D23, D25, D29, D36	M
440	K-th Smallest in Lex Order	H	Math/DFS	Lex order nums	35	386	D23, D25, D29, D36	M
446	Arithmetic Slices II Subsequence	H	DP	Subseq AP count	40	413	D23, D25, D29, D36	M
458	Poor Pigs	H	Math	Information theory	30	319	D23, D25, D29, D36	M

Day 24 — July 24, 2026 | Phase: Intensify | 30 problems | Focus: Cloud deployment patterns — show you can ship, not just code locally.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
228	Summary Ranges	E	Array	Consecutive ranges	15	163	D24, D26, D30, D37	M
203	Remove Linked List Elements	E	Linked List	Dummy head delete	12	237	D24, D26, D30, D37	M
543	Diameter of Binary Tree	E	Tree	Longest path	15	124	D24, D26, D30, D37	VH
997	Find Town Judge	E	Graph	In-degree n-1	12	207	D24, D26, D30, D37	M
646	Maximum Length of Pair Chain	M	Greedy	Interval chain	20	435	D24, D26, D30, D37	H
91	Decode Ways	M	DP	String segmentation	25	639	D24, D26, D30, D37	H
729	My Calendar I	M	TreeMap	Overlap check	25	715	D24, D26, D30, D37	H
73	Set Matrix Zeroes	M	Matrix	In-place markers	25	289	D24, D26, D30, D37	H
593	Valid Square	M	HashMap	Distance set	25	149	D24, D26, D30, D37	M
1658	Min Ops Reduce X Zero	M	Sliding Window	Two ends shrink	30	209	D24, D26, D30, D37	H
540	Single Element in Sorted Array	M	Binary Search	Odd occurrence	20	136	D24, D26, D30, D37	H
1249	Min Remove Valid Parentheses	M	Stack	Balance remove	25	301	D24, D26, D30, D37	H
449	Serialize Deserialize BST	M	Tree	BST codec	30	297	D24, D26, D30, D37	M
1091	Shortest Path Binary Matrix	M	Graph BFS	8-direction	25	200	D24, D26, D30, D37	H
659	Split Array into Consecutive Subsequences	M	Greedy/Heap	Consecutive groups	30	846	D24, D26, D30, D37	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
96	Unique Binary Search Trees	M	DP	Catalan number	25	95	D24, D26, D30, D37	H
731	My Calendar II	M	TreeMap	Double booking	30	729	D24, D26, D30, D37	M
137	Single Number II	M	Bit	Mod 3 bits	25	137	D24, D26, D30, D37	H
609	Find Duplicate File in System	M	HashMap	Content hash	25	49	D24, D26, D30, D37	M
465	Optimal Account Balancing	H	Backtracking	Min transfers	40	1300	D24, D26, D30, D37	M
466	Count The Repetitions	H	DP	String cycle	35	418	D24, D26, D30, D37	M
483	Smallest Good Base	H	Math	Base conversion	40	168	D24, D26, D30, D37	M
499	Max Value of Equation	H	Heap	Line equation	35	149	D24, D26, D30, D37	M
514	Freedom Trail	H	DP	Ring typing	40	72	D24, D26, D30, D37	M
548	Split Array with Equal Sum	H	Prefix Sum	4-partition	35	416	D24, D26, D30, D37	M
587	Erect the Fence	H	Geometry	Convex hull	40	149	D24, D26, D30, D37	M
588	Design In-Memory File System	H	Trie/HashMap	Path trie	45	1166	D24, D26, D30, D37	M
629	K Inverse Pairs Array	H	DP	Inversion count	40	315	D24, D26, D30, D37	M
639	Decode Ways II	H	DP	Wildcard decode	35	91	D24, D26, D30, D37	M
668	Kth Smallest in Lex Order	H	Math	Lex order	35	440	D24, D26, D30, D37	M

Day 25 — July 25, 2026 | Phase: Intensify | 31 problems | Focus: Event-driven patterns bridge Node and Spring — critical for system design depth.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
237	Delete Node in a Linked List	E	Linked List	Copy next	10	19	D25, D27, D31, D38	M
572	Subtree of Another Tree	E	Tree	Subtree match	15	100	D25, D27, D31, D38	H
231	Power of Two	E	Bit	n & (n-1)	10	326	D25, D27, D31, D38	M
100	Same Tree	E	Tree	Structural compare	10	101	D25, D27, D31, D38	H
1197	Min Knight Moves	M	Graph BFS	Chess BFS	25	1091	D25, D27, D31, D38	H
667	Beautiful Arrangement II	M	Greedy	Diff sequence	25	526	D25, D27, D31, D38	M
97	Interleaving String	M	DP	2D boolean DP	30	72	D25, D27, D31, D38	H
1352	Last K Product	M	Design	Product stream	25	295	D25, D27, D31, D38	M
189	Rotate Array	M	Array	Reverse segments	20	61	D25, D27, D31, D38	H
781	Rabbits in Forest	M	HashMap	Group sizing	20	649	D25, D27, D31, D38	M
1695	Max Erasure Value	M	Sliding Window	Unique subarray	25	3	D25, D27, D31, D38	H
658	Find K Closest Elements	M	Binary Search	Window shrink	25	973	D25, D27, D31, D38	H
1762	Buildings Ocean View	M	Monotonic Stack	Right view	25	739	D25, D27, D31, D38	H
513	Find Bottom Left Tree Value	M	Tree BFS	Last left leaf	20	102	D25, D27, D31, D38	M
1466	Reorder Routes	M	Graph DFS	Edge reorient	25	207	D25, D27, D31, D38	H
678	Valid Parenthesis String	M	Greedy	Wildcard paren	25	32	D25, D27, D31, D38	H
120	Triangle	M	DP	Bottom-up min path	20	64	D25, D27, D31, D38	M
1670	Design Front Middle Back Queue	M	Design	Dual deque	30	622	D25, D27, D31, D38	M
201	Bitwise AND of Numbers Range	M	Bit	Common prefix	20	191	D25, D27, D31, D38	M
923	3Sum With Multiplicity	M	HashMap	Count triplets	30	15	D25, D27, D31, D38	H
675	Cut Off Trees for Golf Event	H	BFS	Multi-target BFS	40	994	D25, D27, D31, D38	M
679	24 Game	H	Backtracking	Expression build	35	241	D25, D27, D31, D38	M
691	Stickers to Spell Word	H	Backtracking/DP	Sticker cover	40	638	D25, D27, D31, D38	M
699	Falling Squares	H	Segment Tree	Height timeline	40	218	D25, D27, D31, D38	M
710	Random Pick with Blacklist	H	HashMap	Remap indices	35	398	D25, D27, D31, D38	M
715	Range Module	H	TreeMap	Interval module	40	352	D25, D27, D31, D38	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
730	Count Different Palindromes	H	DP	Distinct palins	45	647	D25, D27, D31, D38	M
732	My Calendar III	H	TreeMap	Triple overlap	35	731	D25, D27, D31, D38	M
736	Parse Lisp Expression	H	Stack/Recursion	Mini interpreter	45	224	D25, D27, D31, D38	M
741	Cherry Pickup	H	DP	Grid two-pass	40	174	D25, D27, D31, D38	M
753	Cracking the Safe	H	Graph/Euler	De Bruijn seq	40	332	D25, D27, D31, D38	M

Day 26 — July 26, 2026 | Phase: Intensify | 32 problems | Focus: WhatsApp design is peak SD practice — focus on messaging at scale.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
101	Symmetric Tree	E	Tree	Mirror check	12	226	D26, D28, D32, D39	H
389	Find the Difference	E	Bit/HashMap	XOR or count	10	136	D26, D28, D32, D39	M
111	Minimum Depth of Binary Tree	E	Tree BFS	First leaf depth	12	104	D26, D28, D32, D39	H
674	Longest Continuous Increasing Subsequence	E	Array	Run length	12	300	D26, D28, D32, D39	M
939	Minimum Area Rectangle	M	HashMap	Diagonal pairs	30	149	D26, D28, D32, D39	H
2009	Min Ops Make Continuous	M	Sliding Window	Sorted window	25	128	D26, D28, D32, D39	H
702	Search in Sorted Array	M	Binary Search	Unknown size	20	278	D26, D28, D32, D39	M
2211	Count Collisions Road	M	Stack	Direction sim	25	735	D26, D28, D32, D39	M
538	Convert BST to Greater Tree	M	Tree	Reverse inorder	20	230	D26, D28, D32, D39	H
1926	Nearest Exit Maze	M	Graph BFS	Grid BFS	25	994	D26, D28, D32, D39	H
738	Monotone Increasing Digits	M	Greedy	Digit decrease	20	402	D26, D28, D32, D39	M
221	Maximal Square	M	DP	Side length DP	25	85	D26, D28, D32, D39	H
204	Count Primes	M	Math	Sieve of Eratosthenes	20	264	D26, D28, D32, D39	M
1296	Divide Array Sets	M	HashMap	K consecutive	25	846	D26, D28, D32, D39	M
981	Time Based Key-Value Store	M	Binary Search	Timestamp lookup	25	362	D26, D28, D32, D39	H
545	Boundary of Binary Tree	M	Tree DFS	Anti-clockwise	25	199	D26, D28, D32, D39	M
769	Max Chunks To Make Sorted	M	Greedy	Max reach chunk	20	768	D26, D28, D32, D39	H
241	Different Ways to Add Parentheses	M	Divide Conquer	Split operators	30	95	D26, D28, D32, D39	M
279	Perfect Squares	M	DP/BFS	Min squares sum	25	322	D26, D28, D32, D39	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
1497	Subarray Sum Divisible	M	HashMap	Mod count	25	974	D26, D28, D32, D39	M
757	Set Intersection Size At Least Two	H	Greedy	Interval cover	35	435	D26, D28, D32, D39	M
768	Max Chunks To Make Sorted II	H	Stack/Greedy	Monotonic chunks	35	763	D26, D28, D32, D39	M
770	Basic Calculator IV	H	Stack	Polynomial eval	45	224	D26, D28, D32, D39	M
772	Basic Calculator III	H	Stack	Nested parens	40	224	D26, D28, D32, D39	M
773	Sliding Puzzle	H	Graph BFS	Board BFS	40	127	D26, D28, D32, D39	M
815	Bus Routes	H	Graph BFS	Route as node	40	127	D26, D28, D32, D39	M
839	Similar String Groups	H	Union Find	Swap similarity	35	721	D26, D28, D32, D39	M
847	Shortest Path All Keys	H	BFS+Bitmask	State compression	45	127	D26, D28, D32, D39	M
854	K Similar Strings	H	BFS	Swap distance	35	72	D26, D28, D32, D39	M
864	Shortest Path All Keys	H	BFS+Bitmask	Keys bitmask	45	847	D26, D28, D32, D39	M
940	Distinct Subsequences II	H	DP	Mod count subseq	35	115	D26, D28, D32, D39	M
996	Number of Squareful Arrays	H	Backtracking	Perfect square adj	40	46	D26, D28, D32, D39	M

Day 27 — July 27, 2026 | Phase: Intensify | 32 problems | Focus: Contract testing + Mongo replication — production readiness topics.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
693	Binary Number with Alternating Bits	E	Bit	Alternate check	12	191	D27, D29, D33	M
747	Largest Number At Least Twice	E	Array	Max/second max	10	215	D27, D29, D33	M
910	Smallest Range I	E	Math	Range shift	12	658	D27, D29, D33	M
1991	Find Middle Index	E	Prefix Sum	Balance index	12	724	D27, D29, D33	M
2131	Longest Palindrome Two Letters	M	HashMap	Pair count	20	409	D27, D29, D33	M
1283	Find Smallest Divisor	M	Binary Search	Threshold sum	25	875	D27, D29, D33	M
549	Binary Tree Longest Consecutive II	M	Tree DFS	Up/down seq	30	298	D27, D29, D33	M
846	Hand of Straights	M	Greedy/HashMap	Consecutive groups	25	1296	D27, D29, D33	M
309	Best Time Buy Sell Cooldown	M	DP	State machine	25	714	D27, D29, D33	H
289	Game of Life	M	Matrix	In-place state	25	73	D27, D29, D33	M
2353	Shared Flavor Pizza	M	HashMap	Set intersection	20	349	D27, D29, D33	M
1539	Kth Missing Positive	M	Binary Search	Missing count	20	41	D27, D29, D33	H
623	Add One Row to Tree	M	Tree BFS	Level insert	20	429	D27, D29, D33	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
1024	Video Stitching	M	Greedy	Min clips cover	30	45	D27, D29, D33	H
375	Guess Number Higher or Lower II	M	DP	Minimax cost	30	464	D27, D29, D33	M
311	Sparse Matrix Multiplication	M	Matrix	Skip zeros	25	73	D27, D29, D33	M
2300	Successful Pairs Spells Potions	M	Binary Search	Pair count	25	875	D27, D29, D33	H
652	Find Duplicate Subtrees	M	Tree	Serialize hash	30	297	D27, D29, D33	H
1642	Furthest Building Can Reach	M	Heap/Greedy	Ladders/bricks	30	871	D27, D29, D33	H
377	Combination Sum IV	M	DP	Order matters	25	518	D27, D29, D33	H
2157	Groups Strings Connected	H	Union Find	One char diff	35	839	D27, D29, D33	M
4	Median of Two Sorted Arrays	H	Binary Search	Partition merge	45	295	D27, D29, D33	VH
23	Merge k Sorted Lists	H	Heap / divide	k-way merge	35	21	D27, D29, D33	VH
25	Reverse Nodes in k-Group	H	Linked List	k-group reverse	35	206	D27, D29, D33	VH
32	Longest Valid Parentheses	H	Stack	Valid paren length	35	20	D27, D29, D33	VH
41	First Missing Positive	H	Arrays	Index placement	30	448	D27, D29, D33	VH
42	Trapping Rain Water	H	Two Pointers	Left/right max	35	407	D27, D29, D33	VH
51	N-Queens	H	Backtracking	Constraint placement	40	52	D27, D29, D33	VH
76	Minimum Window Substring	H	Sliding Window	Cover all chars	35	3	D27, D29, D33	VH
84	Largest Rectangle in Histogram	H	Monotonic Stack	Bar expansion	35	85	D27, D29, D33	VH
124	Binary Tree Maximum Path Sum	H	Tree	Global max path	35	543	D27, D29, D33	VH
127	Word Ladder	H	BFS	Shortest transform	35	126	D27, D29, D33	VH

Day 28 — July 28, 2026 | Phase: Intensify | 33 problems | Focus: Week 4 mock centers on system design — YouTube prep in morning, mock in afternoon.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
1	Two Sum	E	HashMap	Complement lookup	15	167	D28, D30, D34	VH
20	Valid Parentheses	E	Stack	Bracket matching	15	32	D28, D30, D34	VH
21	Merge Two Sorted Lists	E	Linked List	Merge sorted lists	15	23	D28, D30, D34	VH
70	Climbing Stairs	E	DP	Fibonacci pattern	12	746	D28, D30, D34	VH
313	Super Ugly Number	M	Heap/DP	Multi-pointer	30	264	D28, D30, D34	M
654	Maximum Binary Tree	M	Tree	Max build	25	105	D28, D30, D34	M
1899	Merge Triplets Form Target	M	Greedy	Triplet merge	25	39	D28, D30, D34	H
413	Arithmetic Slices	M	DP	Subarray count	20	446	D28, D30, D34	M
316	Remove Duplicate Letters	M	Stack/Greedy	Lex smallest	25	1081	D28, D30, D34	H
662	Maximum Width of Binary Tree	M	Tree BFS	Index width	25	102	D28, D30, D34	H
2279	Max Bags Full Capacity	M	Greedy	Sort fill	20	881	D28, D30, D34	M
427	Construct Quad Tree	M	Divide Conquer	Quad tree build	30	308	D28, D30, D34	M
318	Maximum Product of Word Lengths	M	Bit	Bitmask words	25	191	D28, D30, D34	M
687	Longest Univalue Path	M	Tree DFS	Same value path	25	124	D28, D30, D34	H
486	Predict Winner	M	DP	Minimax game	30	464	D28, D30, D34	M
319	Bulb Switcher	M	Math	Perfect squares	20	1178	D28, D30, D34	M
865	Smallest Subtree All Deepest	M	Tree DFS	Deepest LCA	25	236	D28, D30, D34	M
516	Longest Palindromic Subsequence	M	DP	Subseq palindrome	30	5	D28, D30, D34	H
337	House Robber III	M	Tree DP	Include/exclude	25	198	D28, D30, D34	H
889	Construct BT Preorder Postorder	M	Tree	Divide conquer	30	105	D28, D30, D34	H
576	Out of Boundary Paths	M	DP	Grid paths mod	30	62	D28, D30, D34	M
212	Word Search II	H	Trie + DFS	Board word search	40	79	D28, D30, D34	VH
239	Sliding Window Maximum	H	Monotonic Deque	Window max	30	862	D28, D30, D34	VH
269	Alien Dictionary	H	Topological Sort	Custom order graph	40	210	D28, D30, D34	VH
295	Find Median from Data Stream	H	Heap	Two heap balance	35	480	D28, D30, D34	VH
297	Serialize and Deserialize Binary Tree	H	Tree	Codec design	35	449	D28, D30, D34	VH
329	Longest Increasing Path in a Matrix	H	Graph DFS + memo	Grid longest path	35	300	D28, D30, D34	VH
332	Reconstruct Itinerary	H	Eulerian path	Lexicographic route	35	207	D28, D30, D34	VH
30	Substring with Concatenation of All Words	H	Sliding Window	Multi-word window	40	438	D28, D30, D34	H
37	Sudoku Solver	H	Backtracking	Constraint propagation	45	36	D28, D30, D34	H
85	Maximal Rectangle	H	Monotonic Stack	2D histogram reduce	40	84	D28, D30, D34	H
126	Word Ladder II	H	BFS + backtrack	All shortest paths	45	127	D28, D30, D34	H
312	Burst Balloons	H	DP	Interval DP	40	1039	D28, D30, D34	H

Day 29 — July 29, 2026 | Phase: Peak | 34 problems | Focus: Peak phase starts — architecture layering shows seniority beyond CRUD.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
94	Binary Tree Inorder Traversal	E	Tree	Inorder DFS/iter	15	144	D29, D31, D35	VH
104	Maximum Depth of Binary Tree	E	Tree	Recursive depth	10	111	D29, D31, D35	VH
141	Linked List Cycle	E	Fast/slow	Cycle detection	15	142	D29, D31, D35	VH
206	Reverse Linked List	E	Linked List	Iterative reverse	15	92	D29, D31, D35	VH
338	Counting Bits	M	DP/Bit	DP on bits	20	191	D29, D31, D35	H
919	Complete Binary Tree Inserter	M	Tree BFS	Level fill	30	116	D29, D31, D35	M
583	Delete Operation for Two Strings	M	DP	LCS delete	25	1143	D29, D31, D35	M
341	Flatten Nested List Iterator	M	Stack/DFS	Lazy flatten	30	385	D29, D31, D35	M
673	Number of Longest Increasing Subsequence	M	DP	LIS count	30	300	D29, D31, D35	H
343	Integer Break	M	DP/Math	Max product split	20	279	D29, D31, D35	M
688	Knight Probability in Chessboard	M	DP	Random walk	30	576	D29, D31, D35	M
368	Largest Divisible Subset	M	DP/Sort	Chain divisibility	30	300	D29, D31, D35	M
712	Minimum ASCII Delete Sum	M	DP	LCS variant	30	1143	D29, D31, D35	M
370	Range Addition	M	Prefix Diff	Difference array	20	1109	D29, D31, D35	M
714	Best Time Buy Sell with Fee	M	DP	Transaction fee	25	309	D29, D31, D35	H
372	Super Pow	M	Math/DP	Modular exponent	25	50	D29, D31, D35	M
718	Maximum Length Repeated Subarray	M	DP	Common subarray	30	1143	D29, D31, D35	H
376	Wiggle Subsequence	M	Greedy/DP	Alternating seq	20	300	D29, D31, D35	M
740	Delete and Earn	M	DP	House robber variant	25	198	D29, D31, D35	M
386	Lexicographical Numbers	M	DFS	Lex order nums	25	440	D29, D31, D35	M
790	Domino and Tromino Tiling	M	DP	Board tiling	30	70	D29, D31, D35	M
862	Shortest Subarray Sum K	H	Monotonic Deque	Prefix deque	35	239	D29, D31, D35	H
987	Vertical Order Traversal	H	Tree BFS	Column sort	35	314	D29, D31, D35	H
992	Subarrays K Different Integers	H	Sliding Window	At-most K trick	35	904	D29, D31, D35	H
44	Wildcard Matching	H	DP	Pattern matching	40	10	D29, D31, D35	H
135	Candy	H	Greedy	Two-pass ratings	30	134	D29, D31, D35	H
140	Word Break II	H	Backtracking +DP	All segmentations	40	139	D29, D31, D35	H
154	Find Minimum in Rotated II	H	Binary Search	Dupes in rotated	30	153	D29, D31, D35	H
188	Best Time to Buy and Sell Stock IV	H	DP	k transactions	40	123	D29, D31, D35	H
218	The Skyline Problem	H	Heap	Sweep line	45	699	D29, D31, D35	H
220	Contains Duplicate III	H	Bucket/SortedSet	Window abs diff	35	219	D29, D31, D35	H
224	Basic Calculator	H	Stack	Expression + parens	35	227	D29, D31, D35	H
301	Remove Invalid Parentheses	H	BFS/Backtrack	Min removals	40	22	D29, D31, D35	H
315	Count of Smaller Numbers After Self	H	Merge Sort/BIT	Inversion count	40	493	D29, D31, D35	H

Day 30 — July 30, 2026 | Phase: Peak | 34 problems | Focus: Connect 5yr MERN experience to interview narratives with metrics.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
226	Invert Binary Tree	E	Tree	Mirror swap	12	101	D30, D32, D36	VH
234	Palindrome Linked List	E	Linked List	Find mid + reverse	20	206	D30, D32, D36	VH
242	Valid Anagram	E	HashMap	Frequency count	10	438	D30, D32, D36	VH
252	Meeting Rooms	E	Sorting	Interval overlap check	15	253	D30, D32, D36	VH
390	Elimination Game	M	Math	Josephus variant	25	779	D30, D32, D36	M
799	Champagne Tower	M	DP	Liquid flow	25	790	D30, D32, D36	M
393	UTF-8 Validation	M	Bit	Byte sequence	25	385	D30, D32, D36	M
877	Stone Game	M	DP	Minimax piles	25	486	D30, D32, D36	M
396	Rotate Function	M	Math	Prefix rotation	25	189	D30, D32, D36	M
931	Minimum Falling Path Sum	M	DP	Grid min path	25	64	D30, D32, D36	H
397	Integer Replacement	M	Greedy/DP	Collatz steps	25	343	D30, D32, D36	M
935	Knight Dialer	M	DP	Phone keypad hops	25	688	D30, D32, D36	H
400	Nth Digit	M	Math	Digit sequence	25	386	D30, D32, D36	M
983	Minimum Cost For Tickets	M	DP	Travel planning	30	322	D30, D32, D36	H
402	Remove K Digits	M	Stack/Greedy	Monotonic stack	25	316	D30, D32, D36	H
1014	Best Sightseeing Pair	M	DP	Max score pair	20	121	D30, D32, D36	H
418	Sentence Screen Fitting	M	DP/Greedy	Word wrap cycle	25	68	D30, D32, D36	M
1035	Uncrossed Lines	M	DP	LCS variant	25	1143	D30, D32, D36	H
419	Battleships in a Board	M	Matrix	One-pass count	25	200	D30, D32, D36	M
1155	Number of Dice Rolls Target	M	DP	Combination count	25	518	D30, D32, D36	M
421	Maximum XOR of Two Numbers	M	Trie/Bit	Max XOR pair	30	136	D30, D32, D36	H
354	Russian Doll Envelopes	H	DP/Binary Search	LIS variant	35	300	D30, D32, D36	H
410	Split Array Largest Sum	H	Binary Search	Minimize max sum	35	875	D30, D32, D36	H
460	LFU Cache	H	HashMap+DLL	Freq eviction	45	146	D30, D32, D36	H
480	Sliding Window Median	H	Heap/BST	Window median	40	295	D30, D32, D36	H
493	Reverse Pairs	H	Merge Sort	Inversion variant	40	315	D30, D32, D36	H
632	Smallest Range Covering K Lists	H	Heap/Two Ptr	Range cover	35	373	D30, D32, D36	H
689	Maximum Sum of 3 Non-Overlapping Subarrays	H	DP/Prefix	Three windows	35	53	D30, D32, D36	H
719	Find K-th Smallest Pair Distance	H	Binary Search	Pair distance	40	658	D30, D32, D36	H
871	Minimum Number Refueling Stops	H	Heap/DP	Max heap greedy	35	134	D30, D32, D36	H
980	Unique Paths III	H	Backtracking	Visit all cells	35	62	D30, D32, D36	H
1383	Max Performance Team	H	Greedy/Sort	Speed * efficiency	35	502	D30, D32, D36	H
1851	Min Interval Include Query	H	Binary Search	Interval query	40	56	D30, D32, D36	H
502	IPO	H	Heap	Capital max	35	621	D30, D32, D36	M

Day 31 — July 31, 2026 | Phase: Peak | 35 problems | Focus: Netflix SD + full-stack rapid review — simulate interview pacing.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
543	Diameter of Binary Tree	E	Tree	Longest path	15	124	D31, D33, D37	VH
14	Longest Common Prefix	E	Trie / scan	Vertical scan	15	208	D31, D33, D37	H
26	Remove Duplicates from Sorted Array	E	Two Pointers	In-place overwrite	15	80	D31, D33, D37	H
35	Search Insert Position	E	Binary Search	Lower bound	12	34	D31, D33, D37	H
108	Convert Sorted Array to Binary Search Tree	E	BST	Balanced build	15	109	D31, D33, D37	H
442	Find All Duplicates	M	Array	Index marking	20	448	D31, D33, D37	H
445	Add Two Numbers II	M	Stack/LL	Reverse add	25	2	D31, D33, D37	M
451	Sort Characters By Frequency	M	Heap/Hash Map	Freq sort	20	347	D31, D33, D37	M
453	Minimum Moves Equal Array	M	Math	Increment/decrement	20	462	D31, D33, D37	M
457	Circular Array Loop	M	Fast/Slow	Cycle direction	25	141	D31, D33, D37	M
464	Can I Win	M	DP/Bitmask	Game theory	30	375	D31, D33, D37	M
470	Implement Rand7 via Rand10	M	Math	Rejection sampling	25	382	D31, D33, D37	M
474	Ones and Zeroes	M	DP Knapsack	2D knapsack	30	416	D31, D33, D37	H
477	Total Hamming Distance	M	Bit	Bit contribution	25	461	D31, D33, D37	M
498	Diagonal Traverse	M	Matrix	Diagonal walk	25	1424	D31, D33, D37	M
531	Lonely Pixel I	M	Matrix	Row/col scan	20	533	D31, D33, D37	M
537	Complex Number Multiply	M	Math	Complex arithmetic	15	415	D31, D33, D37	M
556	Next Greater Element III	M	String/Math	Next permutation	25	31	D31, D33, D37	M
638	Shopping Offers	M	Backtracking/DP	Offer bundles	30	39	D31, D33, D37	M
649	Dota2 Senate	M	Queue/Greedy	Ban simulation	25	621	D31, D33, D37	M
650	2 Keys Keyboard	M	DP/Math	Copy paste ops	25	343	D31, D33, D37	M
692	Top K Frequent Words	M	Heap/Trie	Lex tie-break	30	347	D31, D33, D37	H
685	Redundant Connection II	H	Union Find	Directed cycle	35	684	D31, D33, D37	M
759	Employee Free Time	H	Heap/Intervals	Common free	35	253	D31, D33, D37	M
778	Swim in Rising Water	H	Dijkstra/UF	Min max path	35	743	D31, D33, D37	M
1203	Sort Items Groups	H	Topological Sort	Grouped topo	40	269	D31, D33, D37	M
65	Valid Number	H	String	State machine parse	35	8	D31, D33, D37	M
87	Scramble String	H	DP	Recursion + memo	40	44	D31, D33, D37	M
132	Palindrome Partitioning II	H	DP	Min cuts	35	131	D31, D33, D37	M
149	Max Points on a Line	H	HashMap	Slope counting	35	228	D31, D33, D37	M
158	Read N Characters Given Read4 II	H	Array	Multiple calls	30	157	D31, D33, D37	M
174	Dungeon Game	H	DP	Reverse min health	35	64	D31, D33, D37	M
214	Shortest Palindrome	H	String	KMP prefix	35	5	D31, D33, D37	M
296	Best Meeting Point	H	Math	Manhattan median	30	286	D31, D33, D37	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
305	Number of Islands II	H	Union Find	Dynamic connectivity	35	200	D31, D33, D37	M

Day 32 — August 1, 2026 | Phase: Peak | 36 problems | Focus: Crypto and security signing — common in payment and API gateway discussions.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
110	Balanced Binary Tree	E	Tree	Height balance check	15	543	D32, D34, D38	H
125	Valid Palindrome	E	Two Pointers	Alphanumeric filter	12	680	D32, D34, D38	H
144	Binary Tree Preorder Traversal	E	Tree	Preorder DFS/iter	15	94	D32, D34, D38	H
160	Intersection of Two Linked Lists	E	Linked List	Pointer switch trick	15	141	D32, D34, D38	H
278	First Bad Version	E	Binary Search	First true predicate	15	35	D32, D34, D38	H
723	Candy Crush	M	Matrix	Gravity sim	30	289	D32, D34, D38	M
754	Reach a Number	M	Math/Binary Search	Sum target	25	45	D32, D34, D38	M
779	K-th Symbol in Grammar	M	Recursion	Tree path	25	390	D32, D34, D38	M
792	Number of Matching Subsequences	M	HashMap/Trie	Stream match	30	392	D32, D34, D38	H
853	Car Fleet	M	Stack/Sort	Fleet merge	25	735	D32, D34, D38	H
855	Exam Room	M	TreeSet	Seat distance	30	849	D32, D34, D38	M
863	All Nodes Distance K	M	Tree DFS+Graph	Parent pointers	30	236	D32, D34, D38	M
866	Prime Palindrome	M	Math	Palindrome prime	25	204	D32, D34, D38	M
921	Minimum Add Parentheses	M	Stack/Greedy	Balance parens	20	32	D32, D34, D38	H
926	Flip String to Monotone Increasing	M	DP/Prefix	Min flips	25	152	D32, D34, D38	H
1048	Longest String Chain	M	DP/Sort	Word chain	25	139	D32, D34, D38	H
1094	Car Pooling	M	Prefix Diff	Capacity timeline	25	253	D32, D34, D38	H
1109	Corporate Flight Bookings	M	Prefix Diff	Range update	20	370	D32, D34, D38	M
1481	Least Unique Integers	M	Heap/HashMap	K unique remove	25	347	D32, D34, D38	M
3	Longest Substring Without Repeating Characters	M	Sliding Window	Unique char window	25	159	D32, D34, D38	VH
5	Longest Palindromic Substring	M	Expand center	Palindrome expansion	30	647	D32, D34, D38	VH
11	Container With Most Water	M	Two Pointers	Greedy shrink	20	42	D32, D34, D38	VH
15	3Sum	M	Two Pointers	Sorted triplets	30	18	D32, D34, D38	VH
308	Range Sum Query 2D Mutable	H	Segment Tree	2D fenwick	40	307	D32, D34, D38	M
317	Shortest Distance from All Buildings	H	Graph BFS	Multi-source sum	40	286	D32, D34, D38	M
327	Count of Range Sum	H	Merge Sort	Range count	45	315	D32, D34, D38	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
330	Patching Array	H	Greedy	Missing coverage	35	45	D32, D34, D38	M
352	Data Stream as Disjoint Intervals	H	Binary Search	Merge intervals	35	56	D32, D34, D38	M
358	Rearrange String k Distance Apart	H	Heap/Greedy	K distance apart	35	767	D32, D34, D38	M
363	Max Sum of Rectangle No Larger Than K	H	TreeSet/DP	Submatrix sum	40	53	D32, D34, D38	M
381	Insert Delete GetRandom O(1) Duplicates	H	HashMap+Set	Dup allowed	35	380	D32, D34, D38	M
391	Perfect Rectangle	H	HashMap	Area + corners	35	85	D32, D34, D38	M
403	Frog Jump	H	DP/HashSet	Stone jump	35	55	D32, D34, D38	M
407	Trapping Rain Water II	H	Heap/BFS	3D rain water	40	42	D32, D34, D38	M
420	Strong Password Checker	H	Greedy	Edit distance rules	35	72	D32, D34, D38	M
432	All O`one Data Structure	H	HashMap+DLL	Freq buckets	40	460	D32, D34, D38	M

Day 33 — August 2, 2026 | Phase: Peak | 37 problems | Focus: Coding implementation day — LRU and Uber design.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
283	Move Zeroes	E	Two Pointers	In-place compaction	12	27	D33, D35, D39	H
496	Next Greater Element I	E	Monotonic Stack	NGE template	15	739	D33, D35, D39	H
572	Subtree of Another Tree	E	Tree	Subtree match	15	100	D33, D35, D39	H
680	Valid Palindrome II	E	Two Pointers	One deletion allowed	15	125	D33, D35, D39	H
746	Min Cost Climbing Stairs	E	DP	Stair cost	15	70	D33, D35, D39	H
17	Letter Combinations of a Phone Number	M	Backtracking	Choice tree DFS	25	22	D33, D35, D39	VH
19	Remove Nth Node From End of List	M	Linked List	Two-pointer gap	20	206	D33, D35, D39	VH
22	Generate Parentheses	M	Backtracking	Valid paren generation	25	301	D33, D35, D39	VH
33	Search in Rotated Sorted Array	M	Binary Search	Pivot detection	25	81	D33, D35, D39	VH
34	Find First and Last Position of Element in Sorted Array	M	Binary Search	Boundary search	25	35	D33, D35, D39	VH
39	Combination Sum	M	Backtracking	Unbounded combinations	25	40	D33, D35, D39	VH
45	Jump Game II	M	Greedy	Min jumps	25	55	D33, D35, D39	VH
46	Permutations	M	Backtracking	Permutation generation	25	47	D33, D35, D39	VH
49	Group Anagrams	M	HashMap	Sorted key grouping	20	242	D33, D35, D39	VH
53	Maximum Subarray	M	Kadane	DP / greedy subarray	20	152	D33, D35, D39	VH
55	Jump Game	M	Greedy	Reachability	20	45	D33, D35, D39	VH
56	Merge Intervals	M	Sorting	Interval merge	25	57	D33, D35, D39	VH
57	Insert Interval	M	Intervals	Merge on insert	25	56	D33, D35, D39	VH

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
72	Edit Distance	M	DP	Levenshtein distance	30	1143	D33, D35, D39	VH
75	Sort Colors	M	Dutch Flag	Three-way partition	20	215	D33, D35, D39	VH
78	Subsets	M	Backtracking	Subset enumeration	20	90	D33, D35, D39	VH
79	Word Search	M	Backtracking	Grid path search	25	212	D33, D35, D39	VH
98	Validate Binary Search Tree	M	BST	Range validation	25	230	D33, D35, D39	VH
440	K-th Smallest in Lex Order	H	Math/DFS	Lex order nums	35	386	D33, D35, D39	M
446	Arithmetic Slices II Subsequence	H	DP	Subseq AP count	40	413	D33, D35, D39	M
458	Poor Pigs	H	Math	Information theory	30	319	D33, D35, D39	M
465	Optimal Account Balancing	H	Backtracking	Min transfers	40	1300	D33, D35, D39	M
466	Count The Repetitions	H	DP	String cycle	35	418	D33, D35, D39	M
483	Smallest Good Base	H	Math	Base conversion	40	168	D33, D35, D39	M
499	Max Value of Equation	H	Heap	Line equation	35	149	D33, D35, D39	M
514	Freedom Trail	H	DP	Ring typing	40	72	D33, D35, D39	M
548	Split Array with Equal Sum	H	Prefix Sum	4-partition	35	416	D33, D35, D39	M
587	Erect the Fence	H	Geometry	Convex hull	40	149	D33, D35, D39	M
588	Design In-Memory File System	H	Trie/HashMap	Path trie	45	1166	D33, D35, D39	M
629	K Inverse Pairs Array	H	DP	Inversion count	40	315	D33, D35, D39	M
639	Decode Ways II	H	DP	Wildcard decode	35	91	D33, D35, D39	M
668	Kth Smallest in Lex Order	H	Math	Lex order	35	440	D33, D35, D39	M

Day 34 — August 3, 2026 | Phase: Peak | 38 problems | Focus: Translate real experience into compelling interview stories.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
876	Middle of the Linked List	E	Fast/slow	Midpoint detection	10	141	D34, D36	H
28	Find the Index of the First Occurrence	E	String	KMP / built-in	15	214	D34, D36	H
88	Merge Sorted Array	E	Two Pointers	Fill from end	15	977	D34, D36	H
100	Same Tree	E	Tree	Structural compare	10	101	D34, D36	H
101	Symmetric Tree	E	Tree	Mirror check	12	226	D34, D36	H
102	Binary Tree Level Order Traversal	M	BFS	Level-by-level	20	107	D34, D36	VH
128	Longest Consecutive Sequence	M	HashSet	Sequence start detection	25	298	D34, D36	VH
133	Clone Graph	M	Graph BFS/DFS	Graph deep copy	25	138	D34, D36	VH
138	Copy List with Random Pointer	M	Linked List	HashMap clone	30	133	D34, D36	VH
139	Word Break	M	DP	Segmentation DP	25	140	D34, D36	VH
142	Linked List Cycle II	M	Fast/slow	Cycle entry point	25	141	D34, D36	VH
146	LRU Cache	M	HashMap + DLL	Ordered eviction	35	460	D34, D36	VH
152	Maximum Product Subarray	M	DP	Min/max product track	25	53	D34, D36	VH
153	Find Minimum in Rotated Sorted Array	M	Binary Search	Min in rotated	20	154	D34, D36	VH
155	Min Stack	M	Stack	O(1) min	20	716	D34, D36	VH
167	Two Sum II - Input Array Is Sorted	M	Two Pointers	Sorted pair search	15	1	D34, D36	VH
198	House Robber	M	DP	Linear choice DP	20	213	D34, D36	VH
200	Number of Islands	M	Graph DFS/BFS	Connected components	25	695	D34, D36	VH
207	Course Schedule	M	Graph DFS	Cycle detection	25	210	D34, D36	VH
208	Implement Trie (Prefix Tree)	M	Trie	Prefix tree ops	25	211	D34, D36	VH
209	Minimum Size Subarray Sum	M	Sliding Window	Shrinkable window	25	3	D34, D36	VH
210	Course Schedule II	M	Topological Sort	Order construction	30	207	D34, D36	VH
213	House Robber II	M	DP	Circular robber	25	198	D34, D36	VH
215	Kth Largest Element in an Array	M	Quickselect / heap	Partition selection	25	347	D34, D36	VH
675	Cut Off Trees for Golf Event	H	BFS	Multi-target BFS	40	994	D34, D36	M
679	24 Game	H	Backtracking	Expression build	35	241	D34, D36	M
691	Stickers to Spell Word	H	Backtracking /DP	Sticker cover	40	638	D34, D36	M
699	Falling Squares	H	Segment Tree	Height timeline	40	218	D34, D36	M
710	Random Pick with Blacklist	H	HashMap	Remap indices	35	398	D34, D36	M
715	Range Module	H	TreeMap	Interval module	40	352	D34, D36	M
730	Count Different Palindromes	H	DP	Distinct palins	45	647	D34, D36	M
732	My Calendar III	H	TreeMap	Triple overlap	35	731	D34, D36	M
736	Parse Lisp Expression	H	Stack/Recursion	Mini interpreter	45	224	D34, D36	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
741	Cherry Pickup	H	DP	Grid two-pass	40	174	D34, D36	M
753	Cracking the Safe	H	Graph/Euler	De Bruijn seq	40	332	D34, D36	M
757	Set Intersection Size At Least Two	H	Greedy	Interval cover	35	435	D34, D36	M
768	Max Chunks To Make Sorted II	H	Stack/Greedy	Monotonic chunks	35	763	D34, D36	M
770	Basic Calculator IV	H	Stack	Polynomial eval	45	224	D34, D36	M

Day 35 — August 4, 2026 | Phase: Peak | 38 problems | Focus: Week 5 final mock — treat as real FAANG panel; Food Delivery SD in morning.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
111	Minimum Depth of Binary Tree	E	Tree BFS	First leaf depth	12	104	D35, D37	H
136	Single Number	E	Bit	XOR cancel	10	137	D35, D37	H
191	Number of 1 Bits	E	Bit	Hamming weight	10	338	D35, D37	H
219	Contains Duplicate II	E	HashMap	Index distance	12	220	D35, D37	H
392	Is Subsequence	E	Two Pointers	Greedy match	12	1143	D35, D37	H
230	Kth Smallest in BST	M	BST	Inorder kth	20	98	D35, D37	VH
235	Lowest Common Ancestor of a BST	M	BST	BST LCA walk	15	236	D35, D37	VH
236	Lowest Common Ancestor of a Binary Tree	M	Tree	LCA postorder	25	235	D35, D37	VH
238	Product of Array Except Self	M	Prefix/Suffix	No-division product	25	42	D35, D37	VH
253	Meeting Rooms II	M	Greedy / heap	Min rooms needed	25	252	D35, D37	VH
300	Longest Increasing Subsequence	M	DP	LIS O(n log n)	30	674	D35, D37	VH
322	Coin Change	M	DP	Min coins	25	518	D35, D37	VH
347	Top K Frequent Elements	M	Heap/Bucket	Frequency top-k	25	692	D35, D37	VH
416	Partition Equal Subset Sum	M	DP	0/1 knapsack	30	494	D35, D37	VH
424	Longest Repeating Character Replacement	M	Sliding Window	Max freq	30	1004	D35, D37	VH
435	Non-overlapping Intervals	M	Greedy	Interval scheduling	25	452	D35, D37	VH
494	Target Sum	M	DP	+/- subset	30	416	D35, D37	VH
547	Number of Provinces	M	Union Find	Connected comps	25	200	D35, D37	VH
560	Subarray Sum Equals K	M	HashMap	Prefix count	25	974	D35, D37	VH
621	Task Scheduler	M	Greedy/Heap	Cooldown slots	25	767	D35, D37	VH
721	Accounts Merge	M	Union Find	Email merge	30	547	D35, D37	VH
739	Daily Temperatures	M	Monotonic Stack	NGE template	25	496	D35, D37	VH
743	Network Delay Time	M	Dijkstra	Shortest path	30	787	D35, D37	VH
787	Cheapest Flights K Stops	M	Bellman-Ford	K-stop path	35	743	D35, D37	VH
772	Basic Calculator III	H	Stack	Nested parens	40	224	D35, D37	M
773	Sliding Puzzle	H	Graph BFS	Board BFS	40	127	D35, D37	M
815	Bus Routes	H	Graph BFS	Route as node	40	127	D35, D37	M
839	Similar String Groups	H	Union Find	Swap similarity	35	721	D35, D37	M

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
847	Shortest Path All Keys	H	BFS+Bitmask	State compression	45	127	D35, D37	M
854	K Similar Strings	H	BFS	Swap distance	35	72	D35, D37	M
864	Shortest Path All Keys	H	BFS+Bitmask	Keys bitmask	45	847	D35, D37	M
940	Distinct Subsequences II	H	DP	Mod count subseq	35	115	D35, D37	M
996	Number of Squareful Arrays	H	Backtracking	Perfect square adj	40	46	D35, D37	M
2157	Groups Strings Connected	H	Union Find	One char diff	35	839	D35, D37	M
4	Median of Two Sorted Arrays	H	Binary Search	Partition merge	45	295	D35, D37	VH
23	Merge k Sorted Lists	H	Heap / divide	k-way merge	35	21	D35, D37	VH
25	Reverse Nodes in k-Group	H	Linked List	k-group reverse	35	206	D35, D37	VH
32	Longest Valid Parentheses	H	Stack	Valid paren length	35	20	D35, D37	VH

Day 36 — August 5, 2026 | Phase: Final Sprint | 38 problems | Focus: Final Sprint — fix mock gaps only, no new topics.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
703	Kth Largest in Stream	E	Heap	Min heap k	15	215	D36, D38	H
27	Remove Element	E	Two Pointers	Partition array	12	283	D36, D38	M
69	Sqrt(x)	E	Binary Search	Integer sqrt	12	367	D36, D38	M
225	Implement Stack using Queues	E	Queue	Stack from queues	15	232	D36, D38	M
232	Implement Queue using Stacks	E	Queue	Queue from stacks	15	225	D36, D38	M
875	Koko Eating Bananas	M	Binary Search	Answer space	25	1011	D36, D38	VH
973	K Closest Points to Origin	M	Heap	k nearest	20	215	D36, D38	VH
994	Rotting Oranges	M	Graph BFS	Multi-source	25	286	D36, D38	VH
1004	Max Consecutive Ones III	M	Sliding Window	Flip K zeros	25	485	D36, D38	VH
1143	Longest Common Subsequence	M	DP	2D string DP	25	72	D36, D38	VH
2	Add Two Numbers	M	Linked List	Digit carry chain	25	445	D36, D38	VH
62	Unique Paths	M	DP	Grid combinatorics	20	63	D36, D38	VH
105	Construct BT from Preorder Inorder	M	Tree	Divide by root	30	106	D36, D38	VH
16	3Sum Closest	M	Two Pointers	Closest sum target	25	15	D36, D38	H
18	4Sum	M	Two Pointers	k-sum extension	35	454	D36, D38	H
36	Valid Sudoku	M	HashSet	Row/col/box sets	25	37	D36, D38	H
74	Search a 2D Matrix	M	Binary Search	Flattened search	20	240	D36, D38	H
92	Reverse Linked List II	M	Linked List	Partial reverse	25	206	D36, D38	H
130	Surrounded Regions	M	Graph DFS	Boundary capture	25	417	D36, D38	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
131	Palindrome Partitioning	M	Backtracking	Partition + palin check	30	5	D36, D38	H
134	Gas Station	M	Greedy	Circuit feasibility	25	135	D36, D38	H
159	Longest Substring with At Most Two Distinct Characters	M	Sliding Window	Distinct limit	25	340	D36, D38	H
162	Find Peak Element	M	Binary Search	Peak finding	20	852	D36, D38	H
211	Design Add and Search Words Data Structure	M	Trie	Wildcard search	30	212	D36, D38	H
41	First Missing Positive	H	Arrays	Index placement	30	448	D36, D38	VH
42	Trapping Rain Water	H	Two Pointers	Left/right max	35	407	D36, D38	VH
51	N-Queens	H	Backtracking	Constraint placement	40	52	D36, D38	VH
76	Minimum Window Substring	H	Sliding Window	Cover all chars	35	3	D36, D38	VH
84	Largest Rectangle in Histogram	H	Monotonic Stack	Bar expansion	35	85	D36, D38	VH
124	Binary Tree Maximum Path Sum	H	Tree	Global max path	35	543	D36, D38	VH
127	Word Ladder	H	BFS	Shortest transform	35	126	D36, D38	VH
212	Word Search II	H	Trie + DFS	Board word search	40	79	D36, D38	VH
239	Sliding Window Maximum	H	Monotonic Deque	Window max	30	862	D36, D38	VH
269	Alien Dictionary	H	Topological Sort	Custom order graph	40	210	D36, D38	VH
295	Find Median from Data Stream	H	Heap	Two heap balance	35	480	D36, D38	VH
297	Serialize and Deserialize Binary Tree	H	Tree	Codec design	35	449	D36, D38	VH
329	Longest Increasing Path in a Matrix	H	Graph DFS + memo	Grid longest path	35	300	D36, D38	VH
332	Reconstruct Itinerary	H	Eulerian path	Lexicographic route	35	207	D36, D38	VH

Day 37 — August 6, 2026 | Phase: Final Sprint | 38 problems | Focus: Teach-back method — if you can teach it, interview answers flow naturally.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
344	Reverse String	E	Two Pointers	In-place swap	8	541	D37, D39	M
346	Moving Average from Data Stream	E	Queue	Fixed-size average	15	933	D37, D39	M
448	Find Disappeared Numbers	E	Array	Index marking	20	442	D37, D39	M
485	Max Consecutive Ones	E	Linear scan	Run length	10	1004	D37, D39	M
530	Minimum Absolute Difference in BST	E	BST	Inorder adjacent	15	783	D37, D39	M
227	Basic Calculator II	M	Stack	Expression parsing	30	224	D37, D39	H
310	Minimum Height Trees	M	Topological Sort	Leaf pruning	30	210	D37, D39	H
340	Longest Substring with At Most K Distinct Characters	M	Sliding Window	K distinct limit	25	159	D37, D39	H
380	Insert Delete GetRandom O(1)	M	HashMap+Array	Swap delete	30	381	D37, D39	H
394	Decode String	M	Stack	Nested decode	25	20	D37, D39	H
406	Queue Reconstruction by Height	M	Greedy	Sort insert	25	763	D37, D39	H
417	Pacific Atlantic Water Flow	M	Graph DFS	Multi-source	30	130	D37, D39	H
438	Find All Anagrams	M	Sliding Window	Fixed window	25	567	D37, D39	H
450	Delete Node in a BST	M	BST	BST deletion	25	98	D37, D39	H
452	Min Arrows Burst Balloons	M	Greedy	End-point sort	25	435	D37, D39	H
518	Coin Change II	M	DP	Combination count	25	322	D37, D39	H
567	Permutation in String	M	Sliding Window	Anagram window	25	438	D37, D39	H
647	Palindromic Substrings	M	Expand	Count palindromes	25	5	D37, D39	H
684	Redundant Connection	M	Union Find	Cycle edge	25	685	D37, D39	H
695	Max Area of Island	M	Graph DFS	Component area	20	200	D37, D39	H
713	Subarray Product Less Than K	M	Sliding Window	Product window	25	209	D37, D39	H
763	Partition Labels	M	Greedy	Last occurrence	20	856	D37, D39	H
767	Reorganize String	M	Heap	No adjacent dup	25	621	D37, D39	H
802	Find Eventual Safe States	M	Topological Sort	Safe nodes	30	207	D37, D39	H
30	Substring with Concatenation of All Words	H	Sliding Window	Multi-word window	40	438	D37, D39	H
37	Sudoku Solver	H	Backtracking	Constraint propagation	45	36	D37, D39	H
85	Maximal Rectangle	H	Monotonic Stack	2D histogram reduce	40	84	D37, D39	H
126	Word Ladder II	H	BFS + backtrack	All shortest paths	45	127	D37, D39	H
312	Burst Balloons	H	DP	Interval DP	40	1039	D37, D39	H
862	Shortest Subarray Sum K	H	Monotonic Deque	Prefix deque	35	239	D37, D39	H
987	Vertical Order Traversal	H	Tree BFS	Column sort	35	314	D37, D39	H
992	Subarrays K Different Integers	H	Sliding Window	At-most K trick	35	904	D37, D39	H
44	Wildcard Matching	H	DP	Pattern matching	40	10	D37, D39	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
135	Candy	H	Greedy	Two-pass ratings	30	134	D37, D39	H
140	Word Break II	H	Backtracking +DP	All segmentations	40	139	D37, D39	H
154	Find Minimum in Rotated II	H	Binary Search	Dupes in rotated	30	153	D37, D39	H
188	Best Time to Buy and Sell Stock IV	H	DP	k transactions	40	123	D37, D39	H
218	The Skyline Problem	H	Heap	Sweep line	45	699	D37, D39	H

Day 38 — August 7, 2026 | Phase: Final Sprint | 38 problems | Focus: Rest and recall — protect sleep; cognitive freshness beats cramming.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
704	Binary Search	E	Binary Search	Classic template	12	35	D38	M
724	Find Pivot Index	E	Prefix Sum	Balance index	15	1991	D38	M
733	Flood Fill	E	Graph DFS/BFS	Paint fill	15	200	D38	M
860	Lemonade Change	E	Greedy	Change making	15	406	D38	M
922	Sort Array By Parity	E	Two Pointers	Even first	10	905	D38	M
881	Boats to Save People	M	Two Pointers	Greedy pair	20	167	D38	H
904	Fruit Into Baskets	M	Sliding Window	At most 2 distinct	25	992	D38	H
974	Subarray Sums Divisible by K	M	HashMap	Mod prefix	25	560	D38	H
986	Interval List Intersections	M	Two Pointers	Merge intervals	25	56	D38	H
990	Satisfiability Equality Equations	M	Union Find	Var union	25	684	D38	H
1011	Capacity Ship Packages	M	Binary Search	Feasibility	30	875	D38	H
1268	Search Suggestions System	M	Trie	Autocomplete	25	208	D38	H
1584	Min Cost Connect Points	M	MST	Manhattan MST	30	743	D38	H
1631	Path Minimum Effort	M	Dijkstra	Min max edge	30	778	D38	H
8	String to Integer (atoi)	M	String	Overflow handling	25	65	D38	H
31	Next Permutation	M	Array	In-place rearrange	25	60	D38	H
40	Combination Sum II	M	Backtracking	No reuse combos	25	39	D38	H
47	Permutations II	M	Backtracking	Duplicate handling	25	46	D38	H
48	Rotate Image	M	Matrix	Transpose + reverse	20	54	D38	H
50	Pow(x, n)	M	Binary Search	Fast exponentiation	20	372	D38	H
54	Spiral Matrix	M	Matrix	Boundary walk	25	59	D38	H
63	Unique Paths II	M	DP	Obstacle grid	20	64	D38	H
64	Minimum Path Sum	M	DP	Grid min sum	20	120	D38	H
73	Set Matrix Zeroes	M	Matrix	In-place markers	25	289	D38	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
220	Contains Duplicate III	H	Bucket/SortedSet	Window abs diff	35	219	D38	H
224	Basic Calculator	H	Stack	Expression + parens	35	227	D38	H
301	Remove Invalid Parentheses	H	BFS/Backtrack	Min removals	40	22	D38	H
315	Count of Smaller Numbers After Self	H	Merge Sort/BIT	Inversion count	40	493	D38	H
354	Russian Doll Envelopes	H	DP/Binary Search	LIS variant	35	300	D38	H
410	Split Array Largest Sum	H	Binary Search	Minimize max sum	35	875	D38	H
460	LFU Cache	H	HashMap+DLL	Freq eviction	45	146	D38	H
480	Sliding Window Median	H	Heap/BST	Window median	40	295	D38	H
493	Reverse Pairs	H	Merge Sort	Inversion variant	40	315	D38	H
632	Smallest Range Covering K Lists	H	Heap/Two Ptr	Range cover	35	373	D38	H
689	Maximum Sum of 3 Non-Overlapping Subarrays	H	DP/Prefix	Three windows	35	53	D38	H
719	Find K-th Smallest Pair Distance	H	Binary Search	Pair distance	40	658	D38	H
871	Minimum Number Refueling Stops	H	Heap/DP	Max heap greedy	35	134	D38	H
980	Unique Paths III	H	Backtracking	Visit all cells	35	62	D38	H

Day 39 — August 8, 2026 | Phase: Final Sprint | 38 problems | Focus: Day before interviews — rest, light review, logistics, confidence.

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
933	Number of Recent Calls	E	Queue	Sliding window queue	12	346	D39	M
977	Squares of Sorted Array	E	Two Pointers	Merge ends	12	88	D39	M
1046	Last Stone Weight	E	Heap	Max-heap simulation	15	295	D39	M
13	Roman to Integer	E	String	Pair parsing	15	12	D39	M
157	Read N Characters Given Read4	E	Array	Buffer read	15	158	D39	M
81	Search in Rotated Sorted Array II	M	Binary Search	Duplicates	25	33	D39	H
90	Subsets II	M	Backtracking	Sorted dedupe	25	78	D39	H
91	Decode Ways	M	DP	String segmentation	25	639	D39	H
96	Unique Binary Search Trees	M	DP	Catalan number	25	95	D39	H
97	Interleaving String	M	DP	2D boolean DP	30	72	D39	H
103	Binary Tree Zigzag Level Order	M	Tree BFS	Alternate direction	25	102	D39	H
106	Construct BT from Inorder Postorder	M	Tree	Postorder root	30	105	D39	H
113	Path Sum II	M	Tree DFS	All root-leaf paths	25	437	D39	H
114	Flatten Binary Tree to Linked List	M	Tree	Morris / reverse	30	430	D39	H
137	Single Number II	M	Bit	Mod 3 bits	25	137	D39	H
143	Reorder List	M	Linked List	Mid reverse merge	30	234	D39	H

#	Title	Diff	Pattern	Concept	Min	Follow-up	Revision	Freq
148	Sort List	M	Linked List	Merge sort LL	35	21	D39	H
150	Evaluate Reverse Polish Notation	M	Stack	RPN eval	20	224	D39	H
151	Reverse Words in a String	M	String	Two-pass reverse	20	186	D39	H
173	Binary Search Tree Iterator	M	Stack	Controlled inorder	25	230	D39	H
189	Rotate Array	M	Array	Reverse segments	20	61	D39	H
199	Binary Tree Right Side View	M	Tree BFS	Last per level	20	545	D39	H
221	Maximal Square	M	DP	Side length DP	25	85	D39	H
279	Perfect Squares	M	DP/BFS	Min squares sum	25	322	D39	H
1383	Max Performance Team	H	Greedy/Sort	Speed * efficiency	35	502	D39	H
1851	Min Interval Include Query	H	Binary Search	Interval query	40	56	D39	H
502	IPO	H	Heap	Capital max	35	621	D39	M
685	Redundant Connection II	H	Union Find	Directed cycle	35	684	D39	M
759	Employee Free Time	H	Heap/Intervals	Common free	35	253	D39	M
778	Swim in Rising Water	H	Dijkstra/UF	Min max path	35	743	D39	M
1203	Sort Items Groups	H	Topological Sort	Grouped topo	40	269	D39	M
65	Valid Number	H	String	State machine parse	35	8	D39	M
87	Scramble String	H	DP	Recursion + memo	40	44	D39	M
132	Palindrome Partitioning II	H	DP	Min cuts	35	131	D39	M
149	Max Points on a Line	H	HashMap	Slope counting	35	228	D39	M
158	Read N Characters Given Read4 II	H	Array	Multiple calls	30	157	D39	M
174	Dungeon Game	H	DP	Reverse min health	35	64	D39	M
214	Shortest Palindrome	H	String	KMP prefix	35	5	D39	M

Section 4: Node.js Backend Curriculum

Node.js is your primary strength — this curriculum ensures interview-crisp depth on event loop, production patterns, and backend architecture beyond CRUD APIs.

Core Topic Map

- Event loop, libuv, and non-blocking I/O architecture
- CommonJS vs ESM modules and module resolution
- Streams API — readable, writable, duplex, transform, backpressure
- Express.js middleware chain, routing, and error handling
- Async patterns — callbacks, Promises, async/await
- Error handling — operational vs programmer errors, graceful shutdown
- Authentication — JWT, sessions, OAuth 2.0, refresh tokens
- Caching strategies — Redis, in-memory, cache-aside, TTL policies
- Database integration — Mongoose ODM, connection pooling, transactions
- Testing — Jest, supertest, mocking, integration test patterns
- Security — OWASP Top 10, input validation, rate limiting, CORS
- Performance — profiling, clustering, worker threads, load balancing
- Microservices — service discovery, API gateway, circuit breakers
- Message queues — Bull/BullMQ, RabbitMQ, Kafka basics
- Production ops — logging (pino), monitoring, health checks, PM2

39-Day Node.js Daily Curriculum

Day	Date	Topic	Exercise
1	July 1, 2026	Node.js Event Loop, libuv, and non-blocking I/O architecture	Implement a task scheduler using setImmediate, process.nextT
2	July 2, 2026	CommonJS vs ESM modules, require vs import, and module resolution	Build a mini plugin loader that dynamically imports ESM modu
3	July 3, 2026	Streams API: readable, writable, duplex, transform, backpressure	Build a file pipeline: gzip compress a large log file using
4	July 4, 2026	Express.js middleware chain, routing, and error-handling patterns	Create an Express API with auth middleware, request logging,
5	July 5, 2026	Async patterns: callbacks, Promises, async/await, and error propagation	Convert a callback-based fs utility library to Promises, the
6	July 6, 2026	Error handling: operational vs programmer errors, domains (legacy), and graceful	Add SIGTERM/SIGINT handlers, drain in-flight requests, close
7	July 7, 2026	Node.js cluster module and worker threads for CPU-bound tasks	Build a cluster-based HTTP server that distributes requests
8	July 8, 2026	Authentication: JWT, sessions, OAuth2/OIDC flows, and refresh token rotation	Implement JWT auth with access/refresh tokens, rotation on r
9	July 9, 2026	Validation, serialization, and API design: REST conventions, status codes, pagin	Build a paginated REST CRUD for products with Joi/Zod valida
10	July 10, 2026	Testing Node apps: Jest/Vitest, supertest, mocking, and TDD workflow	Write unit tests for a service layer and integration tests f
11	July 11, 2026	MongoDB with Mongoose: schemas, middleware, population, and transactions	Build a blog API with Mongoose schemas, pre-save hooks, popu
12	July 12, 2026	Redis caching: cache-aside, write-through, TTL strategies, and cache invalidatio	Add Redis cache-aside to product API; implement cache stampe
13	July 13, 2026	Rate limiting and throttling: token bucket, sliding window, express-rate-limit,	Implement distributed rate limiter using Redis sliding windo
14	July 14, 2026	Message queues with Bull/BullMQ and RabbitMQ: producers, consumers, retries, DLQ	Build email notification worker using BullMQ with retry back
15	July 15, 2026	File uploads: multipart, streaming uploads, S3 presigned URLs, and virus scan ho	Implement direct-to-S3 upload flow with presigned URLs and m

Day	Date	Topic	Exercise
16	July 16, 2026	WebSockets and Socket.io: rooms, namespaces, scaling with Redis adapter	Build real-time chat room with Socket.io, JWT auth on handsh
17	July 17, 2026	Logging and observability: structured logging (pino/winston), correlation IDs, l	Add pino structured logging with request correlation ID midd
18	July 18, 2026	Security best practices: OWASP Top 10, helmet, CORS, CSRF, XSS, SQL/NoSQL inject	Harden an Express API: helmet headers, parameterized queries
19	July 19, 2026	Microservices in Node: service boundaries, inter-service communication, API cont	Split monolith into user-service and order-service communica
20	July 20, 2026	GraphQL with Node: schema design, resolvers, DataLoader, and N+1 prevention	Build GraphQL API for blog with Query/Mutation, DataLoader f
21	July 21, 2026	Performance tuning Node: profiling, clinic.js, memory leaks, and V8 optimization	Profile a slow endpoint with clinic doctor; fix memory leak
22	July 22, 2026	Design patterns in Node: factory, strategy, observer, middleware as composite	Refactor notification system using strategy pattern for emai
23	July 23, 2026	TypeScript for Node backends: strict mode, generics, decorators, and type-safe A	Migrate Express JS project to TypeScript with strict null ch
24	July 24, 2026	Serverless Node: AWS Lambda, cold starts, API Gateway, and serverless framework	Deploy Express app as Lambda with serverless-http; optimize
25	July 25, 2026	Event-driven architecture: EventEmitter, domain events, event sourcing intro, CQ	Implement order flow publishing OrderCreated/OrderShipped ev
26	July 26, 2026	Database connection management in Node: pg pool, Mongoose connections, health ch	Configure pg Pool and Mongoose with connection limits, retry
27	July 27, 2026	API documentation and contract testing: OpenAPI/Swagger, Pact, and consumer-driv	Generate OpenAPI spec from Express routes; write Pact consum
28	July 28, 2026	Node.js internals review: V8, libuv thread pool, C++ addons overview, and N-API	Write a simple N-API addon that computes fibonacci synchrono
29	July 29, 2026	Senior Node topics: hexagonal architecture, DDD tactical patterns, and clean arc	Restructure a CRUD app into domain/use-case/infrastructure l
30	July 30, 2026	Node production war stories prep: scaling MERN apps, lessons learned, and archit	Write architecture decision record (ADR) for choosing MongoD
31	July 31, 2026	Full Node backend mock review: synthesize event loop, Express, auth, cache, queu	Timed 45-min build: REST API with auth, Redis cache, and Bul
32	August 1, 2026	Low-level Node: Buffer, crypto module, streams performance, and binary protocols	Implement HMAC-SHA256 request signing middleware for API aut
33	August 2, 2026	Interview coding in Node: parse JSON streams, implement LRU cache, design in-mem	Implement LRU cache from scratch with O(1) get/put using Map
34	August 3, 2026	Behavioral + technical combo: explain past projects as system design stories	Prepare 3 project deep-dives mapping to SD components: API,
35	August 4, 2026	Node.js final review: 50 rapid-fire conceptual questions self-drill	Complete 50-question Node checklist; flag any answer taking
36	August 5, 2026	Light Node review: focus only on flagged weak areas from mock	Re-do one coding exercise from weakest Node area; explain al
37	August 6, 2026	Node confidence builder: teach-back session — explain core concepts to imaginary	30-min teach-back: record yourself explaining Node event loo
38	August 7, 2026	Rest day active recall: Node flashcards only, no new coding	Browse Node.js docs for 30 min on one weak API; close docs a
39	August 8, 2026	Interview day prep: Node mental model one-page cheat sheet review	Write one-page Node cheat sheet from memory; compare to note

Sample Interview Questions by Week

Week 1 (Days 1–7)

Day 1: Explain the Node.js event loop phases and what runs in each.
Day 2: How does Node resolve module paths for require()?
Day 3: What are the four stream types in Node.js?
Day 4: How does Express middleware execution order work?
Day 5: What is callback hell and how do Promises solve it?
Day 6: What is the difference between operational and programmer errors?
Day 7: When should you use cluster vs worker_threads?

Week 2 (Days 8–14)

Day 8: Compare session-based auth vs JWT stateless auth.
Day 9: What makes an API RESTful vs merely HTTP-based?
Day 10: How do you mock external dependencies in Node tests?
Day 11: How do Mongoose middleware hooks work (pre/post)?
Day 12: Explain cache-aside vs read-through vs write-through patterns.
Day 13: Compare fixed window vs sliding window rate limiting.
Day 14: Why use a message queue instead of synchronous HTTP calls?

Week 3 (Days 15–21)

Day 15: How do multipart form uploads work in Express?
Day 16: How does WebSocket differ from HTTP for real-time apps?
Day 17: Why use structured logging over console.log?
Day 18: Explain the top 3 OWASP risks relevant to Node APIs.
Day 19: How do you define service boundaries in a microservices architecture?
Day 20: When would you choose GraphQL over REST?
Day 21: How do you profile CPU vs memory issues in Node?

Week 4 (Days 22–28)

Day 22: How does the middleware pattern relate to composite pattern?
Day 23: What does strict mode catch that loose TypeScript misses?
Day 24: What causes Lambda cold starts and how do you mitigate them?
Day 25: What is the difference between commands and events in EDA?
Day 26: How do you size database connection pools in Node?
Day 27: How does OpenAPI improve API development workflow?
Day 28: What runs on libuv's thread pool vs the event loop?

Week 5 (Days 29–35)

Day 29: Explain hexagonal architecture and its benefits for testing.
Day 30: Describe scaling a MERN app from 1K to 100K users — what breaks first?
Day 31: Rapid-fire: explain event loop in 60 seconds.
Day 32: What is a Buffer and when do you use it vs TypedArray?
Day 33: Implement rate limiter — talk through while coding.
Day 34: Whiteboard your most complex MERN project architecture.
Day 35: Difference cluster vs worker_threads?

Week 6 (Days 36–39)

Day 36: Review only flagged questions from Day 35 checklist.
Day 37: If you can teach it simply, you know it — event loop.
Day 38: Active recall: event loop phases.
Day 39: What is your 60-second Node backend pitch?

Section 5: Java Spring Boot Curriculum

Java/Spring Boot is the growth area — frame as MERN veteran expanding backend breadth. Daily exercises build toward one portfolio-ready Spring Boot REST project by Day 25.

Core Topic Map

- JVM, JRE, JDK, bytecode, and compilation pipeline
- Data types, operators, control flow, and OOP fundamentals
- Interfaces, abstract classes, composition vs inheritance
- Collections — ArrayList, LinkedList, HashMap, HashSet internals
- Exception handling — checked vs unchecked, try-with-resources
- Generics, lambdas, and Stream API
- Multithreading — ExecutorService, CompletableFuture, synchronization
- Spring Boot fundamentals — auto-configuration, starters, Actuator
- Dependency injection and IoC container
- Spring MVC — REST controllers, validation, exception handlers
- Spring Data JPA — entities, repositories, relationships, N+1 fixes
- Spring Security — filter chain, JWT, method-level authorization
- Testing — JUnit 5, Mockito, @WebMvcTest, Testcontainers
- Transactions — @Transactional, isolation levels, propagation
- Docker + deployment — multi-stage builds, health probes, K8s basics

39-Day Java/Spring Daily Curriculum

Day	Date	Topic	Exercise
1	July 1, 2026	Java platform overview: JVM, JRE, JDK, bytecode, and compilation pipeline	Write a HelloWorld plus a class demonstrating primitives, wrapper
2	July 2, 2026	Java data types, variables, operators, and control flow	Implement FizzBuzz plus a switch-based menu CLI that validates nu
3	July 3, 2026	Object-oriented programming: classes, inheritance, polymorphism, encapsulat	Model a simple banking system with Account, SavingsAccount, and C
4	July 4, 2026	Interfaces, abstract classes, and composition vs inheritance	Refactor the banking exercise to use interfaces (Payable, Transfe
5	July 5, 2026	Collections framework: List, Set, Map, Queue — ArrayList vs LinkedList vs H	Implement word frequency counter using HashMap and sort results b
6	July 6, 2026	Exception handling: checked vs unchecked, try-with-resources, custom except	Build a file parser that uses try-with-resources and throws custo
7	July 7, 2026	Generics: type parameters, bounded types, wildcards, and type erasure	Implement a generic Pair<T,U> and a bounded method findMax(List<T
8	July 8, 2026	Java 8+ features: lambdas, functional interfaces, Stream API, Optional	Refactor word frequency to use Stream API: filter, map, collect,
9	July 9, 2026	String, StringBuilder, StringBuffer, and immutability performance implicati	Benchmark string concatenation in a loop: String vs StringBuilder
10	July 10, 2026	JUnit 5 basics: annotations, assertions, parameterized tests, and test life	Write JUnit 5 tests for the banking system with @BeforeEach setup
11	July 11, 2026	Spring Boot introduction: project structure, auto-configuration, and depend	Create a Spring Boot REST app with one GET /health endpoint using
12	July 12, 2026	Spring Core IoC: @Component, @Service, @Repository, @Autowired, bean scopes	Build a layered app: Controller → Service → Repository with const
13	July 13, 2026	Spring Boot REST controllers: @RestController, @RequestMapping, DTOs, Respo	Build CRUD REST API for Todo items with proper HTTP status codes

Day	Date	Topic	Exercise
14	July 14, 2026	Spring Data JPA: entities, repositories, @Query, relationships, and N+1 pro	Create JPA entities for User and Order with @OneToMany; implement
15	July 15, 2026	Spring Boot validation: @Valid, Bean Validation, custom constraints, and ex	Add @Valid to Todo API with custom @UniqueEmail constraint and @C
16	July 16, 2026	Spring Boot configuration: application.properties/yml, profiles, @Configura	Externalize config with dev/prod profiles and type-safe @Configur
17	July 17, 2026	Spring Boot logging: SLF4J, Logback, MDC, and structured JSON logs	Configure Logback JSON appenders and propagate correlation ID via
18	July 18, 2026	Spring Security basics: filter chain, authentication, authorization, BCrypt	Add Spring Security with form login disabled, HTTP Basic or JWT f
19	July 19, 2026	Spring Boot microservices: service discovery intro, RestTemplate/WebClient,	Create two Spring Boot services; call service B from A using WebC
20	July 20, 2026	Spring Boot testing: @WebMvcTest, @DataJpaTest, Testcontainers for integrat	Write @WebMvcTest for Todo controller and Testcontainers PostgreS
21	July 21, 2026	JVM performance: heap tuning, GC algorithms (G1, ZGC), and JVM flags basics	Run Spring Boot app with different heap sizes; observe GC logs an
22	July 22, 2026	Java design patterns: Singleton, Factory, Builder, Observer — idiomatic Jav	Implement Builder pattern for complex Order object; thread-safe S
23	July 23, 2026	Java concurrency: ExecutorService, CompletableFuture, synchronized, Reentra	Fetch data from 3 simulated APIs in parallel using CompletableFut
24	July 24, 2026	Spring Boot on AWS: Elastic Beanstalk, ECS Fargate overview, and containeri	Dockerize Spring Boot app; write Dockerfile with multi-stage buil
25	July 25, 2026	Spring events: ApplicationEventPublisher, @EventListener, async events, and	Publish OrderCreated event on save; handle with @EventListener; d
26	July 26, 2026	Spring Boot Actuator: health endpoints, metrics, custom indicators, and Pro	Add Actuator with custom DB health indicator and expose prometheu
27	July 27, 2026	SpringDoc OpenAPI: auto-generating Swagger UI, schema annotations, and API	Add springdoc-openapi to Todo API with @Operation annotations and
28	July 28, 2026	Java module system (JPMS) overview and Spring Boot 3 on Java 17+ features	Run Spring Boot 3 app on Java 17; use records for DTOs and sealed
29	July 29, 2026	Spring Boot clean architecture: domain layer, use cases, adapters, and pack	Reorganize Todo app into domain/application/infrastructure packag
30	July 30, 2026	Spring Boot production readiness: graceful shutdown, connection limits, and	Configure graceful shutdown, request timeout, and liveness/readin
31	July 31, 2026	Full Spring Boot mock review: REST + JPA + Security + testing stack synthes	Timed: add security + validation + one integration test to existi
32	August 1, 2026	Java Cryptography: MessageDigest, Mac, KeyGenerator, and secure coding prac	Implement HMAC verification utility in Java mirroring Node middle
33	August 2, 2026	Interview coding in Java: LRU cache, hash map design, and stream-based file	Implement LRU cache in Java using LinkedHashMap access-order mode
34	August 3, 2026	Java/Spring project story prep: learning Spring Boot project framed for int	Document a Spring Boot portfolio project with README sections: ar
35	August 4, 2026	Java/Spring final review: 50 rapid-fire conceptual questions self-drill	Complete 50-question Java/Spring checklist with same <30 second t
36	August 5, 2026	Light Java review: focus only on flagged weak areas from mock	Re-do one Spring Boot exercise from mock gaps; record 5-min expla
37	August 6, 2026	Java confidence builder: teach-back Spring Boot request lifecycle end-to-en	30-min teach-back: Spring DI, REST controller, JPA repository flo
38	August 7, 2026	Rest day active recall: Java flashcards only	Browse Spring docs for 30 min on weakest area; write minimal exam
39	August 8, 2026	Interview day prep: Java/Spring one-page cheat sheet review	Write one-page Spring Boot cheat sheet from memory; compare to no

Spring Boot Project Milestones

Day	Milestone
Day 5	Project scaffold — Spring Initializr, layered architecture, first REST endpoint
Day 10	JPA entities, repositories, PostgreSQL integration, basic CRUD
Day 15	Spring Security + JWT filter chain, role-based access
Day 20	Test suite — JUnit 5, @WebMvcTest, Testcontainers integration tests
Day 25	Dockerfile, README, API docs, demo-ready for interviews
Day 30	Performance tuning, Actuator metrics, production checklist review

Section 6: SQL Mastery Curriculum

SQL is a known gap vs NoSQL comfort — daily timed drills target window functions, complex joins, and EXPLAIN-driven optimization until speed matches MongoDB fluency.

Core Topic Progression

- ■ SELECT, WHERE, ORDER BY, LIMIT, and basic filtering
- ■ INNER, LEFT, RIGHT, and FULL OUTER JOIN patterns
- ■ GROUP BY, HAVING, and aggregate functions
- ■ Subqueries — scalar, correlated, EXISTS, IN, derived tables
- ■ Window functions — ROW_NUMBER, RANK, DENSE_RANK, LEAD, LAG
- ■ Indexing — B-tree, composite, covering indexes, EXPLAIN plans
- ■ Normalization — 1NF through 3NF, denormalization tradeoffs
- ■ Transactions — ACID, isolation levels, deadlocks
- ■ Query optimization — join order, index selection, query plans
- ■ Stored procedures, triggers, and views
- ■ Schema design — keys, constraints, foreign keys, cascades
- ■ Advanced patterns — CTEs, recursive queries, pivot/unpivot
- ■ Performance tuning — slow query analysis, partitioning
- ■ Replication, sharding, and read replicas
- ■ Interview SQL patterns — top-N, gaps, running totals, dedup

39-Day SQL Daily Topics & Problems

Day 1 — July 1, 2026: SQL fundamentals: SELECT, WHERE, ORDER BY, LIMIT, and basic filtering

1. Select all employees earning above the department average using a subquery.
2. Find the top 5 products by revenue in the last 30 days.
3. List customers who registered but never placed an order.

Day 2 — July 2, 2026: INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN patterns

1. Join orders to customers and show order count per customer city.
2. Find products with zero sales using a LEFT JOIN anti-pattern.
3. Return employees and their managers; include employees without managers.

Day 3 — July 3, 2026: GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MIN, MAX)

1. Find departments with average salary greater than company-wide average.
2. Count active users per signup month for the past year.
3. List categories where total sales exceed $100,000 using HAVING.

Day 4 — July 4, 2026: Subqueries: scalar, correlated, EXISTS, IN, and derived tables

1. Find employees earning more than their department average (correlated subquery).
2. List products that exist in wishlists but were never purchased.
3. Use EXISTS to find customers with at least 3 orders in Q1.

Day 5 — July 5, 2026: Window functions: ROW_NUMBER, RANK, DENSE_RANK, LEAD, LAG, PARTITION BY

1. Rank employees by salary within each department.
2. Find the second highest salary per department using DENSE_RANK.
3. Calculate month-over-month revenue change using LAG.

Day 6 — July 6, 2026: Indexing: B-tree indexes, composite indexes, covering indexes, and EXPLAIN plans

1. Given a slow query on orders(user_id, created_at), design optimal indexes.
2. Use EXPLAIN to compare query plans with and without a composite index.
3. Identify redundant indexes on a denormalized analytics table.

Day 7 — July 7, 2026: Transactions: ACID properties, isolation levels, deadlocks in SQL

1. Simulate a bank transfer with BEGIN/COMMIT/ROLLBACK and row-level locking.
2. Demonstrate dirty read vs repeatable read under different isolation levels.
3. Resolve a deadlock scenario between two concurrent update transactions.

Day 8 — July 8, 2026: Normalization: 1NF through 3NF, denormalization tradeoffs, and star schema intro

1. Normalize an unnormalized order spreadsheet into 3NF tables.
2. Identify update anomalies in a denormalized product catalog table.
3. Design a star schema for sales analytics from OLTP tables.

Day 9 — July 9, 2026: UNION, INTERSECT, EXCEPT, and set operations on query results

1. Find users who bought product A but not product B using EXCEPT.
2. Combine monthly sales from two regions with UNION ALL and dedupe.
3. Use INTERSECT to find VIP customers active in both web and mobile.

Day 10 — July 10, 2026: CTEs (WITH clauses): recursive CTEs, readability, and complex query decomposition

1. Use recursive CTE to generate a date series for gap-filling reports.
2. Build an org hierarchy report with recursive employee-manager CTE.
3. Rewrite a nested subquery as a readable CTE for monthly cohort analysis.

Day 11 — July 11, 2026: SQL performance tuning: query rewriting, N+1 detection, and slow query logs

1. Rewrite a correlated subquery as a JOIN for better performance.
2. Identify and fix an ORM-caused N+1 query pattern.
3. Optimize a report query scanning 10M rows using partitioning hints.

Day 12 — July 12, 2026: Stored procedures, functions, and triggers — when to use vs application logic

1. Write a trigger to audit all UPDATE operations on a sensitive table.
2. Create a stored function to calculate discounted price by customer tier.
3. Debate: move business logic to stored proc vs service layer — implement both.

Day 13 — July 13, 2026: Database design: ER modeling, cardinality, and schema migration strategies

1. Design ER diagram for e-commerce: users, orders, products, payments.
2. Write migration SQL to add soft-delete column with backfill.
3. Model many-to-many author-book relationship with junction table.

Day 14 — July 14, 2026: SQL joins deep dive: self-joins, cross joins, and join order optimization

1. Find consecutive login days per user using self-join or window functions.
2. Detect overlapping employee shift schedules with a self-join.
3. Analyze join order impact on a 5-table query using EXPLAIN.

Day 15 — July 15, 2026: PostgreSQL-specific: JSONB columns, arrays, and full-text search (tsvector)

1. Query JSONB metadata field for products matching nested attributes.
2. Implement full-text search on article title and body with ranking.
3. Use unnest on array column to find tags shared across posts.

Day 16 — July 16, 2026: Database replication: leader-follower, read replicas, replication lag, and failover

1. Design read routing: writes to primary, analytics reads to replica.
2. Write query to detect replication lag using heartbeat table.
3. Plan failover steps when primary goes down (conceptual SQL + ops).

Day 17 — July 17, 2026: Sharding and partitioning: horizontal vs vertical, shard keys, and cross-shard queries

1. Design shard key for a multi-tenant SaaS orders table.
2. Write queries for a range-partitioned events table by month.
3. Identify hot shard problem and propose resharding strategy.

Day 18 — July 18, 2026: SQL injection prevention: prepared statements, ORM safety, and input validation

1. Demonstrate vulnerable dynamic SQL then fix with parameterized queries.
2. Audit ORM raw query usage for injection risks.
3. Write safe dynamic ORDER BY using allowlist approach.

Day 19 — July 19, 2026: CAP theorem applied to databases: consistency vs availability tradeoffs

1. Given network partition, choose CP vs AP for payment vs social feed DB.
2. Design multi-region DB strategy with RPO/RTO constraints.
3. Compare strong consistency read vs eventual consistency read queries.

Day 20 — July 20, 2026: Materialized views, query caching, and read-model denormalization

1. Create materialized view for daily sales summary and refresh strategy.
2. Design denormalized read table fed by CDC from normalized OLTP.
3. Compare materialized view refresh vs incremental rollup table.

Day 21 — July 21, 2026: Complex interview SQL: running totals, gaps-and-islands, and pivot patterns

1. Compute running total of orders per customer ordered by date.
2. Find gap days in server uptime logs (gaps-and-islands).
3. Pivot sales by product category into columns per quarter.

Day 22 — July 22, 2026: LeetCode-style SQL: rank, dedupe, consecutive sequences, and top-N per group

1. Delete duplicate emails keeping row with lowest id.
2. Find customers who bought all products in a category.
3. Select top 3 earners per department handling ties.

Day 23 — July 23, 2026: Database connection pooling: sizing, timeouts, and leak detection

1. Calculate optimal pool size given DB max connections and app instances.
2. Diagnose connection leak from unclosed result sets.
3. Configure read/write pool split for replica routing.

Day 24 — July 24, 2026: Time-series data modeling: partitioning by time, retention policies, and downsampling

1. Design metrics table partitioned by day with auto-drop after 90 days.
2. Write query to downsample minute data to hourly averages.
3. Index strategy for time-range queries on billion-row event table.

Day 25 — July 25, 2026: Optimistic vs pessimistic locking: version columns, SELECT FOR UPDATE

1. Implement optimistic locking with version column on inventory table.
2. Use SELECT FOR UPDATE to prevent double-spend in wallet table.
3. Compare lost update problem solutions in high-contention scenario.

Day 26 — July 26, 2026: Data warehousing intro: ETL vs ELT, fact/dimension tables, and batch pipelines

1. Design fact table for orders and dimension tables for time, product, customer.
2. Write SQL transform step for daily ETL load from OLTP to warehouse.
3. Handle slowly changing dimension Type 2 for customer address.

Day 27 — July 27, 2026: Interview recap: 10 classic SQL patterns rapid-fire practice

1. Second highest salary — solve with subquery and with window function.
2. Monthly active users retention cohort for 6 months.
3. Find median salary per department (PostgreSQL percentile_cont).

Day 28 — July 28, 2026: SQL mock interview set: 5 problems in 90 minutes timed

1. Employees earning more than managers (self-join classic).
2. Running difference between consecutive row values.
3. Department top 3 salaries with tie handling.

Day 29 — July 29, 2026: Database interview rapid review: ACID, indexes, joins, windows, transactions

1. Design schema for ride-sharing app core entities.
2. Optimize slow query provided in prompt using indexes.
3. Explain CAP tradeoff choice for notification preferences store.

Day 30 — July 30, 2026: SQL system design integration: choosing SQL vs NoSQL for hybrid architectures

1. Design polyglot persistence for e-commerce catalog (SQL) + cart (Redis) + logs (Mongo).
2. Write migration plan from single PostgreSQL to read replicas.
3. Model idempotency keys table for payment retry safety.

Day 31 — July 31, 2026: FAANG SQL difficulty practice: 3 hard problems

1. Trips and users — consecutive days problem.
2. Market analysis — window functions with multiple partitions.
3. Tree traversal in SQL — employee hierarchy depth.

Day 32 — August 1, 2026: Database security: row-level security, encryption at rest, and audit logging

1. Implement PostgreSQL row-level security for multi-tenant data.
2. Design audit log table with tamper-evident constraints.
3. Query encrypted column strategy tradeoffs (app-level vs DB-level).

Day 33 — August 2, 2026: SQL speed run: 5 medium problems in 60 minutes

1. Customers who never order vs customers who order every month.
2. Product recommendation — co-purchase analysis.
3. Server utilization — sessions overlap counting.

Day 34 — August 3, 2026: Review weakest SQL patterns identified during mocks

1. Re-do missed mock SQL problems without looking at solutions.
2. Window function drill: 3 variations on ranking problems.
3. Complex JOIN drill: 4-table query for reporting.

Day 35 — August 4, 2026: SQL final review: 20 must-know patterns checklist

1. Top N per group — window function template.
2. Date spine generation — recursive CTE template.
3. Duplicate detection and removal template.

Day 36 — August 5, 2026: Light SQL: redo 3 missed problems only

1. Reattempt #1 missed SQL mock problem.
2. Reattempt #2 missed SQL mock problem.
3. Reattempt #3 missed SQL mock problem.

Day 37 — August 6, 2026: SQL confidence: verbalize approach before writing query for 3 classics

1. Second highest salary — verbalize then write.
2. Consecutive sessions — verbalize then write.
3. Cumulative sum — verbalize then write.

Day 38 — August 7, 2026: Rest day: one SQL problem timed at interview pace

1. Single timed SQL problem — full interview simulation.
2. Review solution and optimize if time permits.
3. Verbalize indexing strategy for the query.

Day 39 — August 8, 2026: Interview day prep: SQL pattern templates one-page review

1. Review window function template card.
2. Review join + group by template card.

3. Review CTE recursive template card.

Section 7: MongoDB

Leverage existing MERN MongoDB strength while filling gaps in aggregation pipelines, indexing strategy, replication, sharding, and transaction semantics.

MongoDB Core Curriculum

- Document model, BSON types, _id generation, CRUD operations
- Schema design — embedding vs referencing, denormalization patterns
- Indexing — single, compound, multikey, text, TTL indexes
- Aggregation pipeline — $match, $group, $lookup, $unwind, $facet
- Transactions — multi-document ACID, session handling
- Replication — replica sets, read preferences, write concerns
- Sharding — shard keys, chunk migration, balancer
- Performance — explain plans, covered queries, index intersection
- Mongoose ODM — schemas, middleware, virtuals, population
- Change streams and capped collections for real-time patterns
- Atlas cloud — backups, monitoring, search indexes
- Interview patterns — design schema for e-commerce, social feed, analytics

Daily MongoDB Topics (39-Day Plan)

Day	Date	MongoDB Topic
3	July 3, 2026	MongoDB document model, BSON types, _id generation, and CRUD insert/find basics
6	July 6, 2026	MongoDB indexing: single, compound, multikey, text indexes, and index intersection
9	July 9, 2026	MongoDB aggregation pipeline: $match, $group, $lookup, $project, $sort
11	July 11, 2026	Mongoose schema design patterns: discriminators, virtuals, and lean queries
12	July 12, 2026	Redis vs MongoDB use cases: when to use each as primary vs auxiliary store
15	July 15, 2026	GridFS and large file storage in MongoDB vs S3 tradeoffs
18	July 18, 2026	MongoDB security: role-based access, field-level encryption, and query injection prevention
21	July 21, 2026	MongoDB performance: explain plans, index hints, aggregation optimization, and schema design for reads
24	July 24, 2026	MongoDB change streams and CDC patterns for event-driven architectures
27	July 27, 2026	MongoDB replica sets: election, read preferences, write concern, and failover behavior
30	July 30, 2026	MongoDB sharding: shard key selection, chunk migration, and balancer
33	August 2, 2026	MongoDB interview questions rapid review: sharding, replication, aggregation, schema design
36	August 5, 2026	MongoDB light review: only weak topics from prior mocks
39	August 8, 2026	MongoDB one-page cheat sheet: CRUD, aggregation, indexes, replication, sharding

Section 8: Operating Systems

Structured OS review for backend interviews — processes, memory, scheduling, synchronization, file systems, containers, and production operations.

OS Fundamentals Checklist

- ■ Processes vs threads — address space, PCB, context switching
- ■ CPU scheduling — FCFS, SJF, round-robin, priority, multilevel queue
- ■ Memory management — paging, segmentation, virtual memory, TLB, page faults
- ■ Deadlocks — four conditions, prevention, avoidance, detection, recovery
- ■ Synchronization — mutex, semaphore, monitor, condition variables, spinlocks
- ■ File systems — inodes, directories, journaling, permissions
- ■ Virtualization vs containers — hypervisors, cgroups, namespaces
- ■ Linux process management — signals, niceness, ps/top/htop
- ■ Docker — images, layers, Dockerfile, multi-stage builds, volumes
- ■ Kubernetes — pods, services, deployments, HPA, ConfigMaps, Secrets
- ■ Monitoring — SLIs, SLOs, SLAs, error budgets, alerting
- ■ CI/CD — build pipelines, blue-green, canary deployments
- ■ Debugging — memory leaks, heap dumps, core dumps, strace

Daily OS Topics (39-Day Plan)

Day	Date	OS Topic
1	July 1, 2026	Processes vs threads: address space, context switching, and scheduling basics
3	July 3, 2026	CPU scheduling: FCFS, SJF, round-robin, and context switch cost
5	July 5, 2026	Memory management: paging, segmentation, virtual memory, page faults
6	July 6, 2026	Deadlocks: four necessary conditions, detection, prevention, and banker's algorithm intro
7	July 7, 2026	Synchronization primitives: mutex, semaphore, monitor, and critical sections
9	July 9, 2026	File systems: inodes, directories, journaling, and ext4 vs NTFS concepts
11	July 11, 2026	Virtualization vs containers: hypervisors, cgroups, namespaces overview
13	July 13, 2026	Docker fundamentals: images, containers, Dockerfile, layers, and multi-stage builds
15	July 15, 2026	Linux process management: ps, top, signals (SIGKILL vs SIGTERM), and niceness
17	July 17, 2026	Concurrency models: preemptive vs cooperative multitasking, async I/O
19	July 19, 2026	Kubernetes basics: pods, services, deployments, and horizontal pod autoscaling
21	July 21, 2026	Monitoring and alerting: SLIs, SLOs, SLAs, error budgets, and on-call practices
23	July 23, 2026	Shell scripting for ops: bash basics, cron, and deployment automation scripts
25	July 25, 2026	CI/CD pipelines: GitHub Actions, build/test/deploy stages, and blue-green deployment
27	July 27, 2026	Memory leak debugging in production: heap dumps, core dumps, and analysis tools
29	July 29, 2026	System design fundamentals review: latency numbers, back-of-envelope calculations
31	July 31, 2026	Operating system rapid review: scheduling, memory, deadlocks, synchronization
33	August 2, 2026	Process synchronization interview questions: mutex, semaphore, deadlock scenarios
35	August 4, 2026	OS final review: 15 core concepts checklist
37	August 6, 2026	OS top 10 rapid recall flashcards
39	August 8, 2026	OS one-liner cheat sheet: 10 concepts

Section 9: Computer Networks

Networking fundamentals for backend engineers — from OSI model through production patterns including load balancing, caching, TLS, and microservice communication.

Networking Fundamentals Checklist

- ■ OSI and TCP/IP models — layers, encapsulation, protocols per layer
- ■ IP addressing — subnets, CIDR notation, NAT, public vs private
- ■ TCP vs UDP — three-way handshake, reliability, use cases, ports
- ■ HTTP/1.1 vs HTTP/2 vs HTTP/3 — multiplexing, head-of-line blocking, QUIC
- ■ DNS — resolution flow, record types (A, AAAA, CNAME, MX), caching
- ■ TLS/SSL — handshake, certificates, cipher suites, HTTPS termination
- ■ Load balancing — L4 vs L7, algorithms, health checks, sticky sessions
- ■ WebSockets vs SSE vs long polling — real-time communication tradeoffs
- ■ CDN — edge caching, cache keys, purge strategies, origin shield
- ■ API Gateway — routing, auth termination, rate limiting, aggregation
- ■ gRPC vs REST — protobuf, HTTP/2, streaming, service contracts
- ■ Reverse proxy vs forward proxy — Nginx configuration concepts
- ■ Service mesh — sidecar proxy, mTLS, traffic management (Istio)
- ■ Consistent hashing — virtual nodes, ring topology, minimal reshuffling
- ■ Kafka networking — partitions, consumer groups, offset management
- ■ HTTP caching — Cache-Control, ETag, conditional requests
- ■ Latency budget — DNS + TCP + TLS + server + DB for typical API call
- ■ DDoS mitigation — rate limiting, WAF, anycast scrubbing
- ■ Zero-trust networking — mTLS between microservices
- ■ Troubleshooting — ping, traceroute, tcpdump, DNS debug methodology

Daily Networking Topics (39-Day Plan)

Day	Date	Networking Topic
1	July 1, 2026	OSI and TCP/IP models: layers, encapsulation, and common protocols per layer
2	July 2, 2026	IP addressing, subnets, CIDR notation, and NAT fundamentals
4	July 4, 2026	TCP vs UDP: handshake, reliability, use cases, and port numbers
6	July 6, 2026	HTTP/1.1 vs HTTP/2 vs HTTP/3: multiplexing, head-of-line blocking, QUIC
8	July 8, 2026	DNS resolution, record types (A, AAAA, CNAME, MX), and caching
10	July 10, 2026	TLS/SSL handshake, certificates, cipher suites, and HTTPS termination
12	July 12, 2026	Load balancing: L4 vs L7, algorithms (round-robin, least connections, consistent hash)
13	July 13, 2026	WebSockets vs SSE vs long polling — real-time communication tradeoffs
14	July 14, 2026	CDN architecture: edge caching, cache keys, purge, and origin shield
16	July 16, 2026	API Gateway patterns: routing, auth termination, rate limiting, and aggregation
17	July 17, 2026	gRPC vs REST: protobuf, HTTP/2, streaming, and when to choose each
18	July 18, 2026	Reverse proxy vs forward proxy: Nginx configuration concepts
20	July 20, 2026	Service mesh intro: sidecar proxy, mTLS, traffic management (Istio concepts)
22	July 22, 2026	Consistent hashing: virtual nodes, ring topology, and minimal reshuffling
23	July 23, 2026	DNS load balancing, anycast, and geo-routing for global services

Day	Date	Networking Topic
25	July 25, 2026	Message broker internals: Kafka partitions, consumer groups, and offset management
26	July 26, 2026	TCP congestion control: slow start, AIMD, and impact on API latency
28	July 28, 2026	HTTP caching: Cache-Control, ETag, conditional requests, and CDN integration
29	July 29, 2026	Latency budget breakdown: DNS + TCP + TLS + server + DB for a typical API call
30	July 30, 2026	DDoS mitigation: rate limiting, WAF, anycast scrubbing, and edge protection
32	August 1, 2026	Zero-trust networking principles and mTLS between microservices
34	August 3, 2026	Network troubleshooting methodology: ping, traceroute, tcpdump, and DNS debug
35	August 4, 2026	Networking final review: 15 core concepts checklist
37	August 6, 2026	Network top 10 rapid recall flashcards
39	August 8, 2026	Network one-liner cheat sheet: 10 concepts

Section 10: System Design — 15 Complete Designs

Each design includes requirements, API, database schema, scaling, caching, queues, partitioning, replication, and explicit tradeoffs. Practice one design every 2–3 days; whiteboard with 45-minute timer.

Design: URL Shortener

Requirements: Shorten long URLs to 7-char codes; 100M URLs/month; 100:1 read/write ratio; custom aliases optional; analytics on clicks.

API Design: POST /api/v1/urls {longUrl, customAlias?} → {shortCode, shortUrl}; GET /{code} → 302 redirect; GET /api/v1/urls/{code}/stats → click analytics.

Database Schema: PostgreSQL or Cassandra for URL mappings (short_code PK, long_url, user_id, created_at); Redis for hot redirect cache; analytics in ClickHouse or time-series DB.

Scaling Strategy: Stateless app servers behind L7 load balancer; horizontal scale reads; separate write and read paths; geo-distributed redirects via CDN edge.

Caching Layer: Cache-aside in Redis for short_code → long_url; 80/20 rule means 20% URLs get 80% traffic; TTL 24h with lazy refresh on miss.

Message Queues: Async click event queue (Kafka) for analytics ingestion; decouple redirect path from stats writes.

Partitioning: Shard URL table by hash(short_code) across DB nodes; consistent hashing for cache nodes.

Replication: Multi-region read replicas for analytics queries; primary for writes; Redis cluster with replication for HA.

Key Tradeoffs:

- Base62 encoding vs UUID — shorter URLs but collision handling needed
- SQL vs NoSQL — SQL simpler for low volume; Cassandra better at billions of rows
- Sync redirect vs async analytics — optimize redirect latency (<50ms p99)
- MD5/SHA hash vs counter-based IDs — hash avoids coordination; counter needs distributed ID generator

---

Design: WhatsApp

Requirements: 1B+ users; 1:1 and group messaging; delivery/read receipts; online status; media sharing; E2E encryption; 99.99% availability.

API Design: WebSocket gateway for send/receive; POST /messages; GET /conversations/{id}/messages?cursor=; POST /groups; PUT /presence.

Database Schema: Cassandra for message storage partitioned by conversation_id; separate user/contact graph in PostgreSQL; HBase optional for message archive.

Scaling Strategy: WebSocket connection servers sharded by user_id; millions of concurrent connections via epoll/async IO; regional data centers.

Caching Layer: Redis for recent messages per conversation, user session, and online presence; CDN for media blobs in object storage.

Message Queues: Kafka for message fan-out to recipients, push notification workers, and offline message delivery pipeline.

Partitioning: Messages partitioned by conversation_id; users sharded geographically for data locality and compliance.

Replication: Multi-datacenter replication with eventual consistency for messages; quorum reads for critical paths.

Key Tradeoffs:

- E2E encryption limits server-side search and moderation

- WebSocket sticky sessions vs message queue decoupling for scale-out
- Cassandra write-heavy vs PostgreSQL — messages need write-optimized store
- Push vs pull for offline users — battery and delivery latency tradeoffs

---

Design: Instagram

Requirements: Photo/video upload and feed; 500M DAU; follow graph; likes/comments; stories (24h TTL); explore/discovery; 500ms feed load p99.

API Design: POST /media/upload (presigned S3); GET /feed?cursor=; POST /media/{id}/like; GET /users/{id}/profile; GET /explore.

Database Schema: Cassandra for posts and feeds; PostgreSQL for user profiles and follow graph; S3 for media; Elasticsearch for search/explore.

Scaling Strategy: Fan-out on write for normal users; fan-out on read for celebrities; hybrid approach based on follower count threshold.

Caching Layer: CDN for media delivery; Redis cache for user feed timelines and hot posts; cache user profile and follower counts.

Message Queues: Kafka for feed fan-out workers, notification generation, and async thumbnail/transcoding pipelines.

Partitioning: Posts by user_id; feed tables by follower_user_id; shard social graph by user_id hash.

Replication: Cross-region S3 replication for media; DB replicas per region; CDN edge caching globally.

Key Tradeoffs:

- Fan-out on write vs read — storage cost vs read latency
- Strong consistency for likes count vs eventual — use approximate counts at scale
- Transcoding sync vs async — async improves upload UX
- Chronological vs ranked feed — ML ranking adds complexity and latency

---

Design: Twitter

Requirements: Post tweets (280 chars); follow users; home timeline; search tweets; trending topics; 400M tweets/day; celebrity read amplification.

API Design: POST /tweets; GET /timeline/home?cursor=; GET /users/{id}/tweets; GET /search?q=; POST /follow/{userId}.

Database Schema: MySQL/PostgreSQL for users and tweet metadata; Cassandra for tweets by user_id; Redis for timelines; Elasticsearch for search.

Scaling Strategy: Hybrid fan-out: write fan-out for normal users, read fan-out for users with >1M followers; tweet ID from Snowflake.

Caching Layer: Redis sorted sets for home timelines (tweet_id as score); cache hot tweets and user objects.

Message Queues: Kafka for timeline fan-out, search indexing, trending aggregation, and notification dispatch.

Partitioning: Tweets sharded by tweet_id or user_id; timelines partitioned by user_id across Redis cluster.

Replication: Read replicas for user profiles; cross-DC replication for disaster recovery; search index replicas.

Key Tradeoffs:

- Fan-out on write storage explosion vs fan-out on read latency for celebrities
- Eventual consistency on timeline vs strong — accept seconds delay
- Search index lag vs write throughput
- Retweet/quote complexity in timeline merge logic

Design: YouTube

Requirements: Video upload, transcode, stream; 2B users; recommendations; comments; 500 hours uploaded/minute; adaptive bitrate streaming.

API Design: POST /videos/upload (chunked); GET /videos/{id}/stream; GET /feed/recommended; POST /videos/{id}/comments; GET /search.

Database Schema: PostgreSQL for metadata; Cassandra for view counts and comments; blob storage for raw/transcoded video; graph DB for recommendations optional.

Scaling Strategy: Upload servers separate from streaming CDN; transcoding farm with priority queues; regional CDN PoPs.

Caching Layer: CDN caches video segments (HLS/DASH); Redis for video metadata, view counts, and trending; edge cache popular content.

Message Queues: SQS/Kafka for transcoding pipeline (360p/720p/1080p); separate queue for thumbnail generation and search indexing.

Partitioning: Videos sharded by video_id; comments partitioned by video_id; view events partitioned by time.

Replication: S3 cross-region replication; CDN multi-PoP; metadata DB read replicas.

Key Tradeoffs:

- Transcoding cost vs storage of multiple formats
- View count accuracy vs real-time — batch aggregation acceptable
- Recommendation freshness vs compute cost
- Copyright detection sync vs async scanning

Design: Netflix

Requirements: Stream movies/shows globally; personalized recommendations; 200M subscribers; resume playback; multiple profiles; 99.99% streaming availability.

API Design: GET /catalog/home; GET /titles/{id}/manifest; POST /playback/start; GET /recommendations; PUT /profiles/{id}/preferences.

Database Schema: Cassandra for viewing history and preferences; PostgreSQL for catalog metadata; S3 for video assets; ML feature store for recommendations.

Scaling Strategy: Open Connect CDN (Netflix-owned); regional content caches; microservices per domain (playback, billing, recommendations).

Caching Layer: Aggressive CDN edge caching of video chunks; EVCache (memcached) for metadata and session state; pre-position popular content.

Message Queues: Kafka for viewing event pipeline, A/B test metrics, and recommendation model feature updates.

Partitioning: Viewing history by user_id; catalog by region/licensing; A/B experiments partitioned by user cohort.

Replication: Multi-region CDN; Cassandra multi-DC; active-active regional serving.

Key Tradeoffs:

- Personalization accuracy vs privacy — viewing data sensitivity
- Codec efficiency (AV1) vs encoding cost and device compatibility
- Regional licensing vs global catalog consistency
- Microservices complexity vs team autonomy

Design: Uber

Requirements: Match riders and drivers; real-time location tracking; ETA calculation; surge pricing; trip history; 15M trips/day; sub-second matching in dense cities.

API Design: POST /trips/request; PUT /drivers/{id}/location; GET /trips/{id}/eta; POST /trips/{id}/complete; GET /surge?lat&lng.

Database Schema: PostgreSQL for trips/users; Redis GEO for driver locations; Cassandra for location history; Kafka for event stream.

Scaling Strategy: Geospatial sharding by city/hex grid (H3/S2); dispatch service per region; WebSocket for real-time updates.

Caching Layer: Redis GEO indexes for nearby drivers; cache surge multipliers by geohash; route ETA cache from Google/Mapbox.

Message Queues: Kafka for location update stream, trip state machine events, and payment settlement.

Partitioning: Partition by geohash prefix; each city cell handles local matching independently.

Replication: Regional active-active; location data replicated for failover; trip records durable multi-AZ.

Key Tradeoffs:

- Strong consistency on driver location vs update frequency — eventual OK within seconds
- Surge pricing fairness vs supply/demand optimization
- Hex grid resolution vs query precision and shard count
- Matching algorithm complexity vs p99 latency budget

Design: Food Delivery

Requirements: Restaurant catalog; order placement; delivery tracking; payments; 30-min delivery SLA; 10M orders/day; restaurant partner portal.

API Design: GET /restaurants/nearby?lat&lng; POST /orders; GET /orders/{id}/track; PUT /orders/{id}/status; POST /payments/charge.

Database Schema: PostgreSQL for orders/restaurants; Redis for active order tracking; Elasticsearch for restaurant search; MongoDB for menus (flexible schema).

Scaling Strategy: Geo-partitioned dispatch; separate services for catalog, ordering, delivery, payments; peak lunch/dinner autoscaling.

Caching Layer: Cache restaurant menus and ratings; Redis for order state machine; CDN for food images.

Message Queues: Kafka/RabbitMQ for order routing to restaurants, driver assignment, and delivery ETA updates.

Partitioning: Orders by region/city; restaurant catalog by geo tile; delivery fleet partitioned by zone.

Replication: Multi-AZ for order DB; idempotent payment processing with outbox pattern.

Key Tradeoffs:

- Menu consistency vs restaurant update frequency
- Order state machine complexity with cancellation/refund paths
- Third-party delivery vs in-house fleet economics
- Real-time tracking battery drain vs location update interval

---

Design: Notification System

Requirements: Multi-channel (push, email, SMS, in-app); 1B notifications/day; priority levels; user preferences; template management; delivery tracking.

API Design: POST /notifications/send {userId, channel, template, data}; GET /notifications/{userId}?cursor=; PUT /preferences; GET /templates.

Database Schema: PostgreSQL for templates and preferences; Cassandra for notification log; Redis for rate limiting and deduplication.

Scaling Strategy: Channel-specific worker pools; priority queues for transactional vs marketing; regional SMS/push providers.

Caching Layer: Cache user preferences and device tokens; template compilation cache; dedup window in Redis.

Message Queues: Separate Kafka topics per channel and priority; retry queues with exponential backoff; DLQ for failed deliveries.

Partitioning: Partition by user_id for ordering guarantees per user; shard providers by region.

Replication: Multi-provider failover for SMS/push; notification log replicated for audit.

Key Tradeoffs:

- At-least-once delivery vs duplicate notifications — idempotency keys
- Marketing batch sends vs transactional low-latency path isolation
- User preference granularity vs send logic complexity
- Template versioning and rollback strategy

Design: Chat System

Requirements: 1:1 and group chat; 50M concurrent connections; message history; typing indicators; read receipts; file sharing; message ordering per conversation.

API Design: WebSocket /ws?token=; POST /conversations; GET /conversations/{id}/messages?before=; POST /messages; PUT /read-receipt.

Database Schema: Cassandra messages by conversation_id + timestamp; PostgreSQL for user/conversation metadata; S3 for attachments.

Scaling Strategy: Connection servers with consistent hash routing; separate message storage from connection layer; regional clusters.

Caching Layer: Redis for recent messages, online status, typing indicators; cache conversation member lists.

Message Queues: Kafka for cross-region message sync, push notifications for offline users, and search indexing.

Partitioning: Messages by conversation_id; connections by user_id to fixed server.

Replication: Multi-region with conflict resolution on message order; attachment cross-region S3 replication.

Key Tradeoffs:

- WebSocket sticky routing vs cross-server message relay complexity
- Message ordering guarantees vs multi-region latency
- Storage cost of infinite history vs pagination/archival
- Group message fan-out inline vs async

Design: Distributed Cache

Requirements: 100GB+ cache cluster; 1M ops/sec; sub-ms latency; TTL support; cache invalidation; 99.99% availability; consistent hashing.

API Design: GET/SET/DEL key; MGET/MSET; EXPIRE key ttl; internal: GET /health, GET /stats/shard.

Database Schema: In-memory only (Redis/Memcached); optional persistence (RDB/AOF) for recovery; no primary DB — cache-aside pattern.

Scaling Strategy: Horizontal sharding with consistent hashing; add nodes with minimal key redistribution via virtual nodes.

Caching Layer: LRU/LFU eviction policies; hot key replication; local in-process L1 cache optional for read-heavy.

Message Queues: Optional async invalidation broadcast via pub/sub or Kafka for cache coherence across regions.

Partitioning: Hash key → shard via consistent hashing ring; 150 virtual nodes per physical node typical.

Replication: Primary-replica per shard; read from replica for scale; automatic failover with sentinel/cluster mode.

Key Tradeoffs:

- Cache-aside vs read-through vs write-through complexity
- Strong consistency vs latency — accept stale reads with TTL
- Hot key problem — local cache replication vs client-side caching
- Eviction policy impact on hit rate for skewed workloads

Design: Search Engine

Requirements: Index 10B web pages; sub-200ms query latency; ranked results; autocomplete; spell correction; 5K QPS peak.

API Design: GET /search?q=&page=; GET /suggest?q=; POST /index (crawler pipeline internal); GET /health.

Database Schema: Inverted index in Elasticsearch/Lucene; document store in HDFS/S3; PageRank scores in offline batch store.

Scaling Strategy: Distributed index shards; query fan-out to all shards then merge; crawler fleet for indexing.

Caching Layer: Cache popular queries and results in Redis; CDN for static assets; query result cache with TTL.

Message Queues: Kafka for crawl pipeline: URL frontier → fetch → parse → index; batch PageRank computation.

Partitioning: Index partitioned by term hash or document ID ranges; geo shards for local search variants.

Replication: Index replicas for query throughput; leader-follower for index updates.

Key Tradeoffs:

- Index freshness vs crawl budget — recrawl frequency
- Relevance (ML ranking) vs query latency budget
- Storage of full inverted index vs compressed postings
- Real-time indexing vs batch for news/social integration

Design: Payment Gateway

Requirements: Process payments; PCI compliance; idempotent transactions; refunds; multi-currency; fraud detection; 99.999% correctness over availability.

API Design: POST /payments {amount, currency, source, idempotencyKey}; POST /refunds; GET /payments/{id}; POST /webhooks/stripe.

Database Schema: PostgreSQL for ledger (double-entry bookkeeping); immutable transaction log; Redis for idempotency keys.

Scaling Strategy: Stateless payment API; shard by merchant_id; async fraud scoring pipeline.

Caching Layer: Cache merchant config and exchange rates; idempotency key cache with 24h TTL.

Message Queues: Kafka for async settlement, webhook delivery, fraud analysis, and reconciliation batch jobs.

Partitioning: Ledger partitioned by merchant_id; strict serializability per account.

Replication: Sync replication for ledger; multi-AZ with automatic failover; audit log immutable.

Key Tradeoffs:

- Strong consistency mandatory — never double-charge; CP over AP
- Sync fraud check vs async — latency vs loss prevention
- Idempotency key storage TTL vs retry window
- PCI scope reduction via tokenization vs direct card handling

---

Design: News Feed

Requirements: Personalized feed; post from friends and pages; 1B users; rank by relevance and recency; 300ms feed generation p99; real-time new post injection.

API Design: GET /feed?cursor=&limit=; POST /posts; POST /reactions; GET /posts/{id}; PUT /feed/preferences.

Database Schema: Cassandra for posts; Redis for precomputed feeds; PostgreSQL for social graph; ML ranker feature store.

Scaling Strategy: Hybrid fan-out (write for normal, read for celebrities); feed generation as rank + merge of candidate sets.

Caching Layer: Precomputed feed chunks in Redis; cache post objects and author metadata; CDN for media.

Message Queues: Kafka for fan-out on post creation, ranker feature updates, and notification triggers.

Partitioning: Posts by author_id; feed cache by user_id; graph partitioned by user_id.

Replication: Feed cache replicas; post storage multi-DC; ranker model served from regional endpoints.

Key Tradeoffs:

- Precompute vs real-time rank — staleness vs compute cost
- Fan-out write storage vs read latency for influencers
- ML ranking complexity vs explainability and debug
- Feed consistency when unfollow/block happens mid-read

Design: Rate Limiter

Requirements: Limit API requests per user/IP; 1M RPS global; distributed across 100 servers; configurable limits; burst allowance; low overhead (<1ms).

API Design: Middleware/gateway integration; GET /ratelimit/status; admin PUT /ratelimit/rules {entity, limit, window}.

Database Schema: Redis for counters; optional PostgreSQL for rule config; in-memory local cache for hot rules.

Scaling Strategy: Redis cluster for centralized counters; local token bucket for edge filtering; gateway-level enforcement.

Caching Layer: Cache rate limit rules in each node; local token bucket reduces Redis roundtrips.

Message Queues: Optional async logging of rate limit violations to Kafka for abuse detection.

Partitioning: Redis hash tags for user_id colocation; shard counters by entity key.

Replication: Redis replicas for HA; rule config replicated; fail-open vs fail-closed policy on Redis outage.

Key Tradeoffs:

- Fixed window vs sliding window vs token bucket accuracy
- Centralized Redis vs distributed gossip — consistency vs latency
- Fail-open (allow) vs fail-closed (block) on infrastructure failure
- Per-IP vs per-user vs per-API-key granularity

Section 11: Behavioral — 39-Day STAR Prompts

Daily behavioral prompt for STAR method practice (Situation, Task, Action, Result). Record 2-minute spoken answers; include metrics. Build a story bank of 8–10 core stories.

STAR Framework Reminder

- Situation — Set context in 2 sentences (team, product, scale)
- Task — Your specific responsibility and constraint
- Action — What YOU did (use 'I', not 'we'); technical and leadership details
- Result — Quantified outcome (latency, revenue, uptime, team velocity)
- Reflection — What you learned; what you'd do differently

Daily STAR Prompts

Day	Date	Behavioral Prompt
1	July 1, 2026	Tell me about a time you took ownership of a production incident and restored service.
2	July 2, 2026	Describe a situation where you had conflicting priorities from stakeholders. How did you decide?
3	July 3, 2026	Give an example of mentoring or upskilling a junior developer on your team.
4	July 4, 2026	Tell me about a time you improved code quality or reduced technical debt.
5	July 5, 2026	Describe a time you missed a deadline. What happened and what did you change?
6	July 6, 2026	Tell me about a feature you shipped that had measurable business impact.
7	July 7, 2026	Walk through your strongest project end-to-end using STAR (Situation, Task, Action, Result).
8	July 8, 2026	Describe a time you disagreed with a technical decision. How was it resolved?
9	July 9, 2026	Tell me about a time you learned a new technology quickly to deliver on a project.
10	July 10, 2026	Describe your most challenging bug and how you debugged it systematically.
11	July 11, 2026	Tell me about a time you collaborated cross-functionally with product or design.
12	July 12, 2026	Give an example of receiving harsh feedback and how you responded.
13	July 13, 2026	Describe a time you had to deliver under ambiguous requirements.
14	July 14, 2026	Week 2 behavioral prep: refine 5 STAR stories covering leadership, conflict, failure, impact, learning.
15	July 15, 2026	Tell me about optimizing application performance — what metrics did you track?
16	July 16, 2026	Describe a time you had to say no to a feature request. How did you communicate it?
17	July 17, 2026	Tell me about a time you improved team processes or documentation.
18	July 18, 2026	Describe a production outage you caused or contributed to. What was your accountability?
19	July 19, 2026	Tell me about a time you had to estimate effort for a complex project.
20	July 20, 2026	Describe your ideal team culture and how you contribute to it.
21	July 21, 2026	Practice concise 2-minute 'Tell me about yourself' tailored for backend roles.
22	July 22, 2026	Tell me about a time you made a data-driven decision.
23	July 23, 2026	Describe handling a disagreement with your manager.
24	July 24, 2026	Tell me about a time you went above and beyond for a customer or user.
25	July 25, 2026	Describe a time you had to balance speed vs quality in a release.
26	July 26, 2026	Tell me about working with a difficult teammate.
27	July 27, 2026	Describe a time you influenced technical direction without formal authority.
28	July 28, 2026	Week 4 STAR story rotation — record yourself and critique clarity and metrics.
29	July 29, 2026	Prepare answers for 'Why are you leaving?' and 'Why this company?' templates.
30	July 30, 2026	Tell me about your biggest professional achievement with quantified results.

Day	Date	Behavioral Prompt
31	July 31, 2026	Practice salary negotiation talking points and competing offer framework.
32	August 1, 2026	Describe a time you failed publicly. How did you recover credibility?
33	August 2, 2026	Answer 'What is your greatest weakness?' with genuine growth story.
34	August 3, 2026	Prepare questions to ask interviewer — 10 thoughtful questions for backend roles.
35	August 4, 2026	Full behavioral mock: 8 questions in 45 minutes with timed responses.
36	August 5, 2026	Rest and rehearse top 3 STAR stories — muscle memory, not new content.
37	August 6, 2026	Review company-specific talking points for top 5 target companies.
38	August 7, 2026	Light behavioral review — read STAR stories once, no rewriting.
39	August 8, 2026	Prepare opening 'Tell me about yourself' 90-second script and tomorrow's logistics checklist.

Core Story Bank Categories

- Production incident / outage recovery
- Technical conflict / architecture disagreement
- Mentoring / upskilling a junior developer
- Missed deadline / failure and recovery
- Measurable business impact feature
- Technical debt reduction / code quality improvement
- Cross-functional stakeholder management
- Learning new technology under time pressure
- Performance optimization with metrics
- Leadership without authority

Section 12: Resume & ATS Optimization

Resume Bullets — Node Version

- Architected and delivered scalable REST APIs using Node.js and Express serving 50K+ daily active users with 99.9% uptime.
- Designed MongoDB document schemas and aggregation pipelines reducing average query latency by 40% through strategic indexing.
- Implemented JWT-based authentication with refresh token rotation and Redis session blacklist for a multi-tenant SaaS platform.
- Built real-time features using Socket.io with Redis adapter enabling horizontal scaling across 4 Node.js cluster workers.
- Integrated BullMQ job queues for async email, PDF generation, and webhook delivery processing 100K+ jobs/day with retry logic.
- Optimized Node.js API performance by profiling with clinic.js, fixing memory leaks and reducing p99 latency from 800ms to 120ms.
- Established Jest and supertest testing pipeline achieving 85% coverage on service and integration layers with CI enforcement.
- Migrated legacy callback-based services to async/await with centralized error handling and structured logging via pino.

Resume Bullets — Java Version

- Building production-grade REST microservices with Java 17 and Spring Boot including JPA, Spring Security, and Actuator monitoring.
- Implemented layered architecture (Controller → Service → Repository) with constructor injection and @Valid request validation.
- Designed PostgreSQL schemas with Spring Data JPA; resolved N+1 query issues using @EntityGraph reducing API response time by 35%.
- Configured Spring Security with JWT filter chain and role-based @PreAuthorize access control for stateless API authentication.
- Wrote comprehensive test suite using JUnit 5, @WebMvcTest, and Testcontainers for PostgreSQL integration testing.
- Containerized Spring Boot applications with multi-stage Dockerfiles and configured health probes for Kubernetes deployment.
- Applied transactional outbox pattern discussion and event publishing with ApplicationEventPublisher for order processing flow.
- Leveraging 5 years of backend API experience from MERN stack to rapidly deliver idiomatic Spring Boot services with clean architecture.

Resume Bullets — General Version

- 5+ years of full-stack software engineering experience building and shipping production web applications with measurable user impact.
- Led end-to-end feature development from requirements gathering through deployment, monitoring, and iterative improvement.
- Designed and implemented RESTful APIs and database schemas for high-traffic applications serving tens of thousands of daily users.
- Collaborated cross-functionally with product, design, and QA to deliver features on schedule in agile two-week sprint cycles.
- Diagnosed and resolved production incidents including performance bottlenecks, data inconsistencies, and third-party integration failures.
- Mentored junior developers through code reviews, pair programming, and documentation improving team delivery velocity.
- Passionate about backend systems — deepening expertise in Java/Spring Boot, system design, and distributed systems for senior roles.
- Strong foundation in JavaScript/TypeScript, Node.js, React, MongoDB, PostgreSQL, Redis, Docker, and AWS cloud services.

ATS Optimization Checklist

- One-page format (two pages max for 5+ years experience)
- Standard fonts — Arial, Calibri, or Helvetica at 10–11pt
- No tables, text boxes, headers/footers, or graphics that break parsing
- Section headers: Experience, Skills, Education (standard labels)
- Keywords from job description woven into bullets naturally
- Quantified impact — users, latency, uptime, revenue where possible
- Separate tailored versions for Node-heavy vs Java-heavy roles
- PDF export from Word or Google Docs (not scanned image)
- Contact info in body text, not header/footer
- Skills section lists exact technologies from target job posts
- File name format: FirstName_LastName_Backend_Engineer.pdf
- LinkedIn URL and GitHub with active Spring Boot portfolio link

Tailoring Guide

Target	Strategy
Node.js roles	Lead with Node bullets; Java as secondary; emphasize MongoDB, Redis, Express
Java/Spring roles	Lead with Java bullets; frame MERN as full-stack breadth; link Spring portfolio
Full-stack roles	Mix General + Node bullets; show React + backend depth
FAANG / big tech	Emphasize scale metrics, system design exposure, DSA readiness
Startups	Emphasize ownership, speed, end-to-end delivery, production incident stories

Section 13: Job Search Strategy

Target list of 40 companies with tiered application strategy. Apply strategically — don't spray and pray. Warm up with Tier 3 before Tier 1.

Target Companies

#	Company	Tier
1	Google	Tier 1
2	Amazon	Tier 1
3	Microsoft	Tier 1
4	Meta	Tier 1
5	Apple	Tier 1
6	Netflix	Tier 1
7	Flipkart	Tier 1
8	Swiggy	Tier 1
9	Zomato	Tier 1
10	Razorpay	Tier 1
11	PhonePe	Tier 2
12	Paytm	Tier 2
13	CRED	Tier 2
14	Meesho	Tier 2
15	ShareChat	Tier 2
16	Freshworks	Tier 2
17	Zoho	Tier 2
18	Atlassian	Tier 2
19	Uber	Tier 2
20	Goldman Sachs	Tier 2
21	JPMorgan Chase	Tier 2
22	Deutsche Bank	Tier 2
23	Barclays	Tier 2
24	Walmart Global Tech	Tier 2
25	Target	Tier 2
26	Adobe	Tier 3
27	Salesforce	Tier 3
28	Intuit	Tier 3
29	LinkedIn	Tier 3
30	Twitter/X	Tier 3
31	Stripe	Tier 3
32	Shopify	Tier 3
33	MongoDB Inc	Tier 3
34	Postman	Tier 3
35	RazorpayX	Tier 3
36	Groww	Tier 3
37	Zerodha	Tier 3
38	Dream11	Tier 3

#	Company	Tier
39	Ola	Tier 3
40	Info Edge (Naukri)	Tier 3

Application Strategy

- Days 1–10: Apply to 2–3 Tier 3 companies daily; practice application flow
- Days 11–20: Increase to 3–4 daily; add Tier 2; begin recruiter outreach (1–2/day)
- Days 21–30: Target 4–5 daily; prioritize referrals; tailor resume per stack
- Days 31–39: Focus on active pipelines; Tier 1 with strong referrals only
- Track every application in Appendix B tracker — follow up at 5 and 10 business days
- LinkedIn: 3 connection requests daily to engineers at target companies
- GitHub: Pin Spring Boot portfolio repo; keep commit activity visible
- Referrals: Ask after establishing rapport — provide resume blurb and role links

Daily Application Targets (from curriculum)

Day	Date	Applications	Recruiter Outreach
1	July 1, 2026	2	1
2	July 2, 2026	2	1
3	July 3, 2026	2	1
4	July 4, 2026	3	1
5	July 5, 2026	3	2
6	July 6, 2026	3	2
7	July 7, 2026	3	2
8	July 8, 2026	3	2
9	July 9, 2026	3	2
10	July 10, 2026	3	2
11	July 11, 2026	4	2
12	July 12, 2026	4	2
13	July 13, 2026	4	3
14	July 14, 2026	4	3
15	July 15, 2026	4	3
16	July 16, 2026	5	3
17	July 17, 2026	5	3
18	July 18, 2026	5	3
19	July 19, 2026	5	3
20	July 20, 2026	5	3
21	July 21, 2026	6	4
22	July 22, 2026	6	4
23	July 23, 2026	6	4
24	July 24, 2026	6	4
25	July 25, 2026	7	4
26	July 26, 2026	7	4
27	July 27, 2026	7	5
28	July 28, 2026	7	5

Day	Date	Applications	Recruiter Outreach
29	July 29, 2026	8	5
30	July 30, 2026	8	5
31	July 31, 2026	9	5
32	August 1, 2026	9	5
33	August 2, 2026	9	5
34	August 3, 2026	10	5
35	August 4, 2026	10	5
36	August 5, 2026	8	4
37	August 6, 2026	7	3
38	August 7, 2026	5	2
39	August 8, 2026	3	1

Section 14: Mock Interview Schedule

Five full mock interview days simulate real interview loops. Record every session, score with Appendix C scorecards, and write a 30-minute post-mortem after each mock.

Mock #1 — Day 7 (July 7, 2026) | Focus: Node + SQL + Behavioral

Time	Round Type	Topics
09:00-09:45	Node.js Technical	Event loop, Express, async patterns, auth basics
10:00-10:45	SQL Round	JOINs, GROUP BY, subqueries, indexing
11:00-11:30	Behavioral	3 STAR stories: incident, conflict, impact
14:00-14:30	Review	Score gaps, update weak area list, adjust Week 2 plan

Mock #2 — Day 14 (July 14, 2026) | Focus: Java Spring + System Design

Time	Round Type	Topics
09:00-09:45	Java/Spring Boot	IoC, REST, JPA, Security filter chain
10:00-11:00	System Design	Distributed Cache — requirements through tradeoffs
11:15-11:45	Behavioral	Leadership, failure, learning stories
14:00-15:00	Review	Java gap analysis, SD whiteboard recording review

Mock #3 — Day 21 (July 21, 2026) | Focus: Mixed stack + DSA medium

Time	Round Type	Topics
09:00-09:45	Coding (DSA)	2 medium problems: arrays/hash map, trees or graphs
10:00-10:45	Node.js Deep Dive	Streams, queues, caching, microservices
11:00-11:45	Java/Spring	JPA, transactions, testing, Actuator
14:00-14:45	SQL + Behavioral	Window functions timed set + 2 STAR stories
15:00-15:30	Review	Timed problem review, speed improvement targets

Mock #4 — Day 28 (July 28, 2026) | Focus: System Design deep-dive

Time	Round Type	Topics
09:00-10:00	System Design 1	YouTube — upload, transcode, CDN, recommendations
10:15-11:15	System Design 2	WhatsApp — messaging, presence, E2E, scale
11:30-12:00	Node.js Rapid Fire	20 conceptual questions under time pressure
14:00-14:45	Behavioral Panel	5 questions simulating panel interview
15:00-16:00	Review	SD recordings, back-of-envelope math audit, fix weak designs

Mock #5 — Day 35 (August 4, 2026) | Focus: FAANG-style panel simulation

Time	Round Type	Topics
09:00-09:45	Coding	1 hard or 2 medium — interview conditions, no hints
10:00-11:00	System Design	Food Delivery or Uber — full 45-min design
11:15-12:00	Backend Deep Dive	Node OR Java — interviewer choice, production scenarios
13:00-13:45	SQL/Data	3 SQL problems + data modeling question

Time	Round Type	Topics
14:00-14:45	Behavioral/HM	Hiring manager round: motivation, team fit, questions for them
15:00-16:00	Final Review	Overall readiness score, Final Sprint priorities for Days 36-39

Mock Interview Best Practices

- Use timer — strict 45 min for coding, 45 min for system design
- No hints during coding rounds; verbalize thought process continuously
- Record audio/video for review — note filler words and pacing
- Score each round immediately using Appendix C scorecards
- Identify top 3 gaps; schedule remedial study for next 2 days
- Alternate peer mocks with paid platforms (Pramp, Interviewing.io) if available

Section 15: Spaced Repetition Revision Calendar

Spaced repetition schedule: revisit material at Day+0, +2, +6, +13, +20, +29. This aligns with LeetCode problem revision dates and daily topic reviews.

Revision Interval Reference

Interval	Action
Day +0	Initial learning — solve problem or study topic
Day +2	First revision — re-solve without notes, 50% time limit
Day +6	Second revision — explain pattern aloud, solve from scratch
Day +13	Third revision — timed re-solve, note any hesitation
Day +20	Fourth revision — mock interview conditions
Day +29	Final revision — must solve in target time with clean code

Topic Revision Calendar by Start Day

Start Day	Revision Days	Topic Focus
Day 1	D1, D3, D7, D14, D21, D30	Map the Node event loop cold — this is your home t
Day 2	D2, D4, D8, D15, D22, D31	Solidify module system knowledge and sketch URL Sh
Day 3	D3, D5, D9, D16, D23, D32	Master streams — frequent in Node interviews — and
Day 4	D4, D6, D10, D17, D24, D33	Express middleware depth — you'll live here in bac
Day 5	D5, D7, D11, D18, D25, D34	Async mastery plus Rate Limiter design — both high
Day 6	D6, D8, D12, D19, D26, D35	Production-grade error handling — differentiate se
Day 7	D7, D9, D13, D20, D27, D36	Week 1 mock day — test Node/SQL/behavioral under p
Day 8	D8, D10, D14, D21, D28, D37	Auth is non-negotiable for backend roles — nail JW
Day 9	D9, D11, D15, D22, D29, D38	API design polish — interviewers probe REST depth
Day 10	D10, D12, D16, D23, D30, D39	Close Foundation phase with testing discipline on
Day 11	D11, D13, D17, D24, D31	Bridge MERN Mongo skills to Mongoose + start Sprin
Day 12	D12, D14, D18, D25, D32	Caching patterns — essential for system design and
Day 13	D13, D15, D19, D26, D33	Rate limiting implementation ties Node skills to t
Day 14	D14, D16, D20, D27, D34	Week 2 mock — Spring + distributed cache design; p
Day 15	D15, D17, D21, D28, D35	File upload flows appear in many backend interview
Day 16	D16, D18, D22, D29, D36	Real-time Node (Socket.io) plus News Feed design —
Day 17	D17, D19, D23, D30, D37	Observability is a senior signal — implement it, d
Day 18	D18, D20, D24, D31, D38	Security depth closes a common gap — OWASP answers
Day 19	D19, D21, D25, D32, D39	Microservices narrative + Search design — connect
Day 20	D20, D22, D26, D33	Close Build phase — GraphQL + Spring testing round
Day 21	D21, D23, D27, D34	Week 3 mock + Payment Gateway — intensity ramps; r
Day 22	D22, D24, D28, D35	Patterns day — map OOP patterns to Node idioms; Tw
Day 23	D23, D25, D29, D36	TypeScript strengthens your Node story; Completabl
Day 24	D24, D26, D30, D37	Cloud deployment patterns — show you can ship, not
Day 25	D25, D27, D31, D38	Event-driven patterns bridge Node and Spring — cri
Day 26	D26, D28, D32, D39	WhatsApp design is peak SD practice — focus on mes
Day 27	D27, D29, D33	Contract testing + Mongo replication — production
Day 28	D28, D30, D34	Week 4 mock centers on system design — YouTube pre

Start Day	Revision Days	Topic Focus
Day 29	D29, D31, D35	Peak phase starts — architecture layering shows se
Day 30	D30, D32, D36	Connect 5yr MERN experience to interview narrative
Day 31	D31, D33, D37	Netflix SD + full-stack rapid review — simulate in
Day 32	D32, D34, D38	Crypto and security signing — common in payment an
Day 33	D33, D35, D39	Coding implementation day — LRU and Uber design.
Day 34	D34, D36	Translate real experience into compelling intervie
Day 35	D35, D37	Week 5 final mock — treat as real FAANG panel; Foo
Day 36	D36, D38	Final Sprint — fix mock gaps only, no new topics.
Day 37	D37, D39	Teach-back method — if you can teach it, interview
Day 38	D38	Rest and recall — protect sleep; cognitive freshne
Day 39	D39	Day before interviews — rest, light review, logist

Weekly Consolidation Days

Day	Consolidation Focus
Day 7	Review all Week 1 problems marked 'Again'; re-read Node event loop notes
Day 14	Review Week 2 gaps from Mock #1 post-mortem; SQL timed set
Day 21	Review Week 3; Mock #3 prep; system design flash cards
Day 28	Full week review before Final Sprint; Mock #4 debrief
Day 35	Mock #5 debrief; prioritize Final Sprint weak areas only

Section 16: Progress Dashboard

Track daily completion across all workstreams. Mark ■ → ■ in your copy. Weekly summary rows aggregate totals for accountability.

Day	Date	Phase	LC	Node	Java	SQL	SD	Apps	Beh
1	July 1	Foun	■	■	■	■	—	■/2	■
2	July 2	Foun	■	■	■	■	■	■/2	■
3	July 3	Foun	■	■	■	■	—	■/2	■
4	July 4	Foun	■	■	■	■	—	■/3	■
5	July 5	Foun	■	■	■	■	■	■/3	■
6	July 6	Foun	■	■	■	■	—	■/3	■
7	July 7	Foun	■	■	■	■	—	■/3	■
8	July 8	Foun	■	■	■	■	■	■/3	■
9	July 9	Foun	■	■	■	■	—	■/3	■
10	July 1	Foun	■	■	■	■	—	■/3	■
11	July 1	Buil	■	■	■	■	■	■/4	■
12	July 1	Buil	■	■	■	■	—	■/4	■
13	July 1	Buil	■	■	■	■	—	■/4	■
14	July 1	Buil	■	■	■	■	■	■/4	■
15	July 1	Buil	■	■	■	■	—	■/4	■
16	July 1	Buil	■	■	■	■	■	■/5	■
17	July 1	Buil	■	■	■	■	—	■/5	■
18	July 1	Buil	■	■	■	■	—	■/5	■
19	July 1	Buil	■	■	■	■	■	■/5	■
20	July 2	Buil	■	■	■	■	—	■/5	■
21	July 2	Inte	■	■	■	■	—	■/6	■
22	July 2	Inte	■	■	■	■	■	■/6	■
23	July 2	Inte	■	■	■	■	—	■/6	■
24	July 2	Inte	■	■	■	■	■	■/6	■
25	July 2	Inte	■	■	■	■	—	■/7	■
26	July 2	Inte	■	■	■	■	■	■/7	■
27	July 2	Inte	■	■	■	■	—	■/7	■
28	July 2	Inte	■	■	■	■	■	■/7	■
29	July 2	Peak	■	■	■	■	—	■/8	■
30	July 3	Peak	■	■	■	■	—	■/8	■
31	July 3	Peak	■	■	■	■	■	■/9	■
32	August	Peak	■	■	■	■	—	■/9	■
33	August	Peak	■	■	■	■	■	■/9	■
34	August	Peak	■	■	■	■	—	■/10	■
35	August	Peak	■	■	■	■	■	■/10	■
36	August	Fina	■	■	■	■	—	■/8	■
37	August	Fina	■	■	■	■	■	■/7	■
38	August	Fina	■	■	■	■	—	■/5	■
39	August	Fina	■	■	■	■	—	■/3	■

Weekly Summary Template

Week	Days	LeetCode	Mocks	Applications	Score Change
Week 1	Days 1–7	Problems: __/119	Mocks: 0	Apps: __	Readiness Δ: __
Week 2	Days 8–14	Problems: __/152	Mocks: 1	Apps: __	Readiness Δ: __
Week 3	Days 15–21	Problems: __/184	Mocks: 1	Apps: __	Readiness Δ: __
Week 4	Days 22–28	Problems: __/217	Mocks: 1	Apps: __	Readiness Δ: __
Week 5	Days 29–35	Problems: __/252	Mocks: 1	Apps: __	Readiness Δ: __
Final	Days 36–39	Problems: __/152	Mocks: 0	Apps: __	Readiness Δ: __
Week 6	Days 36–39	Review only	Mocks: 1	Apps: __	Final score: __

Key Metrics to Track

- LeetCode problems solved / re-solved within time target
- SQL problems completed under 25-minute timer
- System designs whiteboarded (target: 15 total)
- Behavioral stories rehearsed with recording (target: 10 stories)
- Mock interview average score (target: 7+/10 by Day 35)
- Applications submitted vs responses vs interviews scheduled

Section 17: Final Week Strategy (Aug 1–8)

Days 36–39 are about consolidation, rest, and interview readiness — not new material. Protect sleep and cognitive freshness over cramming.

Day	Date	Phase	Strategy
36	August 5, 2026	Final Sprint	Final Sprint — fix mock gaps only, no new topics.
37	August 6, 2026	Final Sprint	Teach-back method — if you can teach it, interview answers flow naturally.
38	August 7, 2026	Final Sprint	Rest and recall — protect sleep; cognitive freshness beats cramming.
39	August 8, 2026	Final Sprint	Day before interviews — rest, light review, logistics, confidence.

Final Week Daily Priorities

Day 36 (August 5, 2026): Final Sprint — fix mock gaps only, no new topics.

- Node: Light Node review: focus only on flagged weak areas from mock
- Java: Light Java review: focus only on flagged weak areas from mock
- SQL: Light SQL: redo 3 missed problems only
- Behavioral: Rest and rehearse top 3 STAR stories — muscle memory, not new content.
- Applications target: 8

Day 37 (August 6, 2026): Teach-back method — if you can teach it, interview answers flow naturally.

- Node: Node confidence builder: teach-back session — explain core concepts to
- Java: Java confidence builder: teach-back Spring Boot request lifecycle end-
- SQL: SQL confidence: verbalize approach before writing query for 3 classics
- Behavioral: Review company-specific talking points for top 5 target companies.
- Applications target: 7
- System Design: Food Delivery

Day 38 (August 7, 2026): Rest and recall — protect sleep; cognitive freshness beats cramming.

- Node: Rest day active recall: Node flashcards only, no new coding
- Java: Rest day active recall: Java flashcards only
- SQL: Rest day: one SQL problem timed at interview pace
- Behavioral: Light behavioral review — read STAR stories once, no rewriting.
- Applications target: 5

Day 39 (August 8, 2026): Day before interviews — rest, light review, logistics, confidence.

- Node: Interview day prep: Node mental model one-page cheat sheet review
- Java: Interview day prep: Java/Spring one-page cheat sheet review
- SQL: Interview day prep: SQL pattern templates one-page review
- Behavioral: Prepare opening 'Tell me about yourself' 90-second script and tomorrow
- Applications target: 3

Offer Negotiation Tips

- Never disclose current compensation first — ask for their range
- Anchor high with researched market data (Levels.fyi, Glassdoor, Blind)
- Negotiate total comp — base, bonus, equity, signing, relocation separately
- Get competing offers to strengthen leverage — share timelines honestly

■ Ask for 48–72 hours to review written offers before accepting
■ Request sign-on bonus to offset unvested equity from current role
■ Clarify equity — vesting schedule, cliff, refresh grants, valuation method
■ Negotiate start date for prep time if needed before Day 1
■ Get role level confirmed in writing — Senior vs Staff affects future growth
■ Ask about remote policy, on-call expectations, and promotion timeline

Day-Before-Interview Checklist

■ Confirm interview time, timezone, and platform (Zoom/Teams/HackerRank)
■ Prepare quiet space, backup internet, charged laptop, water
■ Review one-page cheat sheets only — no new problems
■ Lay out clothes; plan to sleep by 22:30
■ Prepare 3 thoughtful questions for the interviewer
■ Rehearse 90-second 'Tell me about yourself' once
■ Set two alarms; calendar reminder 30 min before

Appendix A: Full 39-Day Daily Playbook

Complete daily checklist for all 39 days. Mark each item when done. Follow the schedule in Section 2 for time blocks.

Day 1 — July 1, 2026 | Foundation | Map the Node event loop cold — this is your home turf, make it interview-crisp.

Daily Checklist
Morning LeetCode Block 1 (15 problems)
LeetCode #448: Find Disappeared Numbers
LeetCode #13: Roman to Integer
LeetCode #36: Valid Sudoku
LeetCode #56: Merge Intervals
LeetCode #11: Container With Most Water
... +10 more problems (see Section 3)
Node.js: Node.js Event Loop, libuv, and non-blocking I/O architecture
Node exercise: Implement a task scheduler using setImmediate, process.nextTick,
Java: Java platform overview: JVM, JRE, JDK, bytecode, and compilation
Java exercise: Write a HelloWorld plus a class demonstrating primitives, wrapper
SQL: SQL fundamentals: SELECT, WHERE, ORDER BY, LIMIT, and basic filte
SQL problem 1: Select all employees earning above the department avera
SQL problem 2: Find the top 5 products by revenue in the last 30 days.
SQL problem 3: List customers who registered but never placed an order
OS: Processes vs threads: address space, context switching, and sched
Networks: OSI and TCP/IP models: layers, encapsulation, and common protocol
Behavioral STAR: Tell me about a time you took ownership of a production incident
Submit 2 job application(s)
Recruiter outreach: 1 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 2 — July 2, 2026 | Foundation | Solidify module system knowledge and sketch URL Shortener end-to-end.

Daily Checklist
Morning LeetCode Block 1 (16 problems)
LeetCode #860: Lemonade Change
LeetCode #70: Climbing Stairs
LeetCode #743: Network Delay Time
LeetCode #146: LRU Cache
LeetCode #53: Maximum Subarray
... +11 more problems (see Section 3)
Node.js: CommonJS vs ESM modules, require vs import, and module resolution
Node exercise: Build a mini plugin loader that dynamically imports ESM modules f
Java: Java data types, variables, operators, and control flow

Daily Checklist
Java exercise: Implement FizzBuzz plus a switch-based menu CLI that validates nu
SQL: INNER JOIN, LEFT JOIN, RIGHT JOIN, and FULL OUTER JOIN patterns
SQL problem 1: Join orders to customers and show order count per custo
SQL problem 2: Find products with zero sales using a LEFT JOIN anti-pa
SQL problem 3: Return employees and their managers; include employees
Networks: IP addressing, subnets, CIDR notation, and NAT fundamentals
System Design: URL Shortener
Behavioral STAR: Describe a situation where you had conflicting priorities from st
Submit 2 job application(s)
Recruiter outreach: 1 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 3 — July 3, 2026 | Foundation | Master streams — frequent in Node interviews — and Mongo document fundamentals.

Daily Checklist
Morning LeetCode Block 1 (16 problems)
LeetCode #665: Non-decreasing Array
LeetCode #746: Min Cost Climbing Stairs
LeetCode #787: Cheapest Flights K Stops
LeetCode #281: Zigzag Iterator
LeetCode #238: Product of Array Except Self
... +11 more problems (see Section 3)
Node.js: Streams API: readable, writable, duplex, transform, backpressure
Node exercise: Build a file pipeline: gzip compress a large log file using strea
Java: Object-oriented programming: classes, inheritance, polymorphism,
Java exercise: Model a simple banking system with Account, SavingsAccount, and C
SQL: GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MIN,
SQL problem 1: Find departments with average salary greater than compa
SQL problem 2: Count active users per signup month for the past year.
SQL problem 3: List categories where total sales exceed $100,000 using
MongoDB: MongoDB document model, BSON types, _id generation, and CRUD inse
OS: CPU scheduling: FCFS, SJF, round-robin, and context switch cost
Behavioral STAR: Give an example of mentoring or upskilling a junior developer on
Submit 2 job application(s)
Recruiter outreach: 1 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 4 — July 4, 2026 | Foundation | Express middleware depth — you'll live here in backend rounds.

Daily Checklist
Morning LeetCode Block 1 (17 problems)
LeetCode #485: Max Consecutive Ones
LeetCode #28: Find the Index of the First Occurrence
LeetCode #454: 4Sum II
LeetCode #324: Wiggle Sort II
LeetCode #18: 4Sum
... +12 more problems (see Section 3)
Node.js: Express.js middleware chain, routing, and error-handling patterns
Node exercise: Create an Express API with auth middleware, request logging, cent
Java: Interfaces, abstract classes, and composition vs inheritance
Java exercise: Refactor the banking exercise to use interfaces (Payable, Transfe
SQL: Subqueries: scalar, correlated, EXISTS, IN, and derived tables
SQL problem 1: Find employees earning more than their department avera
SQL problem 2: List products that exist in wishlists but were never pu

Daily Checklist
SQL problem 3: Use EXISTS to find customers with at least 3 orders in
Networks: TCP vs UDP: handshake, reliability, use cases, and port numbers
Behavioral STAR: Tell me about a time you improved code quality or reduced technic
Submit 3 job application(s)
Recruiter outreach: 1 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 5 — July 5, 2026 | Foundation | Async mastery plus Rate Limiter design — both high-frequency interview topics.

Daily Checklist
Morning LeetCode Block 1 (18 problems)
LeetCode #724: Find Pivot Index
LeetCode #415: Add Strings
LeetCode #560: Subarray Sum Equals K
LeetCode #912: Sort an Array
LeetCode #167: Two Sum II - Input Array Is Sorted
... +13 more problems (see Section 3)
Node.js: Async patterns: callbacks, Promises, async/await, and error propa
Node exercise: Convert a callback-based fs utility library to Promises, then to
Java: Collections framework: List, Set, Map, Queue — ArrayList vs Linke
Java exercise: Implement word frequency counter using HashMap and sort results b
SQL: Window functions: ROW_NUMBER, RANK, DENSE_RANK, LEAD, LAG, PARTIT
SQL problem 1: Rank employees by salary within each department.
SQL problem 2: Find the second highest salary per department using DEN
SQL problem 3: Calculate month-over-month revenue change using LAG.
OS: Memory management: paging, segmentation, virtual memory, page fau
System Design: Rate Limiter
Behavioral STAR: Describe a time you missed a deadline. What happened and what did
Submit 3 job application(s)
Recruiter outreach: 2 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 6 — July 6, 2026 | Foundation | Production-grade error handling — differentiate senior from mid-level answers.

Daily Checklist
Morning LeetCode Block 1 (18 problems)
LeetCode #136: Single Number
LeetCode #541: Reverse String II
LeetCode #974: Subarray Sums Divisible by K
LeetCode #539: Minimum Time Difference
LeetCode #611: Valid Triangle Number
... +13 more problems (see Section 3)
Node.js: Error handling: operational vs programmer errors, domains (legacy
Node exercise: Add SIGTERM/SIGINT handlers, drain in-flight requests, close DB c
Java: Exception handling: checked vs unchecked, try-with-resources, cus
Java exercise: Build a file parser that uses try-with-resources and throws custo
SQL: Indexing: B-tree indexes, composite indexes, covering indexes, an
SQL problem 1: Given a slow query on orders(user_id, created_at), desi
SQL problem 2: Use EXPLAIN to compare query plans with and without a c

Daily Checklist
SQL problem 3: Identify redundant indexes on a denormalized analytics
MongoDB: MongoDB indexing: single, compound, multikey, text indexes, and i
OS: Deadlocks: four necessary conditions, detection, prevention, and
Networks: HTTP/1.1 vs HTTP/2 vs HTTP/3: multiplexing, head-of-line blocking
Behavioral STAR: Tell me about a feature you shipped that had measurable business
Submit 3 job application(s)
Recruiter outreach: 2 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 7 — July 7, 2026 | Foundation | Week 1 mock day — test Node/SQL/behavioral under pressure, then review gaps.

Daily Checklist
Morning LeetCode Block 1 (19 problems)
LeetCode #709: To Lower Case
LeetCode #1: Two Sum
LeetCode #881: Boats to Save People
LeetCode #567: Permutation in String
LeetCode #162: Find Peak Element
... +14 more problems (see Section 3)
Node.js: Node.js cluster module and worker threads for CPU-bound tasks
Node exercise: Build a cluster-based HTTP server that distributes requests acros
Java: Generics: type parameters, bounded types, wildcards, and type era
Java exercise: Implement a generic Pair<T,U> and a bounded method findMax(List<T
SQL: Transactions: ACID properties, isolation levels, deadlocks in SQL
SQL problem 1: Simulate a bank transfer with BEGIN/COMMIT/ROLLBACK and
SQL problem 2: Demonstrate dirty read vs repeatable read under differe
SQL problem 3: Resolve a deadlock scenario between two concurrent upda
OS: Synchronization primitives: mutex, semaphore, monitor, and critic
Behavioral STAR: Walk through your strongest project end-to-end using STAR (Situat
Submit 3 job application(s)
Recruiter outreach: 2 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today
Mock interview: Week 1 Full Mock (Node + SQL + Behavioral)

Day 8 — July 8, 2026 | Foundation | Auth is non-negotiable for backend roles — nail JWT and OAuth flows.

Daily Checklist
Morning LeetCode Block 1 (20 problems)
LeetCode #242: Valid Anagram
LeetCode #252: Meeting Rooms
LeetCode #26: Remove Duplicates from Sorted Array
LeetCode #713: Subarray Product Less Than K
LeetCode #875: Koko Eating Bananas
... +15 more problems (see Section 3)
Node.js: Authentication: JWT, sessions, OAuth2/OIDC flows, and refresh tok
Node exercise: Implement JWT auth with access/refresh tokens, rotation on refres
Java: Java 8+ features: lambdas, functional interfaces, Stream API, Opt
Java exercise: Refactor word frequency to use Stream API: filter, map, collect,
SQL: Normalization: 1NF through 3NF, denormalization tradeoffs, and st
SQL problem 1: Normalize an unnormalized order spreadsheet into 3NF ta
SQL problem 2: Identify update anomalies in a denormalized product cat

Daily Checklist
SQL problem 3: Design a star schema for sales analytics from OLTP tabl
Networks: DNS resolution, record types (A, AAAA, CNAME, MX), and caching
System Design: Notification System
Behavioral STAR: Describe a time you disagreed with a technical decision. How was
Submit 3 job application(s)
Recruiter outreach: 2 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 9 — July 9, 2026 | Foundation | API design polish — interviewers probe REST depth even for Node roles.

Daily Checklist
Morning LeetCode Block 1 (20 problems)
LeetCode #205: Isomorphic Strings
LeetCode #27: Remove Element
LeetCode #35: Search Insert Position
LeetCode #150: Evaluate Reverse Polish Notation
LeetCode #2: Add Two Numbers
... +15 more problems (see Section 3)
Node.js: Validation, serialization, and API design: REST conventions, stat
Node exercise: Build a paginated REST CRUD for products with Joi/Zod validation,
Java: String, StringBuilder, StringBuffer, and immutability performance
Java exercise: Benchmark string concatenation in a loop: String vs StringBuilder
SQL: UNION, INTERSECT, EXCEPT, and set operations on query results
SQL problem 1: Find users who bought product A but not product B using
SQL problem 2: Combine monthly sales from two regions with UNION ALL a
SQL problem 3: Use INTERSECT to find VIP customers active in both web
MongoDB: MongoDB aggregation pipeline: $match, $group, $lookup, $project,
OS: File systems: inodes, directories, journaling, and ext4 vs NTFS c
Behavioral STAR: Tell me about a time you learned a new technology quickly to deli
Submit 3 job application(s)
Recruiter outreach: 2 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 10 — July 10, 2026 | Foundation | Close Foundation phase with testing discipline on both stacks.

Daily Checklist
Morning LeetCode Block 1 (21 problems)
LeetCode #20: Valid Parentheses
LeetCode #225: Implement Stack using Queues
LeetCode #21: Merge Two Sorted Lists
LeetCode #109: Convert Sorted List to BST
LeetCode #701: Insert into BST
... +16 more problems (see Section 3)
Node.js: Testing Node apps: Jest/Vitest, supertest, mocking, and TDD workf
Node exercise: Write unit tests for a service layer and integration tests for Ex
Java: JUnit 5 basics: annotations, assertions, parameterized tests, and
Java exercise: Write JUnit 5 tests for the banking system with @BeforeEach setup
SQL: CTEs (WITH clauses): recursive CTEs, readability, and complex que
SQL problem 1: Use recursive CTE to generate a date series for gap-fil
SQL problem 2: Build an org hierarchy report with recursive employee-m

Daily Checklist
SQL problem 3: Rewrite a nested subquery as a readable CTE for monthly
Networks: TLS/SSL handshake, certificates, cipher suites, and HTTPS termina
Behavioral STAR: Describe your most challenging bug and how you debugged it system
Submit 3 job application(s)
Recruiter outreach: 2 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 11 — July 11, 2026 | Build | Bridge MERN Mongo skills to Mongoose + start Spring Boot journey.

Daily Checklist
Morning LeetCode Block 1 (22 problems)
LeetCode #108: Convert Sorted Array to Binary Search Tree
LeetCode #1046: Last Stone Weight
LeetCode #14: Longest Common Prefix
LeetCode #417: Pacific Atlantic Water Flow
LeetCode #1136: Parallel Courses
... +17 more problems (see Section 3)
Node.js: MongoDB with Mongoose: schemas, middleware, population, and trans
Node exercise: Build a blog API with Mongoose schemas, pre-save hooks, populate
Java: Spring Boot introduction: project structure, auto-configuration,
Java exercise: Create a Spring Boot REST app with one GET /health endpoint using
SQL: SQL performance tuning: query rewriting, N+1 detection, and slow
SQL problem 1: Rewrite a correlated subquery as a JOIN for better perf
SQL problem 2: Identify and fix an ORM-caused N+1 query pattern.
SQL problem 3: Optimize a report query scanning 10M rows using partiti
MongoDB: Mongoose schema design patterns: discriminators, virtuals, and le
OS: Virtualization vs containers: hypervisors, cgroups, namespaces ov
System Design: Chat System
Behavioral STAR: Tell me about a time you collaborated cross-functionally with pro
Submit 4 job application(s)
Recruiter outreach: 2 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 12 — July 12, 2026 | Build | Caching patterns — essential for system design and Node production apps.

Daily Checklist
Morning LeetCode Block 1 (22 problems)
LeetCode #219: Contains Duplicate II
LeetCode #125: Valid Palindrome
LeetCode #69: Sqrt(x)
LeetCode #173: Binary Search Tree Iterator
LeetCode #61: Rotate List
... +17 more problems (see Section 3)
Node.js: Redis caching: cache-aside, write-through, TTL strategies, and ca
Node exercise: Add Redis cache-aside to product API; implement cache stampede pr
Java: Spring Core IoC: @Component, @Service, @Repository, @Autowired, b
Java exercise: Build a layered app: Controller → Service → Repository with const
SQL: Stored procedures, functions, and triggers — when to use vs appli
SQL problem 1: Write a trigger to audit all UPDATE operations on a sen

Daily Checklist
SQL problem 2: Create a stored function to calculate discounted price
SQL problem 3: Debate: move business logic to stored proc vs service l
MongoDB: Redis vs MongoDB use cases: when to use each as primary vs auxili
Networks: Load balancing: L4 vs L7, algorithms (round-robin, least connecti
Behavioral STAR: Give an example of receiving harsh feedback and how you responded
Submit 4 job application(s)
Recruiter outreach: 2 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 13 — July 13, 2026 | Build | Rate limiting implementation ties Node skills to tomorrow's system design review.

Daily Checklist
Morning LeetCode Block 1 (23 problems)
LeetCode #283: Move Zeroes
LeetCode #278: First Bad Version
LeetCode #496: Next Greater Element I
LeetCode #82: Remove Duplicates from Sorted List II
LeetCode #114: Flatten Binary Tree to Linked List
... +18 more problems (see Section 3)
Node.js: Rate limiting and throttling: token bucket, sliding window, expre
Node exercise: Implement distributed rate limiter using Redis sliding window log
Java: Spring Boot REST controllers: @RestController, @RequestMapping, D
Java exercise: Build CRUD REST API for Todo items with proper HTTP status codes
SQL: Database design: ER modeling, cardinality, and schema migration s
SQL problem 1: Design ER diagram for e-commerce: users, orders, produc
SQL problem 2: Write migration SQL to add soft-delete column with back
SQL problem 3: Model many-to-many author-book relationship with juncti
OS: Docker fundamentals: images, containers, Dockerfile, layers, and
Networks: WebSockets vs SSE vs long polling — real-time communication trade
Behavioral STAR: Describe a time you had to deliver under ambiguous requirements.
Submit 4 job application(s)
Recruiter outreach: 3 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 14 — July 14, 2026 | Build | Week 2 mock — Spring + distributed cache design; prioritize gap review.

Daily Checklist
Morning LeetCode Block 1 (24 problems)
LeetCode #157: Read N Characters Given Read4
LeetCode #290: Word Pattern
LeetCode #344: Reverse String
LeetCode #904: Fruit Into Baskets
LeetCode #1011: Capacity Ship Packages
... +19 more problems (see Section 3)
Node.js: Message queues with Bull/BullMQ and RabbitMQ: producers, consumer
Node exercise: Build email notification worker using BullMQ with retry backoff,
Java: Spring Data JPA: entities, repositories, @Query, relationships, a
Java exercise: Create JPA entities for User and Order with @OneToMany; implement
SQL: SQL joins deep dive: self-joins, cross joins, and join order opti
SQL problem 1: Find consecutive login days per user using self-join or
SQL problem 2: Detect overlapping employee shift schedules with a self

Daily Checklist
SQL problem 3: Analyze join order impact on a 5-table query using EXPL
Networks: CDN architecture: edge caching, cache keys, purge, and origin shi
System Design: Distributed Cache
Behavioral STAR: Week 2 behavioral prep: refine 5 STAR stories covering leadership
Submit 4 job application(s)
Recruiter outreach: 3 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today
Mock interview: Week 2 Full Mock (Java Spring + System Design)

Day 15 — July 15, 2026 | Build | File upload flows appear in many backend interviews — know S3 patterns cold.

Daily Checklist
Morning LeetCode Block 1 (24 problems)
LeetCode #349: Intersection of Two Arrays
LeetCode #680: Valid Palindrome II
LeetCode #704: Binary Search
LeetCode #388: Longest Absolute File Path
LeetCode #143: Reorder List
... +19 more problems (see Section 3)
Node.js: File uploads: multipart, streaming uploads, S3 presigned URLs, an
Node exercise: Implement direct-to-S3 upload flow with presigned URLs and metada
Java: Spring Boot validation: @Valid, Bean Validation, custom constrain
Java exercise: Add @Valid to Todo API with custom @UniqueEmail constraint and @C
SQL: PostgreSQL-specific: JSONB columns, arrays, and full-text search
SQL problem 1: Query JSONB metadata field for products matching nested
SQL problem 2: Implement full-text search on article title and body wi
SQL problem 3: Use unnest on array column to find tags shared across p
MongoDB: GridFS and large file storage in MongoDB vs S3 tradeoffs
OS: Linux process management: ps, top, signals (SIGKILL vs SIGTERM),
Behavioral STAR: Tell me about optimizing application performance — what metrics d
Submit 4 job application(s)
Recruiter outreach: 3 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 16 — July 16, 2026 | Build | Real-time Node (Socket.io) plus News Feed design — high-yield combo.

Daily Checklist
Morning LeetCode Block 1 (25 problems)
LeetCode #350: Intersection of Two Arrays II
LeetCode #922: Sort Array By Parity
LeetCode #374: Guess Number Higher or Lower
LeetCode #456: 132 Pattern
LeetCode #148: Sort List
... +20 more problems (see Section 3)
Node.js: WebSockets and Socket.io: rooms, namespaces, scaling with Redis a
Node exercise: Build real-time chat room with Socket.io, JWT auth on handshake,
Java: Spring Boot configuration: application.properties/yml, profiles,
Java exercise: Externalize config with dev/prod profiles and type-safe @Configur
SQL: Database replication: leader-follower, read replicas, replication
SQL problem 1: Design read routing: writes to primary, analytics reads
SQL problem 2: Write query to detect replication lag using heartbeat t

Daily Checklist
SQL problem 3: Plan failover steps when primary goes down (conceptual
Networks: API Gateway patterns: routing, auth termination, rate limiting, a
System Design: News Feed
Behavioral STAR: Describe a time you had to say no to a feature request. How did y
Submit 5 job application(s)
Recruiter outreach: 3 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 17 — July 17, 2026 | Build | Observability is a senior signal — implement it, don't just talk about it.

Daily Checklist
Morning LeetCode Block 1 (26 problems)
LeetCode #359: Logger Rate Limiter
LeetCode #977: Squares of Sorted Array
LeetCode #232: Implement Queue using Stacks
LeetCode #369: Plus One Linked List
LeetCode #156: Binary Tree Upside Down
... +21 more problems (see Section 3)
Node.js: Logging and observability: structured logging (pino/winston), cor
Node exercise: Add pino structured logging with request correlation ID middlewar
Java: Spring Boot logging: SLF4J, Logback, MDC, and structured JSON log
Java exercise: Configure Logback JSON appenders and propagate correlation ID via
SQL: Sharding and partitioning: horizontal vs vertical, shard keys, an
SQL problem 1: Design shard key for a multi-tenant SaaS orders table.
SQL problem 2: Write queries for a range-partitioned events table by m
SQL problem 3: Identify hot shard problem and propose resharding strat
OS: Concurrency models: preemptive vs cooperative multitasking, async
Networks: gRPC vs REST: protobuf, HTTP/2, streaming, and when to choose eac
Behavioral STAR: Tell me about a time you improved team processes or documentation
Submit 5 job application(s)
Recruiter outreach: 3 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 18 — July 18, 2026 | Build | Security depth closes a common gap — OWASP answers must be automatic.

Daily Checklist
Morning LeetCode Block 1 (26 problems)
LeetCode #346: Moving Average from Data Stream
LeetCode #141: Linked List Cycle
LeetCode #94: Binary Tree Inorder Traversal
LeetCode #490: The Maze
LeetCode #47: Permutations II
... +21 more problems (see Section 3)
Node.js: Security best practices: OWASP Top 10, helmet, CORS, CSRF, XSS, S
Node exercise: Harden an Express API: helmet headers, parameterized queries, inp
Java: Spring Security basics: filter chain, authentication, authorizati
Java exercise: Add Spring Security with form login disabled, HTTP Basic or JWT f
SQL: SQL injection prevention: prepared statements, ORM safety, and in
SQL problem 1: Demonstrate vulnerable dynamic SQL then fix with parame
SQL problem 2: Audit ORM raw query usage for injection risks.

Daily Checklist
SQL problem 3: Write safe dynamic ORDER BY using allowlist approach.
MongoDB: MongoDB security: role-based access, field-level encryption, and
Networks: Reverse proxy vs forward proxy: Nginx configuration concepts
Behavioral STAR: Describe a production outage you caused or contributed to. What w
Submit 5 job application(s)
Recruiter outreach: 3 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 19 — July 19, 2026 | Build | Microservices narrative + Search design — connect your MERN experience to scale.

Daily Checklist
Morning LeetCode Block 1 (27 problems)
LeetCode #933: Number of Recent Calls
LeetCode #160: Intersection of Two Linked Lists
LeetCode #104: Maximum Depth of Binary Tree
LeetCode #530: Minimum Absolute Difference in BST
LeetCode #529: Minesweeper
... +22 more problems (see Section 3)
Node.js: Microservices in Node: service boundaries, inter-service communic
Node exercise: Split monolith into user-service and order-service communicating
Java: Spring Boot microservices: service discovery intro, RestTemplate/
Java exercise: Create two Spring Boot services; call service B from A using WebC
SQL: CAP theorem applied to databases: consistency vs availability tra
SQL problem 1: Given network partition, choose CP vs AP for payment vs
SQL problem 2: Design multi-region DB strategy with RPO/RTO constraint
SQL problem 3: Compare strong consistency read vs eventual consistency
OS: Kubernetes basics: pods, services, deployments, and horizontal po
System Design: Search Engine
Behavioral STAR: Tell me about a time you had to estimate effort for a complex pro
Submit 5 job application(s)
Recruiter outreach: 3 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 20 — July 20, 2026 | Build | Close Build phase — GraphQL + Spring testing round out full-stack credibility.

Daily Checklist
Morning LeetCode Block 1 (28 problems)
LeetCode #110: Balanced Binary Tree
LeetCode #703: Kth Largest in Stream
LeetCode #733: Flood Fill
LeetCode #163: Missing Ranges
LeetCode #686: Repeated String Match
... +23 more problems (see Section 3)
Node.js: GraphQL with Node: schema design, resolvers, DataLoader, and N+1
Node exercise: Build GraphQL API for blog with Query/Mutation, DataLoader for au
Java: Spring Boot testing: @WebMvcTest, @DataJpaTest, Testcontainers fo
Java exercise: Write @WebMvcTest for Todo controller and Testcontainers PostgreS
SQL: Materialized views, query caching, and read-model denormalization
SQL problem 1: Create materialized view for daily sales summary and re
SQL problem 2: Design denormalized read table fed by CDC from normaliz

Daily Checklist
SQL problem 3: Compare materialized view refresh vs incremental rollup
Networks: Service mesh intro: sidecar proxy, mTLS, traffic management (Isti
Behavioral STAR: Describe your ideal team culture and how you contribute to it.
Submit 5 job application(s)
Recruiter outreach: 3 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 21 — July 21, 2026 | Intensify | Week 3 mock + Payment Gateway — intensity ramps; review DSA weak spots.

Daily Checklist
Morning LeetCode Block 1 (28 problems)
LeetCode #387: First Unique Character
LeetCode #88: Merge Sorted Array
LeetCode #206: Reverse Linked List
LeetCode #144: Binary Tree Preorder Traversal
LeetCode #565: Array Nesting
... +23 more problems (see Section 3)
Node.js: Performance tuning Node: profiling, clinic.js, memory leaks, and
Node exercise: Profile a slow endpoint with clinic doctor; fix memory leak from
Java: JVM performance: heap tuning, GC algorithms (G1, ZGC), and JVM fl
Java exercise: Run Spring Boot app with different heap sizes; observe GC logs an
SQL: Complex interview SQL: running totals, gaps-and-islands, and pivo
SQL problem 1: Compute running total of orders per customer ordered by
SQL problem 2: Find gap days in server uptime logs (gaps-and-islands).
SQL problem 3: Pivot sales by product category into columns per quarte
MongoDB: MongoDB performance: explain plans, index hints, aggregation opti
OS: Monitoring and alerting: SLIs, SLOs, SLAs, error budgets, and on-
Behavioral STAR: Practice concise 2-minute 'Tell me about yourself' tailored for b
Submit 6 job application(s)
Recruiter outreach: 4 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today
Mock interview: Week 3 Full Mock (Mixed stack + DSA medium)

Day 22 — July 22, 2026 | Intensify | Patterns day — map OOP patterns to Node idioms; Twitter design in afternoon.

Daily Checklist
Morning LeetCode Block 1 (29 problems)
LeetCode #190: Reverse Bits
LeetCode #705: Design HashSet
LeetCode #392: Is Subsequence
LeetCode #234: Palindrome Linked List
LeetCode #428: Serialize N-ary Tree
... +24 more problems (see Section 3)
Node.js: Design patterns in Node: factory, strategy, observer, middleware
Node exercise: Refactor notification system using strategy pattern for email/SMS
Java: Java design patterns: Singleton, Factory, Builder, Observer — idi
Java exercise: Implement Builder pattern for complex Order object; thread-safe S
SQL: LeetCode-style SQL: rank, dedupe, consecutive sequences, and top-
SQL problem 1: Delete duplicate emails keeping row with lowest id.

Daily Checklist
SQL problem 2: Find customers who bought all products in a category.
SQL problem 3: Select top 3 earners per department handling ties.
Networks: Consistent hashing: virtual nodes, ring topology, and minimal res
System Design: Payment Gateway
Behavioral STAR: Tell me about a time you made a data-driven decision.
Submit 6 job application(s)
Recruiter outreach: 4 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 23 — July 23, 2026 | Intensify | TypeScript strengthens your Node story; CompletableFuture mirrors async/await mentally.

Daily Checklist
Morning LeetCode Block 1 (30 problems)
LeetCode #191: Number of 1 Bits
LeetCode #706: Design HashMap
LeetCode #876: Middle of the Linked List
LeetCode #226: Invert Binary Tree
LeetCode #909: Snakes and Ladders
... +25 more problems (see Section 3)
Node.js: TypeScript for Node backends: strict mode, generics, decorators,
Node exercise: Migrate Express JS project to TypeScript with strict null checks,
Java: Java concurrency: ExecutorService, CompletableFuture, synchronize
Java exercise: Fetch data from 3 simulated APIs in parallel using CompletableFut
SQL: Database connection pooling: sizing, timeouts, and leak detection
SQL problem 1: Calculate optimal pool size given DB max connections an
SQL problem 2: Diagnose connection leak from unclosed result sets.
SQL problem 3: Configure read/write pool split for replica routing.
OS: Shell scripting for ops: bash basics, cron, and deployment automa
Networks: DNS load balancing, anycast, and geo-routing for global services
Behavioral STAR: Describe handling a disagreement with your manager.
Submit 6 job application(s)
Recruiter outreach: 4 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 24 — July 24, 2026 | Intensify | Cloud deployment patterns — show you can ship, not just code locally.

Daily Checklist
Morning LeetCode Block 1 (30 problems)
LeetCode #228: Summary Ranges
LeetCode #203: Remove Linked List Elements
LeetCode #543: Diameter of Binary Tree
LeetCode #997: Find Town Judge
LeetCode #646: Maximum Length of Pair Chain
... +25 more problems (see Section 3)
Node.js: Serverless Node: AWS Lambda, cold starts, API Gateway, and server
Node exercise: Deploy Express app as Lambda with serverless-http; optimize cold
Java: Spring Boot on AWS: Elastic Beanstalk, ECS Fargate overview, and
Java exercise: Dockerize Spring Boot app; write Dockerfile with multi-stage buil
SQL: Time-series data modeling: partitioning by time, retention polici
SQL problem 1: Design metrics table partitioned by day with auto-drop
SQL problem 2: Write query to downsample minute data to hourly average

Daily Checklist
SQL problem 3: Index strategy for time-range queries on billion-row ev
MongoDB: MongoDB change streams and CDC patterns for event-driven architec
System Design: Twitter
Behavioral STAR: Tell me about a time you went above and beyond for a customer or
Submit 6 job application(s)
Recruiter outreach: 4 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 25 — July 25, 2026 | Intensify | Event-driven patterns bridge Node and Spring — critical for system design depth.

Daily Checklist
Morning LeetCode Block 1 (31 problems)
LeetCode #237: Delete Node in a Linked List
LeetCode #572: Subtree of Another Tree
LeetCode #231: Power of Two
LeetCode #100: Same Tree
LeetCode #1197: Min Knight Moves
... +26 more problems (see Section 3)
Node.js: Event-driven architecture: EventEmitter, domain events, event sou
Node exercise: Implement order flow publishing OrderCreated/OrderShipped events
Java: Spring events: ApplicationEventPublisher, @EventListener, async e
Java exercise: Publish OrderCreated event on save; handle with @EventListener; d
SQL: Optimistic vs pessimistic locking: version columns, SELECT FOR UP
SQL problem 1: Implement optimistic locking with version column on inv
SQL problem 2: Use SELECT FOR UPDATE to prevent double-spend in wallet
SQL problem 3: Compare lost update problem solutions in high-contentio
OS: CI/CD pipelines: GitHub Actions, build/test/deploy stages, and bl
Networks: Message broker internals: Kafka partitions, consumer groups, and
Behavioral STAR: Describe a time you had to balance speed vs quality in a release.
Submit 7 job application(s)
Recruiter outreach: 4 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 26 — July 26, 2026 | Intensify | WhatsApp design is peak SD practice — focus on messaging at scale.

Daily Checklist
Morning LeetCode Block 1 (32 problems)
LeetCode #101: Symmetric Tree
LeetCode #389: Find the Difference
LeetCode #111: Minimum Depth of Binary Tree
LeetCode #674: Longest Continuous Increasing Subsequence
LeetCode #939: Minimum Area Rectangle
... +27 more problems (see Section 3)
Node.js: Database connection management in Node: pg pool, Mongoose connect
Node exercise: Configure pg Pool and Mongoose with connection limits, retry logi
Java: Spring Boot Actuator: health endpoints, metrics, custom indicator
Java exercise: Add Actuator with custom DB health indicator and expose prometheu
SQL: Data warehousing intro: ETL vs ELT, fact/dimension tables, and ba
SQL problem 1: Design fact table for orders and dimension tables for t
SQL problem 2: Write SQL transform step for daily ETL load from OLTP t

Daily Checklist
SQL problem 3: Handle slowly changing dimension Type 2 for customer ad
Networks: TCP congestion control: slow start, AIMD, and impact on API laten
System Design: Instagram
Behavioral STAR: Tell me about working with a difficult teammate.
Submit 7 job application(s)
Recruiter outreach: 4 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 27 — July 27, 2026 | Intensify | Contract testing + Mongo replication — production readiness topics.

Daily Checklist
Morning LeetCode Block 1 (32 problems)
LeetCode #693: Binary Number with Alternating Bits
LeetCode #747: Largest Number At Least Twice
LeetCode #910: Smallest Range I
LeetCode #1991: Find Middle Index
LeetCode #2131: Longest Palindrome Two Letters
... +27 more problems (see Section 3)
Node.js: API documentation and contract testing: OpenAPI/Swagger, Pact, an
Node exercise: Generate OpenAPI spec from Express routes; write Pact consumer te
Java: SpringDoc OpenAPI: auto-generating Swagger UI, schema annotations
Java exercise: Add springdoc-openapi to Todo API with @Operation annotations and
SQL: Interview recap: 10 classic SQL patterns rapid-fire practice
SQL problem 1: Second highest salary — solve with subquery and with wi
SQL problem 2: Monthly active users retention cohort for 6 months.
SQL problem 3: Find median salary per department (PostgreSQL percentil
MongoDB: MongoDB replica sets: election, read preferences, write concern,
OS: Memory leak debugging in production: heap dumps, core dumps, and
Behavioral STAR: Describe a time you influenced technical direction without formal
Submit 7 job application(s)
Recruiter outreach: 5 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 28 — July 28, 2026 | Intensify | Week 4 mock centers on system design — YouTube prep in morning, mock in afternoon.

Daily Checklist
Morning LeetCode Block 1 (33 problems)
LeetCode #1: Two Sum
LeetCode #20: Valid Parentheses
LeetCode #21: Merge Two Sorted Lists
LeetCode #70: Climbing Stairs
LeetCode #313: Super Ugly Number
... +28 more problems (see Section 3)
Node.js: Node.js internals review: V8, libuv thread pool, C++ addons overv
Node exercise: Write a simple N-API addon that computes fibonacci synchronously;
Java: Java module system (JPMS) overview and Spring Boot 3 on Java 17+
Java exercise: Run Spring Boot 3 app on Java 17; use records for DTOs and sealed
SQL: SQL mock interview set: 5 problems in 90 minutes timed
SQL problem 1: Employees earning more than managers (self-join classic
SQL problem 2: Running difference between consecutive row values.

Daily Checklist
SQL problem 3: Department top 3 salaries with tie handling.
Networks: HTTP caching: Cache-Control, ETag, conditional requests, and CDN
System Design: WhatsApp
Behavioral STAR: Week 4 STAR story rotation — record yourself and critique clarity
Submit 7 job application(s)
Recruiter outreach: 5 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today
Mock interview: Week 4 Full Mock (System Design deep-dive)

Day 29 — July 29, 2026 | Peak | Peak phase starts — architecture layering shows seniority beyond CRUD.

Daily Checklist
Morning LeetCode Block 1 (34 problems)
LeetCode #94: Binary Tree Inorder Traversal
LeetCode #104: Maximum Depth of Binary Tree
LeetCode #141: Linked List Cycle
LeetCode #206: Reverse Linked List
LeetCode #338: Counting Bits
... +29 more problems (see Section 3)
Node.js: Senior Node topics: hexagonal architecture, DDD tactical patterns
Node exercise: Restructure a CRUD app into domain/use-case/infrastructure layers
Java: Spring Boot clean architecture: domain layer, use cases, adapters
Java exercise: Reorganize Todo app into domain/application/infrastructure packag
SQL: Database interview rapid review: ACID, indexes, joins, windows, t
SQL problem 1: Design schema for ride-sharing app core entities.
SQL problem 2: Optimize slow query provided in prompt using indexes.
SQL problem 3: Explain CAP tradeoff choice for notification preference
OS: System design fundamentals review: latency numbers, back-of-envel
Networks: Latency budget breakdown: DNS + TCP + TLS + server + DB for a typ
Behavioral STAR: Prepare answers for 'Why are you leaving?' and 'Why this company?
Submit 8 job application(s)
Recruiter outreach: 5 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 30 — July 30, 2026 | Peak | Connect 5yr MERN experience to interview narratives with metrics.

Daily Checklist
Morning LeetCode Block 1 (34 problems)
LeetCode #226: Invert Binary Tree
LeetCode #234: Palindrome Linked List
LeetCode #242: Valid Anagram
LeetCode #252: Meeting Rooms
LeetCode #390: Elimination Game
... +29 more problems (see Section 3)
Node.js: Node production war stories prep: scaling MERN apps, lessons lear
Node exercise: Write architecture decision record (ADR) for choosing MongoDB vs
Java: Spring Boot production readiness: graceful shutdown, connection l
Java exercise: Configure graceful shutdown, request timeout, and liveness/readin
SQL: SQL system design integration: choosing SQL vs NoSQL for hybrid a
SQL problem 1: Design polyglot persistence for e-commerce catalog (SQL
SQL problem 2: Write migration plan from single PostgreSQL to read rep

Daily Checklist
SQL problem 3: Model idempotency keys table for payment retry safety.
MongoDB: MongoDB sharding: shard key selection, chunk migration, and balan
Networks: DDoS mitigation: rate limiting, WAF, anycast scrubbing, and edge
Behavioral STAR: Tell me about your biggest professional achievement with quantifi
Submit 8 job application(s)
Recruiter outreach: 5 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 31 — July 31, 2026 | Peak | Netflix SD + full-stack rapid review — simulate interview pacing.

Daily Checklist
Morning LeetCode Block 1 (35 problems)
LeetCode #543: Diameter of Binary Tree
LeetCode #14: Longest Common Prefix
LeetCode #26: Remove Duplicates from Sorted Array
LeetCode #35: Search Insert Position
LeetCode #108: Convert Sorted Array to Binary Search Tree
... +30 more problems (see Section 3)
Node.js: Full Node backend mock review: synthesize event loop, Express, au
Node exercise: Timed 45-min build: REST API with auth, Redis cache, and BullMQ b
Java: Full Spring Boot mock review: REST + JPA + Security + testing sta
Java exercise: Timed: add security + validation + one integration test to existi
SQL: FAANG SQL difficulty practice: 3 hard problems
SQL problem 1: Trips and users — consecutive days problem.
SQL problem 2: Market analysis — window functions with multiple partit
SQL problem 3: Tree traversal in SQL — employee hierarchy depth.
OS: Operating system rapid review: scheduling, memory, deadlocks, syn
System Design: YouTube
Behavioral STAR: Practice salary negotiation talking points and competing offer fr
Submit 9 job application(s)
Recruiter outreach: 5 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 32 — August 1, 2026 | Peak | Crypto and security signing — common in payment and API gateway discussions.

Daily Checklist
Morning LeetCode Block 1 (36 problems)
LeetCode #110: Balanced Binary Tree
LeetCode #125: Valid Palindrome
LeetCode #144: Binary Tree Preorder Traversal
LeetCode #160: Intersection of Two Linked Lists
LeetCode #278: First Bad Version
... +31 more problems (see Section 3)
Node.js: Low-level Node: Buffer, crypto module, streams performance, and b
Node exercise: Implement HMAC-SHA256 request signing middleware for API authenti
Java: Java Cryptography: MessageDigest, Mac, KeyGenerator, and secure c
Java exercise: Implement HMAC verification utility in Java mirroring Node middle
SQL: Database security: row-level security, encryption at rest, and au
SQL problem 1: Implement PostgreSQL row-level security for multi-tenan
SQL problem 2: Design audit log table with tamper-evident constraints.

Daily Checklist
SQL problem 3: Query encrypted column strategy tradeoffs (app-level vs
Networks: Zero-trust networking principles and mTLS between microservices
Behavioral STAR: Describe a time you failed publicly. How did you recover credibil
Submit 9 job application(s)
Recruiter outreach: 5 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 33 — August 2, 2026 | Peak | Coding implementation day — LRU and Uber design.

Daily Checklist
Morning LeetCode Block 1 (37 problems)
LeetCode #283: Move Zeroes
LeetCode #496: Next Greater Element I
LeetCode #572: Subtree of Another Tree
LeetCode #680: Valid Palindrome II
LeetCode #746: Min Cost Climbing Stairs
... +32 more problems (see Section 3)
Node.js: Interview coding in Node: parse JSON streams, implement LRU cache
Node exercise: Implement LRU cache from scratch with O(1) get/put using Map + do
Java: Interview coding in Java: LRU cache, hash map design, and stream-
Java exercise: Implement LRU cache in Java using LinkedHashMap access-order mode
SQL: SQL speed run: 5 medium problems in 60 minutes
SQL problem 1: Customers who never order vs customers who order every
SQL problem 2: Product recommendation — co-purchase analysis.
SQL problem 3: Server utilization — sessions overlap counting.
MongoDB: MongoDB interview questions rapid review: sharding, replication,
OS: Process synchronization interview questions: mutex, semaphore, de
System Design: Netflix
Behavioral STAR: Answer 'What is your greatest weakness?' with genuine growth stor
Submit 9 job application(s)
Recruiter outreach: 5 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 34 — August 3, 2026 | Peak | Translate real experience into compelling interview stories.

Daily Checklist
Morning LeetCode Block 1 (38 problems)
LeetCode #876: Middle of the Linked List
LeetCode #28: Find the Index of the First Occurrence
LeetCode #88: Merge Sorted Array
LeetCode #100: Same Tree
LeetCode #101: Symmetric Tree
... +33 more problems (see Section 3)
Node.js: Behavioral + technical combo: explain past projects as system des
Node exercise: Prepare 3 project deep-dives mapping to SD components: API, DB, c
Java: Java/Spring project story prep: learning Spring Boot project fram
Java exercise: Document a Spring Boot portfolio project with README sections: ar
SQL: Review weakest SQL patterns identified during mocks
SQL problem 1: Re-do missed mock SQL problems without looking at solut

Daily Checklist
SQL problem 2: Window function drill: 3 variations on ranking problems
SQL problem 3: Complex JOIN drill: 4-table query for reporting.
Networks: Network troubleshooting methodology: ping, traceroute, tcpdump, a
Behavioral STAR: Prepare questions to ask interviewer — 10 thoughtful questions fo
Submit 10 job application(s)
Recruiter outreach: 5 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 35 — August 4, 2026 | Peak | Week 5 final mock — treat as real FAANG panel; Food Delivery SD in morning.

Daily Checklist
Morning LeetCode Block 1 (38 problems)
LeetCode #111: Minimum Depth of Binary Tree
LeetCode #136: Single Number
LeetCode #191: Number of 1 Bits
LeetCode #219: Contains Duplicate II
LeetCode #392: Is Subsequence
... +33 more problems (see Section 3)
Node.js: Node.js final review: 50 rapid-fire conceptual questions self-dri
Node exercise: Complete 50-question Node checklist; flag any answer taking >30 s
Java: Java/Spring final review: 50 rapid-fire conceptual questions self
Java exercise: Complete 50-question Java/Spring checklist with same <30 second t
SQL: SQL final review: 20 must-know patterns checklist
SQL problem 1: Top N per group — window function template.
SQL problem 2: Date spine generation — recursive CTE template.
SQL problem 3: Duplicate detection and removal template.
OS: OS final review: 15 core concepts checklist
Networks: Networking final review: 15 core concepts checklist
System Design: Uber
Behavioral STAR: Full behavioral mock: 8 questions in 45 minutes with timed respon
Submit 10 job application(s)
Recruiter outreach: 5 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today
Mock interview: Week 5 Final Mock (FAANG-style panel simulation)

Day 36 — August 5, 2026 | Final Sprint | Final Sprint — fix mock gaps only, no new topics.

Daily Checklist
Morning LeetCode Block 1 (38 problems)
LeetCode #703: Kth Largest in Stream
LeetCode #27: Remove Element
LeetCode #69: Sqrt(x)
LeetCode #225: Implement Stack using Queues
LeetCode #232: Implement Queue using Stacks
... +33 more problems (see Section 3)
Node.js: Light Node review: focus only on flagged weak areas from mock
Node exercise: Re-do one coding exercise from weakest Node area; explain aloud a
Java: Light Java review: focus only on flagged weak areas from mock
Java exercise: Re-do one Spring Boot exercise from mock gaps; record 5-min expla
SQL: Light SQL: redo 3 missed problems only

Daily Checklist
SQL problem 1: Reattempt #1 missed SQL mock problem.
SQL problem 2: Reattempt #2 missed SQL mock problem.
SQL problem 3: Reattempt #3 missed SQL mock problem.
MongoDB: MongoDB light review: only weak topics from prior mocks
Behavioral STAR: Rest and rehearse top 3 STAR stories — muscle memory, not new con
Submit 8 job application(s)
Recruiter outreach: 4 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 37 — August 6, 2026 | Final Sprint | Teach-back method — if you can teach it, interview answers flow naturally.

Daily Checklist
Morning LeetCode Block 1 (38 problems)
LeetCode #344: Reverse String
LeetCode #346: Moving Average from Data Stream
LeetCode #448: Find Disappeared Numbers
LeetCode #485: Max Consecutive Ones
LeetCode #530: Minimum Absolute Difference in BST
... +33 more problems (see Section 3)
Node.js: Node confidence builder: teach-back session — explain core concep
Node exercise: 30-min teach-back: record yourself explaining Node event loop, Ex
Java: Java confidence builder: teach-back Spring Boot request lifecycle
Java exercise: 30-min teach-back: Spring DI, REST controller, JPA repository flo
SQL: SQL confidence: verbalize approach before writing query for 3 cla
SQL problem 1: Second highest salary — verbalize then write.
SQL problem 2: Consecutive sessions — verbalize then write.
SQL problem 3: Cumulative sum — verbalize then write.
OS: OS top 10 rapid recall flashcards
Networks: Network top 10 rapid recall flashcards
System Design: Food Delivery
Behavioral STAR: Review company-specific talking points for top 5 target companies
Submit 7 job application(s)
Recruiter outreach: 3 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 38 — August 7, 2026 | Final Sprint | Rest and recall — protect sleep; cognitive freshness beats cramming.

Daily Checklist
Morning LeetCode Block 1 (38 problems)
LeetCode #704: Binary Search
LeetCode #724: Find Pivot Index
LeetCode #733: Flood Fill
LeetCode #860: Lemonade Change
LeetCode #922: Sort Array By Parity
... +33 more problems (see Section 3)
Node.js: Rest day active recall: Node flashcards only, no new coding
Node exercise: Browse Node.js docs for 30 min on one weak API; close docs and wr
Java: Rest day active recall: Java flashcards only
Java exercise: Browse Spring docs for 30 min on weakest area; write minimal exam
SQL: Rest day: one SQL problem timed at interview pace
SQL problem 1: Single timed SQL problem — full interview simulation.

Daily Checklist
SQL problem 2: Review solution and optimize if time permits.
SQL problem 3: Verbalize indexing strategy for the query.
Behavioral STAR: Light behavioral review — read STAR stories once, no rewriting.
Submit 5 job application(s)
Recruiter outreach: 2 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Day 39 — August 8, 2026 | Final Sprint | Day before interviews — rest, light review, logistics, confidence.

Daily Checklist
Morning LeetCode Block 1 (38 problems)
LeetCode #933: Number of Recent Calls
LeetCode #977: Squares of Sorted Array
LeetCode #1046: Last Stone Weight
LeetCode #13: Roman to Integer
LeetCode #157: Read N Characters Given Read4
... +33 more problems (see Section 3)
Node.js: Interview day prep: Node mental model one-page cheat sheet review
Node exercise: Write one-page Node cheat sheet from memory; compare to notes; sl
Java: Interview day prep: Java/Spring one-page cheat sheet review
Java exercise: Write one-page Spring Boot cheat sheet from memory; compare to no
SQL: Interview day prep: SQL pattern templates one-page review
SQL problem 1: Review window function template card.
SQL problem 2: Review join + group by template card.
SQL problem 3: Review CTE recursive template card.
MongoDB: MongoDB one-page cheat sheet: CRUD, aggregation, indexes, replica
OS: OS one-liner cheat sheet: 10 concepts
Networks: Network one-liner cheat sheet: 10 concepts
Behavioral STAR: Prepare opening 'Tell me about yourself' 90-second script and tom
Submit 3 job application(s)
Recruiter outreach: 1 message(s)
Evening spaced repetition review (30 min)
Update progress tracker (Section 16)
Journal: top learning + top gap today

Appendix B: Company Application Tracker

Track every application, referral, and follow-up. Update status within 24 hours of any change.

Company	Applied	Referral	Status	Next Action
Google	■	■	Not Applied	Research + tailor resume
Amazon	■	■	Not Applied	Research + tailor resume
Microsoft	■	■	Not Applied	Research + tailor resume
Meta	■	■	Not Applied	Research + tailor resume
Apple	■	■	Not Applied	Research + tailor resume
Netflix	■	■	Not Applied	Research + tailor resume
Flipkart	■	■	Not Applied	Research + tailor resume
Swiggy	■	■	Not Applied	Research + tailor resume
Zomato	■	■	Not Applied	Research + tailor resume
Razorpay	■	■	Not Applied	Research + tailor resume
PhonePe	■	■	Not Applied	Research + tailor resume
Paytm	■	■	Not Applied	Research + tailor resume
CRED	■	■	Not Applied	Research + tailor resume
Meesho	■	■	Not Applied	Research + tailor resume
ShareChat	■	■	Not Applied	Research + tailor resume
Freshworks	■	■	Not Applied	Research + tailor resume
Zoho	■	■	Not Applied	Research + tailor resume
Atlassian	■	■	Not Applied	Research + tailor resume
Uber	■	■	Not Applied	Research + tailor resume
Goldman Sachs	■	■	Not Applied	Research + tailor resume
JPMorgan Chase	■	■	Not Applied	Research + tailor resume
Deutsche Bank	■	■	Not Applied	Research + tailor resume
Barclays	■	■	Not Applied	Research + tailor resume
Walmart Global Tech	■	■	Not Applied	Research + tailor resume
Target	■	■	Not Applied	Research + tailor resume
Adobe	■	■	Not Applied	Research + tailor resume
Salesforce	■	■	Not Applied	Research + tailor resume
Intuit	■	■	Not Applied	Research + tailor resume
LinkedIn	■	■	Not Applied	Research + tailor resume
Twitter/X	■	■	Not Applied	Research + tailor resume
Stripe	■	■	Not Applied	Research + tailor resume
Shopify	■	■	Not Applied	Research + tailor resume
MongoDB Inc	■	■	Not Applied	Research + tailor resume
Postman	■	■	Not Applied	Research + tailor resume
RazorpayX	■	■	Not Applied	Research + tailor resume
Groww	■	■	Not Applied	Research + tailor resume
Zerodha	■	■	Not Applied	Research + tailor resume
Dream11	■	■	Not Applied	Research + tailor resume
Ola	■	■	Not Applied	Research + tailor resume
Info Edge (Naukri)	■	■	Not Applied	Research + tailor resume

Status Legend

Status	Meaning
Not Applied	On target list, not yet submitted
Applied	Application submitted, awaiting response
Phone Screen	Recruiter or HM initial call scheduled/completed
Technical	Coding or technical round in progress
Onsite	Virtual onsite or panel scheduled
Offer	Offer received — enter negotiation (Section 17)
Rejected	Log reason; revisit in 6 months if company still target
Withdrawn	Removed from pipeline intentionally

Appendix C: Interview Scorecards

Use these scorecards after every mock and real interview. Rate 1–10 per criterion. Target average 7+ before real Tier 1 interviews.

DSA / Coding

Criterion	Score (1–10)	Notes
Problem understanding and clarifying questions	___	
Approach explanation before coding	___	
Correctness of algorithm and edge cases	___	
Time and space complexity analysis	___	
Code quality — naming, structure, readability	___	
Testing with examples and dry run	___	
Communication throughout — thought process audible	___	
Speed — finished within time limit	___	
TOTAL / Average	___ / 10	

Node.js Backend

Criterion	Score (1–10)	Notes
Event loop and async model accuracy	___	
Express/middleware architecture knowledge	___	
Error handling and production patterns	___	
Authentication and security awareness	___	
Database integration (MongoDB/SQL)	___	
Caching, queues, and scaling strategies	___	
Testing and debugging approach	___	
Real project examples with metrics	___	
TOTAL / Average	___ / 10	

Java / Spring Boot

Criterion	Score (1–10)	Notes
JVM and core Java fundamentals	___	
Spring IoC and dependency injection	___	
REST API design and validation	___	
JPA/Hibernate and database mapping	___	
Spring Security and JWT implementation	___	
Transaction management understanding	___	
Testing strategy (unit, integration)	___	
Project demo readiness and code walkthrough	___	
TOTAL / Average	___ / 10	

System Design

Criterion	Score (1–10)	Notes
Requirements clarification and scope definition	___	
High-level architecture diagram	___	
API design and data model	___	
Scaling strategy with justification	___	
Caching, queues, and async patterns	___	
Tradeoff analysis — explicit pros/cons	___	
Back-of-envelope calculations	___	
Failure modes, monitoring, and operational concerns	___	
TOTAL / Average	___ / 10	

Behavioral

Criterion	Score (1–10)	Notes
STAR structure adherence	___	
Specific metrics and quantified results	___	
Clear individual contribution (uses 'I')	___	
Relevance to question asked	___	
Concise delivery (2–3 minutes)	___	
Authenticity and self-awareness	___	
Leadership and collaboration examples	___	
Strong closing reflection or learning	___	
TOTAL / Average	___ / 10	